Athleticore.AI

run.py

```
from flask import Flask, render_template
from app.api.auth_routes import register_auth_routes
from app.api.tdee_routes import register_tdee_routes
from app.api.food_feed_routes import register_food_feed_routes
from app.api.calories_routes import register_calories_routes
from app.api.workout_routes import register_workout_routes
from app.api.chat_routes import register_chat_routes
from app.api.settings_routes import register_settings_routes
from app.api.dashboard_routes import register_dashboard_routes



app = Flask(__name__, template_folder="app/templates",
static_folder="app/static")



@app.route("/")
def home():
    return "athleticore running"

@app.route("/login")
def login_page():
    return render_template("login.html")

@app.route("/register")
def register_page():
    return render_template("register.html")

@app.route("/dashboard")
def dashboard_page():
    return render_template("dashboard.html")

@app.route("/tdee")
def tdee_page():
    return render_template("tdee.html")
```

```python
@app.route("/calories")
def calories_page():
    return render_template("calories.html")

@app.route("/workout")
def workout_page():
    return render_template("workout.html")

@app.route("/coach")
def coach_page():
    return render_template("coach.html")

@app.route("/settings")
def settings_page():
    return render_template("settings.html")

@app.route("/create-log")
def create_log_page():
    return render_template("create_log.html")

@app.route("/forgot-password")
def forgot_password_page():
    return render_template("forgot_password.html")



# Register routes from other files
register_auth_routes(app)
register_tdee_routes(app)
register_food_feed_routes(app)
register_calories_routes(app)
register_workout_routes(app)
register_chat_routes(app)
register_settings_routes(app)
register_dashboard_routes(app)



if __name__ == "__main__":
    app.run(debug=True)
```

Requirements.txt

```
blinker==1.9.0
click==8.3.1
colorama==0.4.6
Flask==3.1.2
groq==0.36.0
itsdangerous==2.2.0
Jinja2==3.1.6
MarkupSafe==3.0.3
Werkzeug==3.1.3
```

.gitignore

```
venv/
__pycache__/
*.pyc
*.db
.env
```

Auth.routes

```
from flask import request, jsonify
from app.services.auth_manager import Authentication_manager


def register_auth_routes(app):
    @app.route("/api/auth/register", methods=["POST"])
    def register():
        data = request.get_json() or {}

        username = data.get("username")
        email = data.get("email")
        password = data.get("password")
        age= data.get("age")
        gender=data.get("gender")
        height=data.get("height")
        weight=data.get("weight")

        if not username or not email or not password or not age or not
gender or not height or not weight:
```

```
            return jsonify({"error": "fill all requirement"}), 400

        user = Authentication_manager.register_user(username, email,
password, age, gender, height, weight)

        if user is None:
            return jsonify({"error": "Username or email already taken"}),
409

        return jsonify({
            "message": "User registered successfully",
            "user": {
                "id": user.id,
                "username": user.username,
                "email": user.email,
                "age":user.age,
                "gender":gender,
                "height":height,
                "weight":weight
            }
        }), 201

    @app.route("/api/auth/login", methods=["POST"])
    def login():
        data = request.get_json() or {}

        identifier = data.get("identifier")
        password = data.get("password")

        if not identifier or not password:
            return jsonify({"error": "identifier and password are
required"}), 400

        user = Authentication_manager.login_user(identifier, password)

        if user is None:
            return jsonify({"error": "Invalid username/email or
password"}), 401

        return jsonify({
```

```
            "message": "Login successful",
            "user": {
                "id": user.id,
                "username": user.username,
                "email": user.email
            }
        }), 200
    @app.route("/api/auth/forgot-password", methods=["POST"])
    def forgot_password():
        data = request.get_json() or {}

        email = data.get("email")

        if not email:
            return jsonify({"error": "email is required"}), 400

        user = Authentication_manager.forget_password(email)

        if user is None:
            return jsonify({"error": "No user found with this email"}),
404

        return jsonify({
            "message": "A temporary password has been set. Please check
your email."
        }), 200
```

Calorie_routes.py

```
from flask import request, jsonify
from app.services.calorie_tracker import Calorie_manager
from app.services.food_chatbot_client import process_food_entry
from app.services.tdee_service import TdeeService
from app.db import db_helper
from app.models.user import User
from datetime import datetime


def register_calories_routes(app):
    @app.route("/api/calories/chat", methods=["POST"])
```

```
    def calories_chat():
        """Handle chat messages for calorie logging with AI."""
        try:
            data = request.get_json() or {}

            message = data.get("message")
            username = data.get("username")
            chat_history = data.get("history", [])

            if not message or not username:
                return jsonify({"error": "message and username are
required"}), 400

            # Get user from database
            user_row = db_helper.fetch_one(
                "SELECT * FROM users WHERE username = ?",
                (username,)
            )
            if not user_row:
                return jsonify({"error": "User not found"}), 404

            user = User.from_row(user_row)

            # Use frontend chat history (simplified - no DB chat storage
for calories)
            final_history = chat_history if chat_history else []

            # Call AI chatbot to process food entry
            reply_text, nutrition_result = process_food_entry(
                message=message,
                chat_history=final_history
            )

            # Prepare response
            response_data = {"reply": reply_text}

            # If AI returned nutrition data, save to calorie_logs and
return it
            if nutrition_result:
                # Check if we have all required fields
```

```
                required_fields = ["food_name", "calories", "protein_g",
"carbs_g", "fat_g"]
                if all(key in nutrition_result for key in
required_fields):
                    try:
                        # Save to calorie_logs table
                        log_entry = Calorie_manager.add_log(
                            user_id=user.id,
                            description=nutrition_result["food_name"],
                            calories=nutrition_result["calories"],
                            protein_g=nutrition_result["protein_g"],
                            carbs_g=nutrition_result["carbs_g"],
                            fat_g=nutrition_result["fat_g"]
                        )

                        # Format response in the expected format
                        current_time = datetime.now().strftime("%I:%M %p")
                        response_data["food_data"] = {
                            "name": nutrition_result["food_name"],
                            "calories":
float(nutrition_result["calories"]),
                            "protein":
float(nutrition_result["protein_g"]),
                            "carbs": float(nutrition_result["carbs_g"]),
                            "fat": float(nutrition_result["fat_g"]),
                            "time": current_time,
                            "id": log_entry.id  # Include log ID for
potential future use
                        }
                    except Exception as e:
                        print(f"Error saving calorie log: {e}")
                        import traceback
                        traceback.print_exc()
                        # Still return reply even if save fails

            return jsonify(response_data), 200
        except Exception as e:
            print(f"Error in calories_chat: {e}")
            import traceback
            traceback.print_exc()
```

```
            return jsonify({"error": str(e)}), 500
    @app.route("/api/calories/log", methods=["POST"])
    def add_calorie_log():
        """Add a new calorie log entry."""
        data = request.get_json() or {}

        username = data.get("username")
        description = data.get("description")
        calories = data.get("calories")
        protein_g = data.get("protein_g")
        carbs_g = data.get("carbs_g")
        fat_g = data.get("fat_g")
        entry_date = data.get("entry_date")

        if not username:
            return jsonify({"error": "username is required"}), 400

        # Get user from database
        user_row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
        if not user_row:
            return jsonify({"error": "User not found"}), 404

        user = User.from_row(user_row)

        try:
            log_entry = Calorie_manager.add_log(
                user_id=user.id,
                description=description,
                calories=float(calories) if calories is not None else
None,
                protein_g=float(protein_g) if protein_g is not None else
None,
                carbs_g=float(carbs_g) if carbs_g is not None else None,
                fat_g=float(fat_g) if fat_g is not None else None,
                entry_date=entry_date
            )
```

```
            return jsonify({
                "message": "Calorie log added successfully",
                "log": log_entry.to_dict()
            }), 201
        except Exception as e:
            return jsonify({"error": str(e)}), 500

    @app.route("/api/calories/logs/<int:user_id>", methods=["GET"])
    def get_calorie_logs(user_id):
        """Get calorie logs for a user, optionally filtered by date."""
        entry_date = request.args.get("date")  # Optional date parameter

        try:
            if entry_date:
                logs = Calorie_manager.get_logs(user_id, entry_date)
            else:
                logs = Calorie_manager.get_logs(user_id)

            return jsonify({
                "logs": [log.to_dict() for log in logs]
            }), 200
        except Exception as e:
            return jsonify({"error": str(e)}), 500

    @app.route("/api/calories/logs/<int:user_id>/today", methods=["GET"])
    def get_today_logs(user_id):
        """Get today's calorie logs for a user, including total calories,
goal calories, and surplus."""
        try:
            logs = Calorie_manager.get_today_logs(user_id)

            # Calculate total calories consumed today
            total_calories =
Calorie_manager.get_today_total_calories(user_id)

            # Get user's goal calories from TDEE profile
            goal_calories = None
            tdee_profile = TdeeService.get_profile_by_user_id(user_id)
            if tdee_profile:
                goal_calories = tdee_profile.goal_calories
```

```
            # Calculate surplus: (Total Calories - Goal Calories)
            # If negative (under goal), show 0. If positive (over goal),
show the difference.
            surplus = 0.0
            if goal_calories is not None:
                difference = total_calories - goal_calories
                if difference > 0:
                    surplus = difference

            return jsonify({
                "logs": [log.to_dict() for log in logs],
                "total_calories": total_calories,
                "goal_calories": goal_calories,
                "surplus": surplus
            }), 200
        except Exception as e:
            return jsonify({"error": str(e)}), 500

    @app.route("/api/calories/log/<int:log_id>", methods=["GET"])
    def get_calorie_log(log_id):
        """Get a specific calorie log by ID."""
        log = Calorie_manager.get_log_by_id(log_id)

        if not log:
            return jsonify({"error": "Log entry not found"}), 404

        return jsonify({
            "log": log.to_dict()
        }), 200

    @app.route("/api/calories/log/<int:log_id>", methods=["PUT"])
    def update_calorie_log(log_id):
        """Update a calorie log entry."""
        data = request.get_json() or {}

        description = data.get("description")
        calories = data.get("calories")
        protein_g = data.get("protein_g")
        carbs_g = data.get("carbs_g")
```

```
        fat_g = data.get("fat_g")

        try:
            updated_log = Calorie_manager.update_log(
                log_id=log_id,
                description=description,
                calories=float(calories) if calories is not None else
None,
                protein_g=float(protein_g) if protein_g is not None else
None,
                carbs_g=float(carbs_g) if carbs_g is not None else None,
                fat_g=float(fat_g) if fat_g is not None else None,
            )

            if not updated_log:
                return jsonify({"error": "Log entry not found"}), 404

            return jsonify({
                "message": "Log updated successfully",
                "log": updated_log.to_dict()
            }), 200
        except Exception as e:
            return jsonify({"error": str(e)}), 500

    @app.route("/api/calories/log/<int:log_id>", methods=["DELETE"])
    def delete_calorie_log(log_id):
        """Delete a calorie log entry."""
        log = Calorie_manager.get_log_by_id(log_id)

        if not log:
            return jsonify({"error": "Log entry not found"}), 404

        Calorie_manager.delete_log(log_id)

        return jsonify({
            "message": "Log deleted successfully"
        }), 200
```

Chat_routes

```python
from flask import request, jsonify
from app.services.chat_manager import ChatManager


def register_chat_routes(app):
    @app.route("/api/chat/send", methods=["POST"])
    def send_chat_message():
        """Send a chat message and get AI response."""
        data = request.get_json() or {}

        user_id = data.get("user_id")
        message = data.get("message")

        if not user_id or not message:
            return jsonify({"error": "user_id and message are required"}),
400

        result = ChatManager.send_message(user_id, message)

        if result is None:
            return jsonify({"error": "Failed to send message"}), 500

        return jsonify({
            "message": "Message sent successfully",
            "user_message": result["user_message"],
            "ai_message": result["ai_message"]
        }), 200

    @app.route("/api/chat/history", methods=["GET"])
    def get_chat_history():
        """Get chat history for a user."""
        user_id = request.args.get("user_id", type=int)

        if not user_id:
            return jsonify({"error": "user_id is required as a query
parameter"}), 400

        messages = ChatManager.get_chat_history(user_id)

        return jsonify({
```

```
            "messages": [msg.to_dict() for msg in messages]
        }), 200

    @app.route("/api/chat/history", methods=["DELETE"])
    def delete_chat_history():
        """Delete all chat messages for a user."""
        data = request.get_json() or {}
        user_id = data.get("user_id")

        if not user_id:
            return jsonify({"error": "user_id is required"}), 400

        deleted = ChatManager.delete_chat_history(user_id)

        if not deleted:
            return jsonify({"error": "Failed to delete chat history"}),
500

        return jsonify({
            "message": "Chat history deleted successfully"
        }), 200
```

Dashboard_routes

```
from flask import request, jsonify
from app.services.dashboard_service import DashboardService


def register_dashboard_routes(app):
    @app.route("/api/dashboard", methods=["GET"])
    def get_dashboard_data():
        """Get dashboard data for the current user."""
        user_id = request.args.get("user_id", type=int)

        if not user_id:
            return jsonify({"error": "user_id is required as a query
parameter"}), 400

        try:
            dashboard = DashboardService.get_dashboard_data(user_id)
```

```
            return jsonify({
                "message": "Dashboard data retrieved successfully",
                **dashboard.to_dict()
            }), 200
        except Exception as e:
            print(f"Error getting dashboard data: {e}")
            return jsonify({"error": "An unexpected error occurred"}), 500
```

Food_feed_routes

```
from flask import request, jsonify
from app.services.food_feed_service import FoodFeedService
from app.services.food_chatbot_client import process_food_entry
from app.db import db_helper
from app.models.user import User


def register_food_feed_routes(app):
    @app.route("/api/food/chat", methods=["POST"])
    def food_chat():
        """Handle chat messages with the food nutrition AI."""
        data = request.get_json() or {}

        message = data.get("message")
        username = data.get("username")
        food_feed_id = data.get("food_feed_id")
        chat_history = data.get("history", [])

        if not message or not username:
            return jsonify({"error": "message and username are
required"}), 400

        if not food_feed_id:
            return jsonify({"error": "food_feed_id is required"}), 400

        # Get user from database
        user_row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
```

```
        if not user_row:
            return jsonify({"error": "User not found"}), 404

        user = User.from_row(user_row)

        # Verify food_feed_id belongs to this user
        feed_entry = FoodFeedService.get_feed_by_id(food_feed_id)
        if not feed_entry or feed_entry.user_id != user.id:
            return jsonify({"error": "Food entry not found"}), 404

        # Load previous chat history from database
        db_chat_history = FoodFeedService.get_chat_history(food_feed_id)
        db_history_formatted = [
            {"role": chat.role, "content": chat.content}
            for chat in db_chat_history
        ]

        # Use frontend history if provided (current session), otherwise
use DB history
        if chat_history:
            final_history = chat_history
        else:
            final_history = db_history_formatted

        # Call AI chatbot
        reply_text, nutrition_result = process_food_entry(
            message=message,
            chat_history=final_history
        )

        # Save user message to database
        FoodFeedService.save_chat_message(food_feed_id, "user", message)

        # Save AI response to database
        FoodFeedService.save_chat_message(food_feed_id, "assistant",
reply_text)

        # If AI returned nutrition data, update the food feed entry
        if nutrition_result and nutrition_result.get("ready_to_save"):
            FoodFeedService.update_food_card(
```

```
                feed_id=food_feed_id,
                food_name=nutrition_result["food_name"],
                calories=nutrition_result["calories"],
                protein_g=nutrition_result["protein_g"],
                carbs_g=nutrition_result["carbs_g"],
                fat_g=nutrition_result["fat_g"]
            )

        # Prepare response
        response_data = {"reply": reply_text}
        if nutrition_result:
            response_data["nutrition_result"] = nutrition_result

        return jsonify(response_data), 200

    @app.route("/api/food/entry", methods=["POST"])
    def create_food_entry():
        """Create a new food entry."""
        data = request.get_json() or {}

        username = data.get("username")
        content = data.get("content")
        entry_date = data.get("entry_date")

        if not username or not content:
            return jsonify({"error": "username and content are
required"}), 400

        # Get user from database
        user_row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
        if not user_row:
            return jsonify({"error": "User not found"}), 404

        user = User.from_row(user_row)

        # Create food entry
        feed_entry = FoodFeedService.add_food_entry(
```

```
            user_id=user.id,
            content=content,
            entry_date=entry_date
        )

        return jsonify({
            "message": "Food entry created successfully",
            "feed_entry": feed_entry.to_dict()
        }), 201

    @app.route("/api/food/feed/<int:user_id>", methods=["GET"])
    def get_food_feed(user_id):
        """Get today's food feed for a user."""
        entry_date = request.args.get("date")  # Optional date parameter

        feed_entries = FoodFeedService.get_today_feed(user_id, entry_date)

        return jsonify({
            "feed": [entry.to_dict() for entry in feed_entries]
        }), 200

    @app.route("/api/food/entry/<int:feed_id>", methods=["GET"])
    def get_food_entry(feed_id):
        """Get a specific food entry with its chat history."""
        feed_entry = FoodFeedService.get_feed_by_id(feed_id)

        if not feed_entry:
            return jsonify({"error": "Food entry not found"}), 404

        # Get chat history for this entry
        chat_history = FoodFeedService.get_chat_history(feed_id)

        return jsonify({
            "feed_entry": feed_entry.to_dict(),
            "chat_history": [chat.to_dict() for chat in chat_history]
        }), 200

    @app.route("/api/food/entry/<int:feed_id>", methods=["DELETE"])
    def delete_food_entry(feed_id):
        """Delete a food entry."""
```

```
        feed_entry = FoodFeedService.get_feed_by_id(feed_id)

        if not feed_entry:
            return jsonify({"error": "Food entry not found"}), 404

        FoodFeedService.delete_feed_entry(feed_id)

        return jsonify({
            "message": "Food entry deleted successfully"
        }), 200
```

Settings_route

```
from flask import request, jsonify
from app.services.settings_service import SettingsService


def register_settings_routes(app):
    @app.route("/api/settings", methods=["GET"])
    def get_settings():
        """Get current user's settings."""
        # GET requests use query parameters, not JSON body
        user_id = request.args.get("user_id", type=int)

        if not user_id:
            return jsonify({"error": "user_id is required as a query
parameter"}), 400

        settings = SettingsService.get_user_settings(user_id)

        if settings is None:
            return jsonify({"error": "User not found"}), 404

        return jsonify({
            "message": "Settings retrieved successfully",
            "settings": settings
        }), 200

    @app.route("/api/settings", methods=["POST"])
    def update_settings():
```

```
        """Update user settings."""
        data = request.get_json() or {}

        user_id = data.get("user_id")
        username = data.get("username")
        email = data.get("email")
        password = data.get("password")
        confirm_password = data.get("confirm_password")
        age = data.get("age")
        height = data.get("height")
        weight = data.get("weight")

        if not user_id:
            return jsonify({"error": "user_id is required"}), 400

        # Basic validation
        if username is not None and not username.strip():
            return jsonify({"error": "username cannot be empty"}), 400

        if email is not None and not email.strip():
            return jsonify({"error": "email cannot be empty"}), 400

        if age is not None and (not isinstance(age, int) or age <= 0):
            return jsonify({"error": "age must be a positive integer"}),
400

        if height is not None and (not isinstance(height, int) or height
<= 0):
            return jsonify({"error": "height must be a positive
integer"}), 400

        if weight is not None and (not isinstance(weight, int) or weight
<= 0):
            return jsonify({"error": "weight must be a positive
integer"}), 400

        # Update settings
        updated_settings = SettingsService.update_user_settings(
            user_id=user_id,
            username=username,
```

```
            email=email,
            password=password,
            confirm_password=confirm_password,
            age=age,
            height=height,
            weight=weight
        )

        if updated_settings is None:
            return jsonify({"error": "Failed to update settings. User not
found or validation failed."}), 400

        return jsonify({
            "message": "Settings updated successfully",
            "settings": updated_settings
        }), 200
```

Tdee_routes

```
from flask import request, jsonify
from app.services.tdee_service import TdeeService
from app.services.chatgpt_client import chat_with_coach
from app.db import db_helper
from app.models.user import User


def register_tdee_routes(app):
    @app.route("/api/tdee/chat", methods=["POST"])
    def tdee_chat():
        """Handle chat messages with the TDEE coach."""
        data = request.get_json() or {}

        message = data.get("message")
        username = data.get("username")
        chat_history = data.get("history", [])
        stats = data.get("stats", {})

        if not message or not username:
            return jsonify({"error": "message and username are
required"}), 400
```

```
        # Get user from database to get their stats
        user_row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
        if not user_row:
            return jsonify({"error": "User not found"}), 404

        user = User.from_row(user_row)

        # Get stats from form or use user's database stats
        age = stats.get("age") or user.age
        gender = stats.get("gender") or user.gender
        height = stats.get("height_cm") or user.height
        weight = stats.get("weight_kg") or user.weight

        # Get or create TDEE profile to get profile_id for chat history
        profile_id = TdeeService.get_or_create_profile_id(user.id)
        profile = TdeeService.get_profile_by_user_id(user.id)

        # Load previous chat history from database
        db_chat_history = TdeeService.get_chat_history(profile.id)
        db_history_formatted = [
            {"role": chat.role, "content": chat.content}
            for chat in db_chat_history
        ]

        # Use frontend history if provided (current session), otherwise
use DB history
        # Frontend history is more up-to-date for the current session
        if chat_history:
            # Frontend is maintaining current session, use it
            final_history = chat_history
        else:
            # No frontend history, use DB history
            final_history = db_history_formatted

        # Call AI coach
        reply_text, tdee_result = chat_with_coach(
```

```
            user_name=user.username,
            age=age,
            gender=gender,
            height=height,
            weight=weight,
            message=message,
            chat_history=final_history
        )

        # Save user message to database
        TdeeService.save_chat_message(profile.id, "user", message)

        # Save AI response to database
        TdeeService.save_chat_message(profile.id, "assistant", reply_text)

        # Prepare response
        response_data = {"reply": reply_text}
        if tdee_result:
            response_data["tdee_result"] = tdee_result

        return jsonify(response_data), 200

    @app.route("/api/tdee/profile", methods=["POST"])
    def save_tdee_profile():
        """Save or update TDEE profile."""
        data = request.get_json() or {}

        user_id = data.get("user_id")
        activity_level = data.get("activity_level")
        tdee_value = data.get("tdee_value")
        goal_type = data.get("goal_type")
        goal_offset = data.get("goal_offset")
        goal_calories = data.get("goal_calories")

        # Validate all required fields
        missing_fields = []
        if not user_id:
            missing_fields.append("user_id")
        if not activity_level:
            missing_fields.append("activity_level")
```

```
        if tdee_value is None:
            missing_fields.append("tdee_value")
        if not goal_type:
            missing_fields.append("goal_type")
        if goal_offset is None:
            missing_fields.append("goal_offset")
        if goal_calories is None:
            missing_fields.append("goal_calories")

        if missing_fields:
            return jsonify({"error": f"Missing required fields: {',
'.join(missing_fields)}"}), 400

        try:
            profile = TdeeService.save_profile(
                user_id=user_id,
                activity_level=activity_level,
                tdee_value=float(tdee_value),
                goal_type=goal_type,
                goal_offset=int(goal_offset),
                goal_calories=float(goal_calories)
            )

            return jsonify({
                "message": "TDEE profile saved successfully",
                "profile": profile.to_dict()
            }), 200
        except Exception as e:
            return jsonify({"error": str(e)}), 500

    @app.route("/api/tdee/profile/<int:user_id>", methods=["GET"])
    def get_tdee_profile(user_id):
        """Get TDEE profile for a user."""
        profile = TdeeService.get_profile_by_user_id(user_id)

        if not profile:
            return jsonify({"error": "Profile not found"}), 404

        # Also get user stats for the frontend
        user_row = db_helper.fetch_one(
```

```
            "SELECT * FROM users WHERE id = ?",
            (user_id,)
        )
        if not user_row:
            return jsonify({"error": "User not found"}), 404

        user = User.from_row(user_row)

        return jsonify({
            "profile": {
                **profile.to_dict(),
                "age": user.age,
                "gender": user.gender,
                "height_cm": user.height,
                "weight_kg": user.weight
            }
        }), 200
```

Workout_routes

```
from flask import request, jsonify
from app.services.workout_service import WorkoutService
import traceback
import sqlite3


def register_workout_routes(app):
    @app.route("/api/workouts", methods=["GET"])
    def get_workouts():
        """Get all workouts for a user."""
        try:
            user_id = request.args.get("user_id", type=int)

            if not user_id:
                return jsonify({"error": "user_id is required as a query
parameter"}), 400

            workouts = WorkoutService.get_workouts_for_user(user_id)

            return jsonify({
```

```
                "workouts": workouts
            }), 200
        except Exception as e:
            print(f"Error getting workouts: {e}")
            traceback.print_exc()
            return jsonify({"error": "An unexpected error occurred"}), 500

    @app.route("/api/workouts", methods=["POST"])
    def create_workout():
        """Create a new workout with exercises."""
        data = request.get_json() or {}

        user_id = data.get("user_id")
        workout_name = data.get("workout_name")
        log_date = data.get("log_date")
        notes = data.get("notes")
        exercises = data.get("exercises", [])

        # Validate required fields
        missing_fields = []
        if not user_id:
            missing_fields.append("user_id")
        if not workout_name:
            missing_fields.append("workout_name")
        if not log_date:
            missing_fields.append("log_date")
        if not exercises:
            missing_fields.append("exercises")

        if missing_fields:
            return jsonify({"error": f"Missing required fields: {',
'.join(missing_fields)}"}), 400

        # Validate exercises structure
        if not isinstance(exercises, list) or len(exercises) == 0:
            return jsonify({"error": "At least one exercise is
required"}), 400

        for i, exercise in enumerate(exercises):
            if not isinstance(exercise, dict):
```

```
                return jsonify({"error": f"Exercise {i + 1}: must be an
object"}), 400
            exercise_name = exercise.get("exercise_name")
            if not exercise_name or not str(exercise_name).strip():
                return jsonify({"error": f"Exercise {i + 1}: exercise_name
is required"}), 400
            sets_value = exercise.get("sets")
            if sets_value is None or (isinstance(sets_value, (int, float))
and sets_value <= 0):
                return jsonify({"error": f"Exercise {i + 1}: sets is
required and must be greater than 0"}), 400
            reps_value = exercise.get("reps")
            if reps_value is None or (isinstance(reps_value, (int, float))
and reps_value <= 0):
                return jsonify({"error": f"Exercise {i + 1}: reps is
required and must be greater than 0"}), 400
            if exercise.get("order_index") is None:
                return jsonify({"error": f"Exercise {i + 1}: order_index
is required"}), 400

        try:
            # Convert user_id to int
            try:
                user_id_int = int(user_id)
            except (ValueError, TypeError):
                return jsonify({"error": "Invalid user_id format"}), 400

            result = WorkoutService.create_workout_for_users(
                user_id=user_id_int,
                workout_name=workout_name,
                log_date=log_date,
                notes=notes,
                exercises_data=exercises
            )

            return jsonify({
                "message": "Workout created successfully",
                **result
            }), 201
        except RuntimeError as e:
```

```
            error_msg = str(e)
            print(f"Error creating workout: {error_msg}")
            traceback.print_exc()
            return jsonify({"error": f"Failed to create workout:
{error_msg}"}), 500
        except ValueError as e:
            error_msg = str(e)
            print(f"Value error in create_workout: {error_msg}")
            traceback.print_exc()
            return jsonify({"error": f"Invalid input: {error_msg}"}), 400
        except sqlite3.OperationalError as e:
            error_msg = str(e)
            print(f"Database error in create_workout: {error_msg}")
            traceback.print_exc()
            if "no such table" in error_msg.lower():
                return jsonify({"error": "Database tables not initialized.
Please run: python app/db/init_db.py"}), 500
            return jsonify({"error": f"Database error: {error_msg}"}), 500
        except Exception as e:
            error_msg = str(e)
            error_type = type(e).__name__
            print(f"Unexpected error in create_workout ({error_type}):
{error_msg}")
            traceback.print_exc()
            # Return the actual error message to help with debugging
            return jsonify({"error": f"An unexpected error occurred:
{error_msg} (Type: {error_type})"}), 500

    @app.route("/api/workouts/<int:workout_id>", methods=["GET"])
    def get_workout(workout_id):
        """Get workout details with all exercises."""
        try:
            # GET requests should use query parameters, not JSON body
            user_id = request.args.get("user_id", type=int)

            if not user_id:
                return jsonify({"error": "user_id is required as a query
parameter"}), 400

            result = WorkoutService.get_workout_detail(
```

```
                workout_id=workout_id,
                user_id=user_id
            )

            if not result:
                return jsonify({"error": "Workout not found or
unauthorized"}), 404

            return jsonify(result), 200
        except Exception as e:
            print(f"Error getting workout detail: {e}")
            traceback.print_exc()
            return jsonify({"error": "An unexpected error occurred"}), 500

    @app.route("/api/workouts/<int:workout_id>", methods=["DELETE"])
    def delete_workout(workout_id):
        """Delete a workout and its exercises."""
        try:
            # DELETE can use query params or JSON body
            user_id = request.args.get("user_id", type=int)
            if not user_id:
                data = request.get_json() or {}
                user_id = data.get("user_id")

            if not user_id:
                return jsonify({"error": "user_id is required"}), 400

            deleted = WorkoutService.delete_workout(
                workout_id=workout_id,
                user_id=int(user_id)
            )

            if not deleted:
                return jsonify({"error": "Workout not found or
unauthorized"}), 404

            return jsonify({"message": "Workout deleted successfully"}),
200
        except Exception as e:
            print(f"Error deleting workout: {e}")
```

```
            traceback.print_exc()
            return jsonify({"error": "An unexpected error occurred"}), 500

    @app.route("/api/workouts/<int:workout_id>", methods=["PATCH"])
    def update_workout(workout_id):
        """Update a workout's fields and exercises (partial update)."""
        data = request.get_json() or {}
        user_id = data.get("user_id")
        workout_name = data.get("workout_name")
        log_date = data.get("log_date")
        notes = data.get("notes")
        exercises = data.get("exercises")

        if not user_id:
            return jsonify({"error": "user_id is required"}), 400

        # If exercises are provided, use the comprehensive update method
        if exercises is not None:
            try:
                result = WorkoutService.update_workout_with_exercises(
                    workout_id=workout_id,
                    user_id=int(user_id),
                    workout_name=workout_name,
                    notes=notes,
                    exercises=exercises
                )

                if not result:
                    return jsonify({"error": "Workout not found or
unauthorized"}), 404

                return jsonify({
                    "message": "Workout updated successfully",
                    **result
                }), 200
            except Exception as e:
                print(f"Error updating workout with exercises: {e}")
                traceback.print_exc()
                return jsonify({"error": f"An unexpected error occurred:
{str(e)}"}), 500
```

```
        # Otherwise, use the simple update method (for backward
compatibility)
        # At least one field must be provided for update
        if workout_name is None and log_date is None and notes is None:
            return jsonify({"error": "At least one field (workout_name,
log_date, notes) must be provided"}), 400

        try:
            result = WorkoutService.update_workout(
                workout_id=workout_id,
                user_id=int(user_id),
                workout_name=workout_name,
                log_date=log_date,
                notes=notes
            )

            if not result:
                return jsonify({"error": "Workout not found or
unauthorized"}), 404

            return jsonify({
                "message": "Workout updated successfully",
                "workout": result
            }), 200
        except Exception as e:
            print(f"Error updating workout: {e}")
            traceback.print_exc()
            return jsonify({"error": "An unexpected error occurred"}), 500
```

Db_helper

```
import sqlite3
from .init_db import DB_PATH
def get_connection():
    conn=sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA foreign_keys = ON;")
    return conn
def execute_query(query, params=()):
```

```
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(query, params)
    conn.commit()
    conn.close()
def fetch_one(query, params=()):

    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(query, params)
    row = cursor.fetchone()
    conn.close()
    return row

def fetch_all(query, params=()):
    conn = get_connection()
    cursor = conn.cursor()
    cursor.execute(query, params)
    rows = cursor.fetchall()
    conn.close()
    return rows
```

Init_db.py

```
import os
import sqlite3


BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_PATH = os.path.join(BASE_DIR, "athleticore.db")


def create_connection():
    conn = sqlite3.connect(DB_PATH)
    conn.execute("PRAGMA foreign_keys = ON;")
    return conn


def create_tables(conn):
    cursor = conn.cursor()

    cursor.execute(
```

```
        """ CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        username TEXT NOT NULL UNIQUE,
        email TEXT NOT NULL UNIQUE,
        password_hash TEXT NOT NULL,
        created_at TEXT NOT NULL,
        updated_at TEXT,
        age INTEGER NOT NULL,
        gender TEXT NOT NULL,
        height INTEGER NOT NULL,
        weight INTEGER NOT NULL
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS tdee_profile (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL UNIQUE,
        activity_level TEXT NOT NULL,
        tdee_value REAL NOT NULL,
        goal_type TEXT NOT NULL,
        goal_offset INTEGER NOT NULL,
        goal_calories REAL NOT NULL,
        created_at TEXT NOT NULL,
        updated_at TEXT,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS tdee_chat (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        tdee_profile_id INTEGER NOT NULL,
        role TEXT NOT NULL,
        content TEXT NOT NULL,
        created_at TEXT NOT NULL,
        FOREIGN KEY (tdee_profile_id) REFERENCES tdee_profile(id) ON
DELETE CASCADE
```

```
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS food_feed (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        content TEXT NOT NULL,
        food_name TEXT,
        calories REAL,
        protein_g REAL,
        carbs_g REAL,
        fat_g REAL,
        entry_date TEXT NOT NULL,
        created_at TEXT NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS food_chat (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        food_feed_id INTEGER NOT NULL,
        role TEXT NOT NULL,
        content TEXT NOT NULL,
        created_at TEXT NOT NULL,
        FOREIGN KEY (food_feed_id) REFERENCES food_feed(id) ON DELETE
CASCADE
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS calorie_logs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        entry_date TEXT NOT NULL,
        description TEXT,
```

```
        calories REAL,
        protein_g REAL,
        carbs_g REAL,
        fat_g REAL,
        created_at TEXT NOT NULL,
        is_deleted INTEGER DEFAULT 0,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS workouts (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        user_id INTEGER NOT NULL,
        workout_name TEXT NOT NULL,
        date TEXT NOT NULL,
        notes TEXT,
        created_at TEXT NOT NULL,
        FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
        );
        """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS chat_messages (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL,
    sender TEXT NOT NULL,          -- "user" or "ai"
    message TEXT NOT NULL,         -- the actual chat content
    created_at TEXT NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
            );
            """
    )

    cursor.execute(
        """ CREATE TABLE IF NOT EXISTS workout_exercises (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        workout_id INTEGER NOT NULL,
```

```
        exercise_name TEXT NOT NULL,
        sets INTEGER,
        reps INTEGER,
        weight_kg REAL,
        previous_weight REAL,
        order_index INTEGER NOT NULL,
        notes TEXT,
        FOREIGN KEY (workout_id) REFERENCES workouts(id) ON DELETE CASCADE
        );
        """
    )

    conn.commit()


def init_db():
    """Create the database file and all tables."""
    os.makedirs(BASE_DIR, exist_ok=True)
    conn = create_connection()
    create_tables(conn)
    conn.close()
    print(f"Database initialized at: {DB_PATH}")


if __name__ == "__main__":
    init_db()
```

Calorie_entry.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime, date, date

@dataclass
class CalorieLog:# name of the class is CalorieLog
    #data fields of the class
    id: Optional[int]
    user_id: int
    entry_date: str
    created_at: str
    # Optional fields mumken yethato aw la'
```

```
    description: Optional[str] = None
    calories: Optional[float] = None
    protein_g: Optional[float] = None
    carbs_g: Optional[float] = None
    fat_g: Optional[float] = None
    is_deleted: int = 0       # 0 = active, 1 = deleted
    @classmethod
    def from_row(cls, row: tuple) -> "CalorieLog":
        """
        method that creates a CalorieLog object from a database row
        """
        return cls(
            id=row[0],
            user_id=row[1],
            entry_date=row[2],
            created_at=row[8],  # created_at is at index 8 in DB
            description=row[3],
            calories=row[4],
            protein_g=row[5],
            carbs_g=row[6],
            fat_g=row[7],
            is_deleted=row[9],
        )
    def to_dict(self) -> dict:
       #retuns object of class by json format
        return {
            "id": self.id,
            "user_id": self.user_id,
            "entry_date": self.entry_date,
            "description": self.description,
            "calories": self.calories,
            "protein_g": self.protein_g,
            "carbs_g": self.carbs_g,
            "fat_g": self.fat_g,
            "created_at": self.created_at,
            "is_deleted": self.is_deleted,
        }
    @staticmethod
    #for timestamp
    def now_iso() -> str:
```

```
        return datetime.utcnow().isoformat()

    @staticmethod
    #for today's date
    def today_iso() -> str:
        return date.today().isoformat()
```

Chat_message.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

@dataclass
class ChatMessage:
    id: Optional[int]
    user_id: int
    sender: str
    message: str
    created_at: str

    @classmethod
    def from_row(cls, row: tuple) -> "ChatMessage":
        """
        row must come from:
        SELECT * FROM chat_messages
        with columns in this exact order:
        id, user_id, sender, message, created_at
        """
        return cls(
            id=row[0],
            user_id=row[1],
            sender=row[2],
            message=row[3],
            created_at=row[4],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "user_id": self.user_id,
```

```
            "sender": self.sender,
            "message": self.message,
            "created_at": self.created_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

dashboard.py

```
from dataclasses import dataclass
from typing import Optional


@dataclass
class Dashboard:
    current_goal: str
    daily_calorie_target: float
    calories_consumed_today: float
    calories_left: float
    calorie_progress_percent: float
    next_workout: Optional[str]

    def to_dict(self) -> dict:
        return {
            "current_goal": self.current_goal,
            "daily_calorie_target": self.daily_calorie_target,
            "calories_consumed_today": self.calories_consumed_today,
            "calories_left": self.calories_left,
            "calorie_progress_percent": self.calorie_progress_percent,
            "next_workout": self.next_workout
        }
```

Food_chat.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime
```

```
@dataclass
class FoodChat:
    id: Optional[int]
    food_feed_id: int
    role: str
    content: str
    created_at: str

    @classmethod
    def from_row(cls, row: tuple) -> "FoodChat":
        """
        row must come from:
        SELECT * FROM food_chat
        with columns in this exact order:
        id, food_feed_id, role, content, created_at
        """
        return cls(
            id=row[0],
            food_feed_id=row[1],
            role=row[2],
            content=row[3],
            created_at=row[4],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "food_feed_id": self.food_feed_id,
            "role": self.role,
            "content": self.content,
            "created_at": self.created_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

Food_feed.py

```
from dataclasses import dataclass
```

```python
from typing import Optional
from datetime import datetime

@dataclass
class FoodFeed:
    id: Optional[int]
    user_id: int
    content: str
    food_name: Optional[str]
    calories: Optional[float]
    protein_g: Optional[float]
    carbs_g: Optional[float]
    fat_g: Optional[float]
    entry_date: str
    created_at: str

    @classmethod
    def from_row(cls, row: tuple) -> "FoodFeed":
        """
        row must come from:
        SELECT * FROM food_feed
        with columns in this exact order:
        id, user_id, content, food_name, calories, protein_g, carbs_g,
fat_g, entry_date, created_at
        """
        return cls(
            id=row[0],
            user_id=row[1],
            content=row[2],
            food_name=row[3],
            calories=row[4],
            protein_g=row[5],
            carbs_g=row[6],
            fat_g=row[7],
            entry_date=row[8],
            created_at=row[9],
        )

    def to_dict(self) -> dict:
        return {
```

```
            "id": self.id,
            "user_id": self.user_id,
            "content": self.content,
            "food_name": self.food_name,
            "calories": self.calories,
            "protein_g": self.protein_g,
            "carbs_g": self.carbs_g,
            "fat_g": self.fat_g,
            "entry_date": self.entry_date,
            "created_at": self.created_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

Tdee_chat.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

@dataclass
class TdeeChat:
    id: Optional[int]
    tdee_profile_id: int
    role: str
    content: str
    created_at: str

    @classmethod
    def from_row(cls, row: tuple) -> "TdeeChat":
        """
        row must come from:
        SELECT * FROM tdee_chat
        with columns in this exact order:
        id, tdee_profile_id, role, content, created_at
        """
        return cls(
            id=row[0],
```

```
            tdee_profile_id=row[1],
            role=row[2],
            content=row[3],
            created_at=row[4],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "tdee_profile_id": self.tdee_profile_id,
            "role": self.role,
            "content": self.content,
            "created_at": self.created_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

Tdee_profile

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

@dataclass
class TdeeProfile:
    id: Optional[int]
    user_id: int
    activity_level: str
    tdee_value: float
    goal_type: str
    goal_offset: int
    goal_calories: float
    created_at: str
    updated_at: Optional[str]

    @classmethod
    def from_row(cls, row: tuple) -> "TdeeProfile":
        """
```

```
        row must come from:
        SELECT * FROM tdee_profile
        with columns in this exact order:
        id, user_id, activity_level, tdee_value, goal_type,
        goal_offset, goal_calories, created_at, updated_at
        """
        return cls(
            id=row[0],
            user_id=row[1],
            activity_level=row[2],
            tdee_value=row[3],
            goal_type=row[4],
            goal_offset=row[5],
            goal_calories=row[6],
            created_at=row[7],
            updated_at=row[8],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "user_id": self.user_id,
            "activity_level": self.activity_level,
            "tdee_value": self.tdee_value,
            "goal_type": self.goal_type,
            "goal_offset": self.goal_offset,
            "goal_calories": self.goal_calories,
            "created_at": self.created_at,
            "updated_at": self.updated_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

user.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime
```

```python
@dataclass
class User:
    id: Optional[int]
    username: str
    email: str
    password_hash: str
    created_at: str
    updated_at: Optional[str]
    age: int
    gender: str
    height: int
    weight: int

    @classmethod
    def from_row(cls, row: tuple) -> "User":
        """
        row must come from:
        SELECT * FROM users
        with columns in this exact order:
        id, username, email, password_hash, created_at,
        updated_at, age, gender, height, weight
        """
        return cls(
            id=row[0],
            username=row[1],
            email=row[2],
            password_hash=row[3],
            created_at=row[4],
            updated_at=row[5],
            age=row[6],
            gender=row[7],
            height=row[8],
            weight=row[9],
        )

    def to_dict(self) -> dict:
        # password_hash is intentionally not exposed here
        return {
            "id": self.id,
```

```
            "username": self.username,
            "email": self.email,
            "created_at": self.created_at,
            "updated_at": self.updated_at,
            "age": self.age,
            "gender": self.gender,
            "height": self.height,
            "weight": self.weight,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

Workout exercise.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

@dataclass
class WorkoutExercise:
    id: Optional[int]
    workout_id: int
    exercise_name: str
    sets: Optional[int]
    reps: Optional[int]
    weight_kg: Optional[float]
    previous_weight: Optional[float]
    order_index: int
    notes: Optional[str]

    @classmethod
    def from_row(cls, row: tuple) -> "WorkoutExercise":
        """
        row must come from:
        SELECT * FROM workout_exercises
        with columns in this exact order:
        id, workout_id, exercise_name, sets, reps, weight_kg,
        previous_weight, order_index, notes
```

```
        """
        return cls(
            id=row[0],
            workout_id=row[1],
            exercise_name=row[2],
            sets=row[3],
            reps=row[4],
            weight_kg=row[5],
            previous_weight=row[6],
            order_index=row[7],
            notes=row[8],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "workout_id": self.workout_id,
            "exercise_name": self.exercise_name,
            "sets": self.sets,
            "reps": self.reps,
            "weight_kg": self.weight_kg,
            "previous_weight": self.previous_weight,
            "order_index": self.order_index,
            "notes": self.notes,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

workout.py

```
from dataclasses import dataclass
from typing import Optional
from datetime import datetime

@dataclass
class Workout:
    id: Optional[int]
    user_id: int
```

```
    workout_name: str
    date: str
    notes: Optional[str]
    created_at: str

    @classmethod
    def from_row(cls, row: tuple) -> "Workout":
        """
        row must come from:
        SELECT * FROM workouts
        with columns in this exact order:
        id, user_id, workout_name, date, notes, created_at
        """
        return cls(
            id=row[0],
            user_id=row[1],
            workout_name=row[2],
            date=row[3],
            notes=row[4],
            created_at=row[5],
        )

    def to_dict(self) -> dict:
        return {
            "id": self.id,
            "user_id": self.user_id,
            "workout_name": self.workout_name,
            "date": self.date,
            "notes": self.notes,
            "created_at": self.created_at,
        }

    @staticmethod
    def now_iso() -> str:
        return datetime.utcnow().isoformat()
```

Ai_chat_client

```
import os
from groq import Groq
```

```
from typing import List, Dict

# Initialize Groq client
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
if not GROQ_API_KEY:
    raise ValueError("GROQ_API_KEY environment variable is not set")

client = Groq(api_key=GROQ_API_KEY)


def create_chat_system_prompt(user_name: str, age: int, gender: str,
height: int, weight: int) -> str:
    """Create system prompt for the general AI fitness coach chatbot."""
    return f"""You are an expert AI fitness coach and nutritionist for
Athleticore.AI. Your role is to provide personalized fitness, nutrition,
and wellness guidance to users.

User Information:
- Name: {user_name}
- Age: {age}
- Gender: {gender}
- Height: {height} cm
- Weight: {weight} kg

Your expertise includes:
- Workout programming and exercise form
- Nutrition and meal planning
- Weight management (cutting, bulking, maintaining)
- Muscle building and strength training
- Cardiovascular fitness
- Recovery and rest
- Supplement guidance
- General health and wellness

Guidelines:
- Be conversational, friendly, and encouraging
- Provide accurate, evidence-based advice
- Ask clarifying questions when needed
- Keep responses concise but informative
- Personalize advice based on the user's stats
```

```
- Be supportive and motivational
- If asked about topics outside fitness/nutrition, politely redirect to
fitness-related topics

Always maintain a professional yet approachable tone. Help users achieve
their fitness goals safely and effectively."""


def chat_with_ai_coach(
    user_name: str,
    age: int,
    gender: str,
    height: int,
    weight: int,
    message: str,
    chat_history: List[Dict[str, str]]
) -> str:
    """
    Send a message to the AI coach and get response.

    Args:
        user_name: User's name
        age: User's age
        gender: User's gender
        height: User's height in cm
        weight: User's weight in kg
        message: Current user message
        chat_history: List of previous messages in format [{"role":
"user"/"ai", "content": "..."}]

    Returns:
        AI response text
    """
    # Create system prompt with user stats
    system_prompt = create_chat_system_prompt(user_name, age, gender,
height, weight)

    # Build messages for the API
    messages = [{"role": "system", "content": system_prompt}]
```

```
    # Add chat history (convert to API format)
    for msg in chat_history:
        role = msg.get("role", "user")
        # Convert "ai" role to "assistant" for Groq API
        if role == "ai":
            role = "assistant"
        messages.append({
            "role": role,
            "content": msg.get("content", "")
        })

    # Add current user message
    messages.append({"role": "user", "content": message})

    try:
        # Call Groq API
        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=messages,
            temperature=0.7,
            max_tokens=1000
        )

        reply_text = response.choices[0].message.content
        return reply_text

    except Exception as e:
        print(f"Error calling Groq API: {e}")
        return f"I'm sorry, I encountered an error. Please try again.
Error: {str(e)}"
```

Auth_manager

```
from app.models.user import User
from app.db import db_helper
from app.services.email_service import send_email  # NEW
import hashlib
import secrets                                     # NEW
import string
```

```
class Authentication_manager:
    @staticmethod
    def hash_password(real_password):
        password_bytes = real_password.encode("utf-8")
        hash_password_obj = hashlib.sha256(password_bytes)
        hash_password_string = hash_password_obj.hexdigest()
        return hash_password_string

    @staticmethod
    def _generate_temp_password(length: int = 12) -> str:
        """Generate a random temporary password."""
        alphabet = string.ascii_letters + string.digits
        return "".join(secrets.choice(alphabet) for _ in range(length))

    @staticmethod
    def register_user(username, email, password, age, gender, height,
weight):
        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
        if row is not None:
            print("Username already taken")
            return None

        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE email = ?",
            (email,)
        )
        if row is not None:
            print("Email already taken")
            return None

        hashed_pass = Authentication_manager.hash_password(password)
        created_at = User.now_iso()

        db_helper.execute_query(
            "INSERT INTO users (username, email, password_hash,
created_at, age, gender, height, weight) VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
```

```
            (username, email, hashed_pass, created_at, age, gender, height
, weight)
        )

        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        )
        new_user = User.from_row(row)
        return new_user

    @staticmethod
    def login_user(username_or_email, password):
        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE username = ? OR email = ?",
            (username_or_email, username_or_email)
        )
        if row is None:
            print("User not found (wrong username or email).")
            return None

        user = User.from_row(row)
        entered_hash = Authentication_manager.hash_password(password)

        if entered_hash != user.password_hash:
            print("WRONG PASSWORD")
            return None

        return user

    @staticmethod
    def forget_password(email):
        # 1) Look up user by email in the database
        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE email = ?",
            (email,)
        )
        if row is None:
            print("No user found with this email.")
            return None
```

```
        # Same as before: build User from row (we need user.id)
        user = User.from_row(row)

        # 2) Generate a secure random temporary password
        temp_pass =
Authentication_manager._generate_temp_password(length=12)

        # 3) Hash the temporary password using your existing function
        temp_pass_hash = Authentication_manager.hash_password(temp_pass)

        # 4) Update password_hash in DB
        db_helper.execute_query(
            "UPDATE users SET password_hash = ? WHERE id = ?",
            (temp_pass_hash, user.id)
        )

        # 5) Send the temporary password by email
        subject = "Your Athleticore.AI Temporary Password"
        body = (
            "Hello,\n\n"
            "You requested a password reset for your Athleticore.AI
account.\n\n"
            f"Your temporary password is:\n\n"
            f"    {temp_pass}\n\n"
            "Please use this temporary password to log in, and then change
your password "
            "from the settings screen.\n\n"
            "If you did not request this, please ignore this email.\n"
        )

        # Use the email string passed into the function (no dependency on
user.email)
        email_sent = send_email(email, subject, body)

        if not email_sent:
            print("Failed to send temporary password email to", email)

        return user
```

Calorie_tracker.py

```python
from app.models.calorie_entry import CalorieLog
from app.db import db_helper
from typing import Optional, List


class Calorie_manager:
    @staticmethod
    #hanadeh men el route haolo add log with these parameters
    def add_log(
        user_id: int,
        description: Optional[str],
        calories: Optional[float],
        protein_g: Optional[float],
        carbs_g: Optional[float],
        fat_g: Optional[float],
        entry_date: Optional[str] = None,
    ) -> CalorieLog:
        # lesa baamel entry date lw log gededa
        if entry_date is None:
            entry_date = CalorieLog.today_iso()

        # 2) Generate created_at timestamp
        created_at = CalorieLog.now_iso()

        # 3) Insert into DB
        db_helper.execute_query(
            """
            INSERT INTO calorie_logs (
                user_id, entry_date, description,
                calories, protein_g, carbs_g, fat_g,
                created_at
            )
            VALUES (?, ?, ?, ?, ?, ?, ?, ?)
            """,
            (
                user_id,
                entry_date,
                description,
                calories,
```

```
                protein_g,
                carbs_g,
                fat_g,
                created_at,
            ),
        )

        # 4) Fetch the last inserted row for this user & date
        row = db_helper.fetch_one(
            """
            SELECT *
            FROM calorie_logs
            WHERE user_id = ? AND entry_date = ?
            ORDER BY id DESC
            LIMIT 1
            """,
            (user_id, entry_date),
        )

        # 5) Convert DB row -> CalorieLog object
        return CalorieLog.from_row(row)

    @staticmethod
    def get_logs(user_id: int, entry_date: Optional[str] = None) ->
List[CalorieLog]:
        """Get all calorie logs for a user, optionally filtered by
date."""
        if entry_date:
            rows = db_helper.fetch_all(
                """
                SELECT * FROM calorie_logs
                WHERE user_id = ? AND entry_date = ? AND is_deleted = 0
                ORDER BY created_at DESC
                """,
                (user_id, entry_date)
            )
        else:
            rows = db_helper.fetch_all(
                """
                SELECT * FROM calorie_logs
```

```
                WHERE user_id = ? AND is_deleted = 0
                ORDER BY entry_date DESC, created_at DESC
                """,
                (user_id,)
            )
        return [CalorieLog.from_row(row) for row in rows]

    @staticmethod
    def get_today_logs(user_id: int) -> List[CalorieLog]:
        """Get all calorie logs for today."""
        today = CalorieLog.today_iso()
        return Calorie_manager.get_logs(user_id, today)

    @staticmethod
    def get_today_total_calories(user_id: int) -> float:
        """Calculate total calories consumed today."""
        today = CalorieLog.today_iso()
        row = db_helper.fetch_one(
            """
            SELECT COALESCE(SUM(calories), 0) as total
            FROM calorie_logs
            WHERE user_id = ? AND entry_date = ? AND is_deleted = 0
            """,
            (user_id, today)
        )
        if row and row[0] is not None:
            return float(row[0])
        return 0.0

    @staticmethod
    def get_log_by_id(log_id: int) -> Optional[CalorieLog]:
        """Get a specific calorie log by ID."""
        row = db_helper.fetch_one(
            "SELECT * FROM calorie_logs WHERE id = ? AND is_deleted = 0",
            (log_id,)
        )
        if row is None:
            return None
        return CalorieLog.from_row(row)
```

```python
    @staticmethod
    def update_log(
        log_id: int,
        description: Optional[str] = None,
        calories: Optional[float] = None,
        protein_g: Optional[float] = None,
        carbs_g: Optional[float] = None,
        fat_g: Optional[float] = None,
    ) -> Optional[CalorieLog]:
        """Update a calorie log entry."""
        # Step 1: Check which fields the user wants to update
        # Only include fields that are not None (meaning user provided a
value)
        fields_to_update = {}

        if description is not None:
            fields_to_update["description"] = description
        if calories is not None:
            fields_to_update["calories"] = calories
        if protein_g is not None:
            fields_to_update["protein_g"] = protein_g
        if carbs_g is not None:
            fields_to_update["carbs_g"] = carbs_g
        if fat_g is not None:
            fields_to_update["fat_g"] = fat_g

        # Step 2: If user didn't provide any fields to update, just return
the current log
        if len(fields_to_update) == 0:
            return Calorie_manager.get_log_by_id(log_id)

        # Step 3: Build two lists - one for SQL parts, one for values
        sql_parts = []
        values = []

        # Step 4: Go through each field we want to update
        for field_name in fields_to_update:
            # Create SQL part like "description = ?"
            sql_part = field_name + " = ?"
            sql_parts.append(sql_part)
```

```
            # Get the actual value (like "Chicken" or 231.0)
            field_value = fields_to_update[field_name]
            values.append(field_value)

        # Step 5: Add the log_id at the end (needed for WHERE id = ?)
        values.append(log_id)

        # Step 6: Join all SQL parts with commas
        # If sql_parts = ["description = ?", "calories = ?"]
        # Then sql_set_clause = "description = ?, calories = ?"
        sql_set_clause = ", ".join(sql_parts)

        # Step 7: Build the complete SQL query
        # Start with "UPDATE calorie_logs SET "
        query_start = "UPDATE calorie_logs SET "
        # Add our SQL parts: "description = ?, calories = ?"
        query_middle = sql_set_clause
        # Add the WHERE part: " WHERE id = ? AND is_deleted = 0"
        query_end = " WHERE id = ? AND is_deleted = 0"
        # Combine all parts
        complete_query = query_start + query_middle + query_end

        # Step 8: Convert values list to tuple and execute query
        values_tuple = tuple(values)
        db_helper.execute_query(complete_query, values_tuple)

        # Step 9: Get and return the updated log
        updated_log = Calorie_manager.get_log_by_id(log_id)
        return updated_log

    @staticmethod
    def delete_log(log_id: int) -> bool:
        """Soft delete a calorie log (set is_deleted = 1)."""
        db_helper.execute_query(
            "UPDATE calorie_logs SET is_deleted = 1 WHERE id = ?",
            (log_id,)
        )
        return True
```

Cht_manager

```
from typing import List, Dict, Optional
from app.models.chat_message import ChatMessage
from app.models.user import User
from app.db import db_helper
from app.services.ai_chat_client import chat_with_ai_coach


class ChatManager:
    @staticmethod
    def save_message(user_id: int, sender: str, message: str) ->
ChatMessage:
        """
        Save a chat message to the database.

        Args:
            user_id: The ID of the user
            sender: "user" or "ai"
            message: The message content

        Returns:
            ChatMessage object
        """
        created_at = ChatMessage.now_iso()

        db_helper.execute_query(
            "INSERT INTO chat_messages (user_id, sender, message,
created_at) VALUES (?, ?, ?, ?)",
            (user_id, sender, message, created_at)
        )

        row = db_helper.fetch_one(
            "SELECT * FROM chat_messages WHERE user_id = ? AND sender = ?
AND message = ? AND created_at = ? ORDER BY id DESC LIMIT 1",
            (user_id, sender, message, created_at)
        )

        if row is None:
            print("Failed to save message")
            return None
```

```
        chat_message = ChatMessage.from_row(row)
        return chat_message

    @staticmethod
    def get_chat_history(user_id: int) -> List[ChatMessage]:
        """
        Get all chat messages for a user, ordered by creation time.

        Args:
            user_id: The ID of the user

        Returns:
            List of ChatMessage objects
        """
        rows = db_helper.fetch_all(
            "SELECT * FROM chat_messages WHERE user_id = ? ORDER BY
created_at ASC",
            (user_id,)
        )

        if rows is None:
            return []

        messages = [ChatMessage.from_row(row) for row in rows]
        return messages

    @staticmethod
    def get_user_info(user_id: int) -> Optional[Dict]:
        """
        Get user information needed for AI chat.

        Args:
            user_id: The ID of the user

        Returns:
            Dictionary with username, age, gender, height, weight, or None
if user not found
        """
        row = db_helper.fetch_one(
```

```
            "SELECT * FROM users WHERE id = ?",
            (user_id,)
        )

        if row is None:
            print("User not found")
            return None

        user = User.from_row(row)
        return {
            "username": user.username,
            "age": user.age,
            "gender": user.gender,
            "height": user.height,
            "weight": user.weight
        }

    @staticmethod
    def send_message(user_id: int, message: str) -> Dict:
        """
        Send a user message, get AI response, and save both to database.

        Args:
            user_id: The ID of the user
            message: The user's message

        Returns:
            Dictionary with user_message and ai_message ChatMessage
objects
        """
        # Get user info
        user_info = ChatManager.get_user_info(user_id)
        if user_info is None:
            print("User not found")
            return None

        # Get chat history
        chat_history = ChatManager.get_chat_history(user_id)
        history_formatted = [
            {"role": msg.sender, "content": msg.message}
```

```
            for msg in chat_history
        ]

        # Save user message first
        user_message_obj = ChatManager.save_message(user_id, "user",
message)
        if user_message_obj is None:
            print("Failed to save user message")
            return None

        # Call AI coach
        ai_response = chat_with_ai_coach(
            user_name=user_info["username"],
            age=user_info["age"],
            gender=user_info["gender"],
            height=user_info["height"],
            weight=user_info["weight"],
            message=message,
            chat_history=history_formatted
        )

        # Save AI response
        ai_message_obj = ChatManager.save_message(user_id, "ai",
ai_response)
        if ai_message_obj is None:
            print("Failed to save AI message")
            return None

        return {
            "user_message": user_message_obj.to_dict(),
            "ai_message": ai_message_obj.to_dict()
        }

    @staticmethod
    def delete_chat_history(user_id: int) -> bool:
        """
        Delete all chat messages for a user.

        Args:
            user_id: The ID of the user
```

```
        Returns:
            True if successful, False otherwise
        """
        db_helper.execute_query(
            "DELETE FROM chat_messages WHERE user_id = ?",
            (user_id,)
        )
        return True
```

Chat_gbtclient.py

```
import os
import json
import re
from groq import Groq
from typing import List, Dict, Optional

# Initialize Groq client
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
if not GROQ_API_KEY:
    raise ValueError("GROQ_API_KEY environment variable is not set")

client = Groq(api_key=GROQ_API_KEY)


def create_system_prompt(user_name: str, age: int, gender: str, height:
int, weight: int) -> str:
    """Create system prompt for the AI coach with user stats."""
    return f"""You are an expert AI fitness coach for Athleticore.AI. Your
role is to help users determine their fitness goals and calculate their
TDEE (Total Daily Energy Expenditure).

User Information:
- Name: {user_name}
- Age: {age}
- Gender: {gender}
- Height: {height} cm
- Weight: {weight} kg
```

```
Your tasks:
1. Help the user determine their fitness goal: "cut" (lose weight), "bulk" (gain weight), or "maintain" (maintain current weight)
2. Determine their activity level factor based on their lifestyle and exercise habits. Use standard activity levels: "sedentary", "lightly_active", "moderately_active", "very_active", or "extremely_active"
3. Calculate their TDEE (Total Daily Energy Expenditure) using your knowledge of fitness calculations
4. Calculate their goal calories based on their goal type and a goal offset (calorie deficit/surplus)

Important:
- Use your AI reasoning to calculate TDEE - do not use built-in formulas, but use your understanding of metabolism and energy expenditure
- Be conversational and helpful
- Ask clarifying questions about their activity level and goals
- When you have enough information, provide the values in a structured format

When you have determined the values, respond with a JSON object in this exact format:
{{
    "activity_level": "moderately_active",
    "tdee_value": 2500.0,
    "goal_type": "cut",
    "goal_offset": 500,
    "goal_calories": 2000.0,
    "ready_to_save": true
}}

Only include this JSON when you have all the information needed. Otherwise, continue the conversation naturally."""


def parse_tdee_result(response_text: str) -> Optional[Dict]:
    """Extract TDEE result from AI response if present."""
    # Try to find JSON in the response
    json_match = re.search(r'\{[^{}]*"activity_level"[^{}]*\}', response_text, re.DOTALL)
```

```
    if json_match:
        try:
            result = json.loads(json_match.group())
            # Validate required fields
            required_fields = ["activity_level", "tdee_value",
"goal_type", "goal_offset", "goal_calories"]
            if all(field in result for field in required_fields):
                result["ready_to_save"] = result.get("ready_to_save",
False)
                return result
        except json.JSONDecodeError:
            pass
    return None


def chat_with_coach(
    user_name: str,
    age: int,
    gender: str,
    height: int,
    weight: int,
    message: str,
    chat_history: List[Dict[str, str]]
) -> tuple[str, Optional[Dict]]:
    """
    Send a message to the AI coach and get response.
    Returns: (reply_text, tdee_result_dict or None)
    """
    # Create system prompt with user stats
    system_prompt = create_system_prompt(user_name, age, gender, height,
weight)

    # Build messages for the API
    messages = [{"role": "system", "content": system_prompt}]

    # Add chat history (convert to API format)
    for msg in chat_history:
        messages.append({
            "role": msg.get("role", "user"),
            "content": msg.get("content", "")
```

```
        })

    # Add current user message
    messages.append({"role": "user", "content": message})

    try:
        # Call Groq API
        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",  # Updated to current model
            messages=messages,
            temperature=0.7,
            max_tokens=1000
        )

        reply_text = response.choices[0].message.content

        # Try to parse TDEE result from response
        tdee_result = parse_tdee_result(reply_text)

        return reply_text, tdee_result

    except Exception as e:
        print(f"Error calling Groq API: {e}")
        return f"I'm sorry, I encountered an error. Please try again.
Error: {str(e)}", None
```

Dashboard_service.py

```
from typing import Optional
from datetime import datetime, date
from app.db import db_helper
from app.models.dashboard import Dashboard
from app.services.tdee_service import TdeeService
from app.services.calorie_tracker import Calorie_manager


class DashboardService:
    @staticmethod
    def get_dashboard_data(user_id: int) -> Dashboard:
        """
```

```
        Get dashboard data for a user.

        Args:
            user_id: The ID of the user

        Returns:
            Dictionary with current_goal, calories_left, next_workout,
etc.
        """
        # Get TDEE profile for goal and calorie target
        tdee_profile = TdeeService.get_profile_by_user_id(user_id)

        # Default values
        current_goal = "maintaining"
        daily_calorie_target = 2000.0
        goal_calories = 2000.0

        if tdee_profile:
            current_goal = tdee_profile.goal_type
            goal_calories = tdee_profile.goal_calories
            daily_calorie_target = goal_calories

        # Get calories consumed today
        calories_consumed_today =
Calorie_manager.get_today_total_calories(user_id)

        # Calculate calories left
        calories_left = max(0, daily_calorie_target -
calories_consumed_today)

        # Calculate progress percentage (0-100)
        if daily_calorie_target > 0:
            calorie_progress_percent = min(100, (calories_consumed_today /
daily_calorie_target) * 100)
        else:
            calorie_progress_percent = 0.0

        # Get next workout
        next_workout = DashboardService._get_next_workout(user_id)
        if next_workout is None:
```

```
            next_workout = "No upcoming workout"

        # Create and return Dashboard model
        return Dashboard(
            current_goal=current_goal,
            daily_calorie_target=daily_calorie_target,
            calories_consumed_today=calories_consumed_today,
            calories_left=calories_left,
            calorie_progress_percent=calorie_progress_percent,
            next_workout=next_workout
        )

    @staticmethod
    def _get_next_workout(user_id: int) -> Optional[str]:
        """
        Get the next scheduled workout for a user.
        First tries to find future workouts (including today),
        then falls back to the most recent past workout.

        Args:
            user_id: The ID of the user

        Returns:
            Formatted string like "Pull Day - Today" or None if no workout
exists
        """
        today = date.today().isoformat()

        # Get all workouts for this user to find the next one
        # We'll do the date comparison in Python to handle any date format
issues
        all_workouts = db_helper.fetch_all(
            """
            SELECT workout_name, date
            FROM workouts
            WHERE user_id = ?
            ORDER BY date ASC
            """,
            (user_id,)
        )
```

```
        if not all_workouts:
            print(f"No workouts found for user_id: {user_id}")
            return None

        print(f"Found {len(all_workouts)} workout(s) for user_id:
{user_id}")
        for w in all_workouts:
            print(f"  - {w[0]} on {w[1]}")

        # Find the next workout (including today or future)
        today_obj = date.today()
        next_workout_row = None

        for workout_row in all_workouts:
            workout_date_str = workout_row[1]
            try:
                # Parse the date string
                if 'T' in workout_date_str:
                    workout_date_obj =
datetime.fromisoformat(workout_date_str).date()
                else:
                    workout_date_obj =
datetime.fromisoformat(workout_date_str + 'T00:00:00').date()

                # If this workout is today or in the future, use it
                if workout_date_obj >= today_obj:
                    next_workout_row = workout_row
                    break
            except Exception as e:
                print(f"Error parsing workout date '{workout_date_str}':
{e}")
                continue

        # If no future workout found, use the most recent one (last in
list since sorted ASC)
        if next_workout_row is None:
            print(f"No future workouts found, using most recent past
workout")
```

```
            next_workout_row = all_workouts[-1]  # Last workout (most
recent)

        row = next_workout_row

        if row is None:
            return None

        workout_name = row[0]
        workout_date = row[1]

        # Format the date nicely
        try:
            # Handle both ISO date strings (YYYY-MM-DD) and datetime
strings
            if 'T' in workout_date:
                date_obj = datetime.fromisoformat(workout_date).date()
            else:
                date_obj = datetime.fromisoformat(workout_date +
'T00:00:00').date()

            today_obj = date.today()

            if date_obj == today_obj:
                return f"{workout_name} - Today"
            else:
                from datetime import timedelta
                tomorrow = today_obj + timedelta(days=1)
                if date_obj == tomorrow:
                    return f"{workout_name} - Tomorrow"
                elif date_obj > today_obj:
                    # Future date - Format as "Month Day" (e.g., "Jan 15")
                    return f"{workout_name} - {date_obj.strftime('%b
%d')}"
                else:
                    # Past date - Format as "Month Day" (e.g., "Jan 15")
                    return f"{workout_name} - {date_obj.strftime('%b
%d')}"
        except Exception as e:
            # If date parsing fails, just return the name
```

```
            print(f"Error parsing workout date '{workout_date}': {e}")
            return workout_name


# app/services/email_service.py

import os
import smtplib
from email.message import EmailMessage


# Read email configuration from environment variables
SMTP_HOST = "smtp-relay.brevo.com"
SMTP_PORT = 587        # default TLS port
SMTP_USER = "9c176f001@smtp-brevo.com"     # <-- replace
SMTP_PASS = "zBqN8hbaw7Ym2MWH"    # <-- replace
FROM_EMAIL = "ahmadmizo9@gmail.com"    # <-- the sender you verified

def send_email(to_email: str, subject: str, body: str) -> bool:
    """
    Sends a plain text email using SMTP.
    Returns True if the email was sent successfully, False otherwise.
    """

    # Ensure credentials exist
    if not SMTP_USER or not SMTP_PASS:
        print("❌ Email not configured correctly (missing SMTP_USER or
SMTP_PASS).")
        return False

    # Build the email
    msg = EmailMessage()
    msg["From"] = FROM_EMAIL
    msg["To"] = to_email
    msg["Subject"] = subject
    msg.set_content(body)

    try:
        # Connect to SMTP server
        with smtplib.SMTP(SMTP_HOST, SMTP_PORT) as server:
```

```
            server.starttls()  # secure connection
            server.login(SMTP_USER, SMTP_PASS)
            server.send_message(msg)

        print(f"📧 Email successfully sent to {to_email}")
        return True

    except Exception as e:
        print(f"❌ Failed to send email to {to_email}: {e}")
        return False

import os
import json
import re
from groq import Groq
from typing import Dict, Optional

# Initialize Groq client
GROQ_API_KEY = os.getenv("GROQ_API_KEY")
if not GROQ_API_KEY:
    raise ValueError("GROQ_API_KEY environment variable is not set")

client = Groq(api_key=GROQ_API_KEY)


def create_food_system_prompt() -> str:
    """Create system prompt for the food nutrition AI."""
    return """You are a helpful nutrition AI assistant for Athleticore.AI.
Your role is to help users log food and nutrition information.

COMMUNICATION STYLE:
- Be conversational and helpful, but concise (1-2 sentences max, typically
10-20 words)
- Avoid robotic one-word answers, but also avoid long paragraphs
- Be friendly and natural in your responses
- Examples of good responses: "Got it! That's about 520 calories." or "I
need a bit more info - what size portion?"

STRICT RULES:
```

```
1. TOPIC RESTRICTION: You MUST ONLY answer questions about food, calories, and nutrition. If asked about anything else (sports, coding, general knowledge, etc.), politely refuse: "I can only help with food and nutrition questions."

2. NO JSON IN RESPONSES: NEVER output raw JSON, code blocks, or structured data in your visible response to the user. Your response must ALWAYS be natural language only. The JSON data structure is handled separately and should never appear in your text.

3. ACCURACY REQUIREMENT: You must be highly accurate with calorie and macro estimations. If you are unsure about portion size, preparation method, or specific ingredients, provide a reasonable range or ask for more details. Do not hallucinate low-confidence numbers. When in doubt, ask clarifying questions.

4. NUTRITION DATA FORMAT: When you have enough information to provide nutrition data, include a JSON object at the END of your response (after your natural language text) in this exact format:
{
    "food_name": "Cheeseburger",
    "calories": 350.0,
    "protein_g": 20.0,
    "carbs_g": 30.0,
    "fat_g": 15.0,
    "ready_to_save": true
}

5. RESPONSE STRUCTURE: Your natural language response comes first, then the JSON (which is parsed separately and not shown to the user). If your text response would be empty after removing JSON, use a friendly message like "Logged!" or "All set!"

Remember: Be helpful and conversational, but concise. Only food/nutrition topics. Never show JSON to users. Be accurate with numbers."""


def parse_nutrition_result(response_text: str) -> Optional[Dict]:
    """Extract nutrition data from AI response if present."""
    # Try to find JSON in the response
```

```
    json_match = re.search(r'\{[^{}]*"food_name"[^{}]*\}', response_text,
re.DOTALL)
    if json_match:
        try:
            result = json.loads(json_match.group())
            # Validate required fields
            required_fields = ["food_name", "calories", "protein_g",
"carbs_g", "fat_g"]
            if all(field in result for field in required_fields):
                result["ready_to_save"] = result.get("ready_to_save",
False)
                return result
        except json.JSONDecodeError:
            pass
    return None


def process_food_entry(
    message: str,
    chat_history: list
) -> tuple[str, Optional[Dict]]:
    """
    Process a food entry message and get AI response with nutrition data.
    Returns: (reply_text, nutrition_result_dict or None)
    """
    # Create system prompt
    system_prompt = create_food_system_prompt()

    # Build messages for the API
    messages = [{"role": "system", "content": system_prompt}]

    # Add chat history (convert to API format)
    for msg in chat_history:
        messages.append({
            "role": msg.get("role", "user"),
            "content": msg.get("content", "")
        })

    # Add current user message
    messages.append({"role": "user", "content": message})
```

```
    try:
        # Call Groq API
        response = client.chat.completions.create(
            model="llama-3.3-70b-versatile",
            messages=messages,
            temperature=0.7,
            max_tokens=1000
        )

        reply_text = response.choices[0].message.content

        # Remove any visible JSON from the reply text (keep it clean for
user)
        # The JSON will be parsed separately and not shown to user
        reply_text_clean = reply_text
        json_match = re.search(r'\{[^{}]*"food_name"[^{}]*\}', reply_text,
re.DOTALL)
        if json_match:
            # Remove the JSON part from the visible text
            reply_text_clean = reply_text.replace(json_match.group(),
"").strip()
            # If reply is empty after removing JSON, use a default message
            if not reply_text_clean:
                reply_text_clean = "Logged"

        # Try to parse nutrition result from response
        nutrition_result = parse_nutrition_result(reply_text)

        return reply_text_clean, nutrition_result

    except Exception as e:
        print(f"Error calling Groq API: {e}")
        return f"I'm sorry, I encountered an error. Please try again.
Error: {str(e)}", None


from app.models.food_feed import FoodFeed
from app.models.food_chat import FoodChat
from app.db import db_helper
```

```python
from typing import Optional, List
from datetime import date


class FoodFeedService:
    @staticmethod
    def add_food_entry(user_id: int, content: str, entry_date:
Optional[str] = None) -> FoodFeed:
        """Add a food entry to the feed."""
        if entry_date is None:
            entry_date = date.today().isoformat()

        created_at = FoodFeed.now_iso()

        db_helper.execute_query(
            """INSERT INTO food_feed
               (user_id, content, entry_date, created_at)
               VALUES (?, ?, ?, ?)""",
            (user_id, content, entry_date, created_at)
        )

        # Fetch the last inserted entry
        row = db_helper.fetch_one(
            "SELECT * FROM food_feed WHERE user_id = ? ORDER BY id DESC
LIMIT 1",
            (user_id,)
        )
        return FoodFeed.from_row(row)

    @staticmethod
    def update_food_card(
        feed_id: int,
        food_name: str,
        calories: float,
        protein_g: float,
        carbs_g: float,
        fat_g: float
    ) -> FoodFeed:
        """Update a feed entry with AI-generated nutrition information."""
        db_helper.execute_query(
```

```
            """UPDATE food_feed
               SET food_name = ?, calories = ?, protein_g = ?, carbs_g =
?, fat_g = ?
               WHERE id = ?""",
            (food_name, calories, protein_g, carbs_g, fat_g, feed_id)
        )

        # Fetch the updated entry
        row = db_helper.fetch_one(
            "SELECT * FROM food_feed WHERE id = ?",
            (feed_id,)
        )
        return FoodFeed.from_row(row)

    @staticmethod
    def get_today_feed(user_id: int, entry_date: Optional[str] = None) ->
List[FoodFeed]:
        """Get all food feed entries for today, ordered by creation
time."""
        if entry_date is None:
            entry_date = date.today().isoformat()

        rows = db_helper.fetch_all(
            "SELECT * FROM food_feed WHERE user_id = ? AND entry_date = ?
ORDER BY created_at ASC",
            (user_id, entry_date)
        )
        return [FoodFeed.from_row(row) for row in rows]

    @staticmethod
    def get_feed_by_id(feed_id: int) -> Optional[FoodFeed]:
        """Get a specific feed entry by ID."""
        row = db_helper.fetch_one(
            "SELECT * FROM food_feed WHERE id = ?",
            (feed_id,)
        )
        if row is None:
            return None
        return FoodFeed.from_row(row)
```

```
    @staticmethod
    def delete_feed_entry(feed_id: int) -> bool:
        """Delete a feed entry."""
        db_helper.execute_query(
            "DELETE FROM food_feed WHERE id = ?",
            (feed_id,)
        )
        return True

    @staticmethod
    def get_chat_history(food_feed_id: int) -> List[FoodChat]:
        """Get all chat messages for a specific food feed entry, ordered by creation time."""
        rows = db_helper.fetch_all(
            "SELECT * FROM food_chat WHERE food_feed_id = ? ORDER BY created_at ASC",
            (food_feed_id,)
        )
        return [FoodChat.from_row(row) for row in rows]

    @staticmethod
    def save_chat_message(food_feed_id: int, role: str, content: str) -> FoodChat:
        """Save a chat message to the database."""
        created_at = FoodChat.now_iso()
        db_helper.execute_query(
            "INSERT INTO food_chat (food_feed_id, role, content, created_at) VALUES (?, ?, ?, ?)",
            (food_feed_id, role, content, created_at)
        )

        # Fetch the last inserted message
        row = db_helper.fetch_one(
            "SELECT * FROM food_chat WHERE food_feed_id = ? ORDER BY id DESC LIMIT 1",
            (food_feed_id,)
        )
        return FoodChat.from_row(row)
```

```python
from typing import Optional, Dict
from app.models.user import User
from app.db import db_helper
from app.services.auth_manager import Authentication_manager


class SettingsService:
    @staticmethod
    def get_user_settings(user_id: int) -> Optional[Dict]:
        """
        Get user settings (username, email, age, height, weight).
        Password is not included.

        Args:
            user_id: The ID of the user

        Returns:
            Dictionary with user settings, or None if user not found
        """
        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE id = ?",
            (user_id,)
        )

        if row is None:
            print("User not found")
            return None

        user = User.from_row(row)

        # Return settings without password
        return {
            "username": user.username,
            "email": user.email,
            "age": user.age,
            "height": user.height,
            "weight": user.weight
        }

    @staticmethod
```

```
    def update_user_settings(
        user_id: int,
        username: Optional[str] = None,
        email: Optional[str] = None,
        password: Optional[str] = None,
        confirm_password: Optional[str] = None,
        age: Optional[int] = None,
        height: Optional[int] = None,
        weight: Optional[int] = None
    ) -> Optional[Dict]:
        """
        Update user settings.

        Args:
            user_id: The ID of the user
            username: Optional new username
            email: Optional new email
            password: Optional new password (must provide confirm_password
if password is provided)
            confirm_password: Optional confirmation password
            age: Optional new age
            height: Optional new height
            weight: Optional new weight

        Returns:
            Dictionary with updated user settings, or None if user not
found or validation failed
        """
        # Check if user exists
        row = db_helper.fetch_one(
            "SELECT * FROM users WHERE id = ?",
            (user_id,)
        )

        if row is None:
            print("User not found")
            return None

        user = User.from_row(row)
```

```
        # Validate password if provided
        if password:
            if not confirm_password:
                print("confirm_password is required when password is
provided")
                return None

            if password != confirm_password:
                print("Password and confirm_password do not match")
                return None

        # Check if username is being changed and if it's already taken
        if username and username != user.username:
            existing_user = db_helper.fetch_one(
                "SELECT * FROM users WHERE username = ? AND id != ?",
                (username, user_id)
            )
            if existing_user is not None:
                print("Username already taken")
                return None

        # Check if email is being changed and if it's already taken
        if email and email != user.email:
            existing_user = db_helper.fetch_one(
                "SELECT * FROM users WHERE email = ? AND id != ?",
                (email, user_id)
            )
            if existing_user is not None:
                print("Email already taken")
                return None

        # Build update query dynamically
        fields_to_update = []
        params = []

        if username is not None:
            fields_to_update.append("username = ?")
            params.append(username)

        if email is not None:
```

```
            fields_to_update.append("email = ?")
            params.append(email)

        if password is not None:
            hashed_password =
Authentication_manager.hash_password(password)
            fields_to_update.append("password_hash = ?")
            params.append(hashed_password)

        if age is not None:
            fields_to_update.append("age = ?")
            params.append(age)

        if height is not None:
            fields_to_update.append("height = ?")
            params.append(height)

        if weight is not None:
            fields_to_update.append("weight = ?")
            params.append(weight)

        # Always update updated_at timestamp
        updated_at = User.now_iso()
        fields_to_update.append("updated_at = ?")
        params.append(updated_at)

        # If no fields to update, return current settings
        if not fields_to_update:
            return {
                "username": user.username,
                "email": user.email,
                "age": user.age,
                "height": user.height,
                "weight": user.weight
            }

        # Add user_id for WHERE clause
        params.append(user_id)

        # Build and execute UPDATE query
```

```
        set_clause = ", ".join(fields_to_update)
        db_helper.execute_query(
            f"UPDATE users SET {set_clause} WHERE id = ?",
            tuple(params)
        )

        # Fetch updated user
        updated_row = db_helper.fetch_one(
            "SELECT * FROM users WHERE id = ?",
            (user_id,)
        )

        if updated_row is None:
            print("Failed to fetch updated user")
            return None

        updated_user = User.from_row(updated_row)

        # Return updated settings without password
        return {
            "username": updated_user.username,
            "email": updated_user.email,
            "age": updated_user.age,
            "height": updated_user.height,
            "weight": updated_user.weight
        }


from app.models.tdee_profile import TdeeProfile
from app.models.tdee_chat import TdeeChat
from app.db import db_helper
from typing import Optional, List


class TdeeService:
    @staticmethod
    def get_profile_by_user_id(user_id: int) -> Optional[TdeeProfile]:
        """Get TDEE profile for a user."""
        row = db_helper.fetch_one(
            "SELECT * FROM tdee_profile WHERE user_id = ?",
```

```
            (user_id,)
        )
        if row is None:
            return None
        return TdeeProfile.from_row(row)

    @staticmethod
    def save_profile(
        user_id: int,
        activity_level: str,
        tdee_value: float,
        goal_type: str,
        goal_offset: int,
        goal_calories: float
    ) -> TdeeProfile:
        """Save or update TDEE profile for a user."""
        from app.models.tdee_profile import TdeeProfile

        # Check if profile exists
        existing = TdeeService.get_profile_by_user_id(user_id)

        if existing:
            # Update existing profile
            updated_at = TdeeProfile.now_iso()
            db_helper.execute_query(
                """UPDATE tdee_profile
                   SET activity_level = ?, tdee_value = ?, goal_type = ?,
                       goal_offset = ?, goal_calories = ?, updated_at = ?
                   WHERE user_id = ?""",
                (activity_level, tdee_value, goal_type, goal_offset,
goal_calories, updated_at, user_id)
            )
        else:
            # Create new profile
            created_at = TdeeProfile.now_iso()
            db_helper.execute_query(
                """INSERT INTO tdee_profile
                   (user_id, activity_level, tdee_value, goal_type,
goal_offset, goal_calories, created_at)
                   VALUES (?, ?, ?, ?, ?, ?, ?)""",
```

```
                (user_id, activity_level, tdee_value, goal_type,
goal_offset, goal_calories, created_at)
            )

        # Return the updated/new profile
        return TdeeService.get_profile_by_user_id(user_id)

    @staticmethod
    def get_chat_history(tdee_profile_id: int) -> List[TdeeChat]:
        """Get all chat messages for a TDEE profile, ordered by creation
time."""
        rows = db_helper.fetch_all(
            "SELECT * FROM tdee_chat WHERE tdee_profile_id = ? ORDER BY
created_at ASC",
            (tdee_profile_id,)
        )
        return [TdeeChat.from_row(row) for row in rows]

    @staticmethod
    def save_chat_message(tdee_profile_id: int, role: str, content: str)
-> TdeeChat:
        """Save a chat message to the database."""
        created_at = TdeeChat.now_iso()
        db_helper.execute_query(
            "INSERT INTO tdee_chat (tdee_profile_id, role, content,
created_at) VALUES (?, ?, ?, ?)",
            (tdee_profile_id, role, content, created_at)
        )

        # Fetch the last inserted message
        row = db_helper.fetch_one(
            "SELECT * FROM tdee_chat WHERE tdee_profile_id = ? ORDER BY id
DESC LIMIT 1",
            (tdee_profile_id,)
        )
        return TdeeChat.from_row(row)

    @staticmethod
    def get_or_create_profile_id(user_id: int) -> int:
```

```
        """Get existing profile ID or create a minimal profile and return
its ID."""
        profile = TdeeService.get_profile_by_user_id(user_id)
        if profile:
            return profile.id

        # Create minimal profile with defaults (will be updated when user
saves)
        # Reuse save_profile to avoid code duplication
        profile = TdeeService.save_profile(
            user_id=user_id,
            activity_level="sedentary",
            tdee_value=2000.0,
            goal_type="maintain",
            goal_offset=0,
            goal_calories=2000.0
        )
        return profile.id


from typing import List, Dict, Any, Optional
from datetime import datetime
from app.db import db_helper
from app.models.workout import Workout
from app.models.workout_exercise import WorkoutExercise


class WorkoutService:
    @staticmethod
    def create_workout_for_users(
        user_id: int,
        workout_name: str,
        log_date: str,
        notes: Optional[str],
        exercises_data: List[Dict[str, Any]],
    ) -> Dict[str, Any]:
        """
        Create a workout with exercises for a user.

        Args:
```

```
            user_id: The ID of the user creating the workout
            workout_name: Name of the workout
            log_date: Date of the workout (ISO format string)
            notes: Optional notes for the workout
            exercises_data: List of exercise dictionaries with keys:
                - exercise_name (str)
                - sets (int)
                - reps (int)
                - weight_kg (float, optional, defaults to 0)
                - previous_weight (float, optional, defaults to 0)
                - order_index (int)
                - notes (str, optional)

        Returns:
            Dictionary with 'workout' and 'exercises' keys containing
            serialized Workout and WorkoutExercise objects

        Raises:
            RuntimeError: If the workout creation fails
        """
        # Generate created_at timestamp
        created_at = Workout.now_iso()

        # Insert workout into database
        # Note: Table column is 'date', but method parameter is 'log_date'
for clarity
        db_helper.execute_query(
            """
            INSERT INTO workouts (user_id, workout_name, date, notes,
created_at)
            VALUES (?, ?, ?, ?, ?)
            """,
            (user_id, workout_name, log_date, notes, created_at)
        )

        # Fetch the inserted workout row
        workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE user_id = ? AND workout_name = ? AND date = ?
```

```
            ORDER BY id DESC
            LIMIT 1
            """,
            (user_id, workout_name, log_date)
        )

        if not workout_row:
            raise RuntimeError("Failed to create workout - workout was not
inserted or could not be retrieved")

        # Convert row to Workout model
        workout = Workout.from_row(workout_row)

        # Insert all exercises into workout_exercises
        for idx, exercise_dict in enumerate(exercises_data):
            exercise_name = exercise_dict.get("exercise_name")
            sets = exercise_dict.get("sets")
            reps = exercise_dict.get("reps")
            weight_kg = exercise_dict.get("weight_kg", 0.0)
            previous_weight = exercise_dict.get("previous_weight", 0.0)
            order_index = exercise_dict.get("order_index")
            exercise_notes = exercise_dict.get("notes")

            # Validate required fields before inserting
            if not exercise_name:
                raise ValueError(f"Exercise {idx + 1}: exercise_name is
required")
            if sets is None:
                raise ValueError(f"Exercise {idx + 1}: sets is required")
            if reps is None:
                raise ValueError(f"Exercise {idx + 1}: reps is required")
            if order_index is None:
                raise ValueError(f"Exercise {idx + 1}: order_index is
required")

            try:
                db_helper.execute_query(
                    """
                    INSERT INTO workout_exercises (
                        workout_id, exercise_name, sets, reps, weight_kg,
```

```
                        previous_weight, order_index, notes
                    )
                    VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                    """,
                    (
                        workout.id,
                        exercise_name,
                        sets,
                        reps,
                        weight_kg,
                        previous_weight,
                        order_index,
                        exercise_notes,
                    )
                )
            except Exception as e:
                raise RuntimeError(f"Failed to insert exercise {idx + 1}
({exercise_name}): {str(e)}")

        # Fetch all exercises back for this workout
        exercise_rows = db_helper.fetch_all(
            """
            SELECT * FROM workout_exercises
            WHERE workout_id = ?
            ORDER BY order_index ASC
            """,
            (workout.id,)
        )

        # Convert each row to WorkoutExercise model
        exercises = [WorkoutExercise.from_row(row) for row in
exercise_rows]

        # Return clean dict suitable for JSON response
        return {
            "workout": workout.to_dict(),
            "exercises": [ex.to_dict() for ex in exercises]
        }

    @staticmethod
```

```python
    def get_workout_detail(workout_id: int, user_id: int) ->
Optional[Dict[str, Any]]:
        """
        Get workout details with all exercises for a specific workout.

        Args:
            workout_id: The ID of the workout to retrieve
            user_id: The ID of the user (for security - ensures user owns
the workout)

        Returns:
            Dictionary with 'workout' and 'exercises' keys containing
            serialized Workout and WorkoutExercise objects, or None if not
found
        """
        # Fetch the workout row
        workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        # If no workout is found, return None
        if not workout_row:
            return None

        # Convert the workout DB row → Workout model
        workout = Workout.from_row(workout_row)

        # Fetch all exercises belonging to this workout
        exercise_rows = db_helper.fetch_all(
            """
            SELECT * FROM workout_exercises
            WHERE workout_id = ?
            ORDER BY order_index ASC
            """,
            (workout_id,)
        )
```

```
        # Convert each exercise row → WorkoutExercise model
        exercises = [WorkoutExercise.from_row(row) for row in
exercise_rows]

        # Return clean dictionary
        return {
            "workout": workout.to_dict(),
            "exercises": [ex.to_dict() for ex in exercises]
        }

    @staticmethod
    def get_workouts_for_user(user_id: int) -> List[Dict[str, Any]]:
        """
        Get all workouts for a specific user.

        Args:
            user_id: The ID of the user

        Returns:
            List of dictionaries, each containing 'workout' and
'exercises' keys
        """
        # Fetch all workouts for this user
        workout_rows = db_helper.fetch_all(
            """
            SELECT * FROM workouts
            WHERE user_id = ?
            ORDER BY date DESC, created_at DESC
            """,
            (user_id,)
        )

        if not workout_rows:
            return []

        # For each workout, get its exercises
        result = []
        for workout_row in workout_rows:
            workout = Workout.from_row(workout_row)
```

```
            # Get exercises for this workout
            exercise_rows = db_helper.fetch_all(
                """
                SELECT * FROM workout_exercises
                WHERE workout_id = ?
                ORDER BY order_index ASC
                """,
                (workout.id,)
            )

            exercises = [WorkoutExercise.from_row(row) for row in
exercise_rows]

            result.append({
                "workout": workout.to_dict(),
                "exercises": [ex.to_dict() for ex in exercises]
            })

        return result

    @staticmethod
    def delete_workout(workout_id: int, user_id: int) -> bool:
        """
        Delete a workout and its associated exercises.

        Args:
            workout_id: The ID of the workout to delete
            user_id: The ID of the user (for security - ensures user owns
the workout)

        Returns:
            True if the workout was deleted, False if it didn't exist or
doesn't belong to the user
        """
        # Verify that the workout belongs to the given user
        workout_row = db_helper.fetch_one(
            """
            SELECT id
            FROM workouts
```

```
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        # If no row is found, return False (nothing deleted, unauthorized or non-existent)
        if not workout_row:
            return False

        # Delete the workout (ON DELETE CASCADE will automatically delete related workout_exercises)
        db_helper.execute_query(
            """
            DELETE FROM workouts
            WHERE id = ?
            """,
            (workout_id,)
        )

        # Return True if deletion happened
        return True

    @staticmethod
    def update_workout(
        workout_id: int,
        user_id: int,
        workout_name: Optional[str] = None,
        log_date: Optional[str] = None,
        notes: Optional[str] = None,
    ) -> Optional[Dict[str, Any]]:
        """
        Update a workout's fields (partial update - only provided fields are updated).

        Args:
            workout_id: The ID of the workout to update
            user_id: The ID of the user (for security - ensures user owns the workout)
            workout_name: Optional new workout name
```

```
            log_date: Optional new log date
            notes: Optional new notes

        Returns:
            Dictionary representation of the updated workout, or None if
not found/unauthorized
        """
        # Check that the workout exists and belongs to the given user
        workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        # If no row is found, return None
        if not workout_row:
            return None

        # Build list of fields to update (only non-None values)
        fields_to_update = []
        params = []

        if workout_name is not None:
            fields_to_update.append("workout_name = ?")
            params.append(workout_name)

        if log_date is not None:
            fields_to_update.append("date = ?")
            params.append(log_date)

        if notes is not None:
            fields_to_update.append("notes = ?")
            params.append(notes)

        # If there are no fields to update, just return the current
workout
        if not fields_to_update:
            workout = Workout.from_row(workout_row)
```

```
            return workout.to_dict()

        # Add workout_id and user_id to params for WHERE clause
        params.extend([workout_id, user_id])

        # Build and execute the UPDATE statement
        set_clause = ", ".join(fields_to_update)
        db_helper.execute_query(
            f"""
            UPDATE workouts
            SET {set_clause}
            WHERE id = ? AND user_id = ?
            """,
            tuple(params)
        )

        # Fetch the updated workout row
        updated_workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        # Convert it using Workout.from_row
        workout = Workout.from_row(updated_workout_row)

        # Return dict representation
        return workout.to_dict()

    @staticmethod
    def update_workout_with_exercises(
        workout_id: int,
        user_id: int,
        workout_name: Optional[str] = None,
        notes: Optional[str] = None,
        exercises: Optional[List[Dict[str, Any]]] = None,
    ) -> Optional[Dict[str, Any]]:
        """
```

```
        Update a workout and its exercises.

        Args:
            workout_id: The ID of the workout to update
            user_id: The ID of the user (for security - ensures user owns
the workout)
            workout_name: Optional new workout name
            notes: Optional new notes
            exercises: Optional list of exercise dictionaries to
update/create

        Returns:
            Dictionary with 'workout' and 'exercises' keys containing
            serialized Workout and WorkoutExercise objects, or None if not
found
        """
        # Verify that the workout exists and belongs to the given user
        workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        # If no row is found, return None
        if not workout_row:
            return None

        # Update workout fields if provided
        if workout_name is not None or notes is not None:
            fields_to_update = []
            params = []

            if workout_name is not None:
                fields_to_update.append("workout_name = ?")
                params.append(workout_name)

            if notes is not None:
                fields_to_update.append("notes = ?")
```

```
                params.append(notes)

            if fields_to_update:
                params.extend([workout_id, user_id])
                set_clause = ", ".join(fields_to_update)
                db_helper.execute_query(
                    f"""
                    UPDATE workouts
                    SET {set_clause}
                    WHERE id = ? AND user_id = ?
                    """,
                    tuple(params)
                )

        # Update exercises if provided
        if exercises is not None:
            # Get existing exercise IDs for this workout
            existing_exercise_rows = db_helper.fetch_all(
                """
                SELECT id FROM workout_exercises
                WHERE workout_id = ?
                """,
                (workout_id,)
            )
            existing_exercise_ids = {row[0] for row in
existing_exercise_rows}

            # Process each exercise in the provided list
            provided_exercise_ids = set()
            for idx, exercise_dict in enumerate(exercises):
                exercise_id = exercise_dict.get("id")
                exercise_name = exercise_dict.get("exercise_name",
"").strip()

                if not exercise_name:
                    continue  # Skip exercises without names

                sets = exercise_dict.get("sets", 0)
                reps = exercise_dict.get("reps", 0)
                weight_kg = exercise_dict.get("weight_kg", 0.0)
```

```
                previous_weight = exercise_dict.get("previous_weight",
0.0)
                exercise_notes = exercise_dict.get("notes", "")
                order_index = exercise_dict.get("order_index", idx)

                if exercise_id and exercise_id in existing_exercise_ids:
                    # Update existing exercise
                    provided_exercise_ids.add(exercise_id)
                    db_helper.execute_query(
                        """
                        UPDATE workout_exercises
                        SET exercise_name = ?, sets = ?, reps = ?,
weight_kg = ?,
                            previous_weight = ?, order_index = ?, notes =
?
                        WHERE id = ? AND workout_id = ?
                        """,
                        (
                            exercise_name, sets, reps, weight_kg,
                            previous_weight, order_index, exercise_notes,
                            exercise_id, workout_id
                        )
                    )
                else:
                    # Insert new exercise
                    db_helper.execute_query(
                        """
                        INSERT INTO workout_exercises (
                            workout_id, exercise_name, sets, reps,
weight_kg,
                            previous_weight, order_index, notes
                        )
                        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
                        """,
                        (
                            workout_id, exercise_name, sets, reps,
weight_kg,
                            previous_weight, order_index, exercise_notes,
                        )
                    )
```

```
            # Delete exercises that were not in the provided list
            exercises_to_delete = existing_exercise_ids -
provided_exercise_ids
            if exercises_to_delete:
                placeholders = ",".join("?" * len(exercises_to_delete))
                db_helper.execute_query(
                    f"""
                    DELETE FROM workout_exercises
                    WHERE id IN ({placeholders}) AND workout_id = ?
                    """,
                    tuple(exercises_to_delete) + (workout_id,)
                )

        # Fetch the updated workout
        updated_workout_row = db_helper.fetch_one(
            """
            SELECT * FROM workouts
            WHERE id = ? AND user_id = ?
            """,
            (workout_id, user_id)
        )

        workout = Workout.from_row(updated_workout_row)

        # Fetch all exercises for this workout
        exercise_rows = db_helper.fetch_all(
            """
            SELECT * FROM workout_exercises
            WHERE workout_id = ?
            ORDER BY order_index ASC
            """,
            (workout_id,)
        )

        exercises = [WorkoutExercise.from_row(row) for row in
exercise_rows]

        # Return clean dictionary
        return {
```

```
            "workout": workout.to_dict(),
            "exercises": [ex.to_dict() for ex in exercises]
        }

/*  Background */
body {
    margin: 0;
    height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    font-family: 'Inter', sans-serif;
    background: linear-gradient(rgba(0, 0, 0, 0.8), rgba(0, 0, 0, 0.9)),
                url('../images/background.jpg');
    background-size: cover;
    background-position: center;
    color: white;
}


/*   The Login Box  */
.login-container {
    width: 350px;
    padding: 40px;
    background: #0A0A0A; /* Solid dark card */
    border: 1px solid #333;
    border-radius: 20px;
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.1); /* Blue glow around box
*/
    text-align: center;
}

/*  The Logo : Futuristic Text */
.logo {
    font-family: 'Orbitron', sans-serif;
    font-size: 28px;
    font-weight: bold;
    margin-bottom: 30px;
    letter-spacing: 2px;
}
```

```
.ai-text { color: #00A8FF; }

/* Inputs */
input {
    width: 100%;
    padding: 15px;
    margin-bottom: 20px;
    background: #1F1F1F;
    border: 1px solid #333;
    border-radius: 8px;
    color: white;
    box-sizing: border-box;
}

input:focus {
    outline: none;
    border-color: #00A8FF; /* Blue border when typing */
}

/* Button */
button {
    width: 100%;
    padding: 15px;
    background: #00A8FF;
    color: white;
    border: none;
    border-radius: 8px;
    font-family: 'Orbitron', sans-serif;
    font-size: 16px;
    cursor: pointer;
}

button:hover {
    background: #008ecf;
    box-shadow: 0 0 15px rgba(0, 168, 255, 0.5); /* Glow effect in button
*/
}

/* Footer Links */
.form-footer {
```

```
    display: flex;
    justify-content: space-between; /* Pushes links apart */
    margin-top: 15px;
    font-size: 13px;
}
.form-footer a {
    color: #666;
    text-decoration: none;
}
.form-footer a:hover { color: #00A8FF; }

/* Subtitle text under the logo */
.subtitle {
    margin-top: -20px;
    margin-bottom: 20px;
    color: #888;
    font-size: 14px;
    letter-spacing: 1px;
    text-transform: uppercase;
}

/* --- NEW TOP NAVIGATION LAYOUT --- */

/* 1. The Container */
.dashboard-body {
    display: block; /* Stack top to bottom */
    background: linear-gradient(rgba(0, 0, 0, 0.7), rgba(0, 0, 0, 0.9)),
                url('../images/background.jpg');
    background-size: cover;
    background-position: center;
    min-height: 100vh;
    overflow-x: hidden;
    overflow-y: auto; /* Allow scrolling for tall content */
}

/* 2. The Glass Top Bar */
.top-nav {
    display: flex;
    justify-content: space-between;
    align-items: center;
```

```
    padding: 20px 50px;
    background: rgba(255, 255, 255, 0.03); /* Ultra subtle glass */
    backdrop-filter: blur(10px); /* Blurs the background behind it */
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

/* 3. The Links (Horizontal) */
.nav-links {
    display: flex; /* Puts them in a row */
    gap: 30px; /* Space between words */
    margin: 0;
}

.nav-links a {
    color: #aaa;
    text-decoration: none;
    font-size: 14px;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 1px;
    transition: 0.3s;
}

.nav-links a:hover, .nav-links a.active {
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5); /* Glow text */
}

/* --- ANIMATED CONTENT --- */

.main-content {
    padding: 60px 50px;
    color: white;
}

.hero-text {
    max-width: 800px;
    margin: 0 auto;
    text-align: left;
}
```

```css
.hero-text h1 {
    font-family: 'Orbitron', sans-serif; /* The Cyberpunk Font */
    font-size: 56px;
    font-weight: 800;
    text-transform: uppercase;
    letter-spacing: 3px;
    margin-bottom: 10px;

    /* The Gradient Color */
    background: linear-gradient(to right, #ffffff, #00A8FF);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;

}

.subtitle {
    color: #888;
    font-size: 18px;
    margin-bottom: 50px;
}

/* --- GLASS CARDS (Upscaled) --- */
.dashboard-cards {
    display: flex;
    flex-direction: row;
    justify-content: center;
    gap: 40px; /* More space between cards */
    margin-top: 60px; /* More space from the top text */
    flex-wrap: wrap;
}

.card {
    background: rgba(255, 255, 255, 0.15); /* Your darker opacity */
    border: 1px solid rgba(255, 255, 255, 0.1);
    padding: 40px; /* More breathing room inside */
    border-radius: 25px; /* Rounder corners */
    width: 320px; /* Wider boxes (Was 250px) */
    min-height: 220px; /* Taller boxes */
```

```
    /* 3D Tilt Setup */
    transform-style: preserve-3d;
    transform: perspective(1000px);

    /* Center content */
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: center;
}

.card {
    background: rgba(13, 13, 13, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 25px;
    width: 320px;
    min-height: 220px;

    /* 3D Tilt Setup */
    transform-style: preserve-3d;
    transform: perspective(1000px);

    /* LAYOUT FIX: Align to top, not center */
    display: flex;
    flex-direction: column;
    align-items: center;
    justify-content: flex-start; /* <--- This aligns them to the top */
    padding-top: 40px; /* <--- This pushes them down equally */
}

.big-number {
    font-size: 40px;
    color: #00A8FF;
    font-weight: 800;
    font-family: 'Orbitron', sans-serif;
    margin-top: 37px;
    text-shadow: 0 0 25px rgba(0, 168, 255, 0.5);
}

/* Specific fix for the Text inside the Circle */
```

```
.cal-value {
    font-family: 'Orbitron', sans-serif;
    font-size: 32px; /* Bigger number inside circle */
    font-weight: bold;
    color: white;
}

.card:hover {
    background: rgba(255, 255, 255, 0.1);
    border-color: #00A8FF;
    transform: translateY(-10px); /* Moves up when hovered */
}

/* --- KEYFRAMES (The Typing Logic) --- */
@keyframes typing {
    from { width: 0 }
    to { width: 100% }
}

@keyframes blink {
    from, to { border-color: transparent }
    50% { border-color: #00A8FF }
}

/* --- CIRCULAR PROGRESS BAR --- */

.progress-container {
    position: relative;
    width: 160px; /* Updated to match new SVG */
    height: 160px; /* Updated to match new SVG */
    margin: 0 auto;
    margin-top: 15px;
}

/* Rotates the circle so it starts at the top (12 o'clock) */
.progress-ring {
    transform: rotate(-90deg);
}

/* The Neon Blue Line Logic */
```

```css
.progress-ring__circle {
    stroke-dasharray: 327; /* The total length of the circle circumference */
    stroke-dashoffset: 327; /* Start completely empty */
    stroke-linecap: round; /* Rounded edges look nicer */
    transition: stroke-dashoffset 1.5s ease-out; /* The Animation Speed */
    filter: drop-shadow(0 0 5px #00A8FF); /* Neon Glow */
}

/* The Text Inside */
.progress-text {
    position: absolute;
    top: 50%;
    left: 50%;
    transform: translate(-50%, -50%);
    text-align: center;
    display: flex;
    flex-direction: column;
}

.cal-value {
    font-family: 'Orbitron', sans-serif;
    font-size: 24px;
    font-weight: bold;
    color: white;
}

.cal-label {
    font-size: 12px;
    color: #888;
    text-transform: uppercase;
}

/* --- TDEE PAGE STYLES --- */

/* 1. The Chat Section */
.chat-section {
    width: 100%;
    max-width: 800px;
    margin: 0 auto 50px auto; /* Centers it and adds space below */
```

```
}

.chat-window {
    background: rgba(0, 0, 0, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    height: 300px;
    padding: 20px;
    overflow-y: auto; /* Scrollable */
    margin-bottom: 15px;
    display: flex;
    flex-direction: column;
    gap: 10px;
}

/* Toast Notification */
.toast-container {
    position: fixed;
    top: 20px;
    right: 20px;
    z-index: 10000;
    pointer-events: none;
}

.toast {
    background: rgba(0, 0, 0, 0.9);
    border: 1px solid #00A8FF;
    border-radius: 8px;
    padding: 16px 24px;
    margin-bottom: 12px;
    min-width: 300px;
    max-width: 400px;
    box-shadow: 0 4px 20px rgba(0, 168, 255, 0.3);
    color: white;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    display: flex;
    align-items: center;
    gap: 12px;
    opacity: 0;
```

```
    transform: translateX(400px);
    transition: all 0.3s ease-in-out;
    pointer-events: auto;
}

.toast.show {
    opacity: 1;
    transform: translateX(0);
}

.toast.success {
    border-color: #4ade80;
    box-shadow: 0 4px 20px rgba(74, 222, 128, 0.3);
}

.toast.error {
    border-color: #ff6b6b;
    box-shadow: 0 4px 20px rgba(255, 107, 107, 0.3);
}

.toast-icon {
    font-size: 20px;
    flex-shrink: 0;
}

.toast.success .toast-icon {
    color: #4ade80;
}

.toast.error .toast-icon {
    color: #ff6b6b;
}

.toast-message {
    flex: 1;
    line-height: 1.5;
}

/* Messages */
.message {
```

```
    padding: 10px 15px;
    border-radius: 10px;
    font-size: 14px;
    max-width: 70%;
    opacity: 1 !important;
    visibility: visible !important;
    word-wrap: break-word;
}
.bot-msg {
    background: rgba(0, 168, 255, 0.2);
    color: #fff !important;
    align-self: flex-start; /* Left side */
    border: 1px solid rgba(0, 168, 255, 0.3);
}
.user-msg {
    background: rgba(255, 255, 255, 0.15);
    color: #fff !important;
    align-self: flex-end; /* Right side */
    border: 1px solid rgba(255, 255, 255, 0.2);
}

/* Chat Input */
.chat-input-area {
    display: flex;
    gap: 10px;
}
.chat-input {
    flex-grow: 1; /* Takes up all space */
    background: rgba(255, 255, 255, 0.1);
    border: none;
    padding: 15px;
    border-radius: 8px;
    color: white;
}
.send-btn {
    width: 100px;
    background: #00A8FF;
    color: white;
    font-weight: bold;
    border: none;
```

```
    border-radius: 8px;
    cursor: pointer;
}

/* 2. The Data Form Section */
.chat-form-section {
    max-width: 800px;
    margin: 0 auto;
    background: rgba(255, 255, 255, 0.05); /* Glass background */
    padding: 30px;
    border-radius: 20px;
    border: 1px solid rgba(255, 255, 255, 0.1);
}

.chat-form-section {
    margin-top: 30px;
}

.section-title {
    font-family: 'Orbitron', sans-serif;
    color: #00A8FF !important;
    margin: 0 auto 20px auto;
    font-size: 24px;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    max-width: 800px;
    text-align: left;
}

.date-display {
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    color: #888;
    padding: 8px 15px;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 8px;
    text-transform: uppercase;
    letter-spacing: 1px;
}
```

```
/* The Grid Layout for Inputs */
.stats-grid {
    display: grid;
    grid-template-columns: 1fr 1fr; /* Two columns */
    gap: 20px;
}

.input-group {
    display: flex;
    flex-direction: column;
}

.input-group label {
    color: #888;
    margin-bottom: 8px;
    font-size: 14px;
}

.input-group input {
    background: #111;
    border: 1px solid #444;
    padding: 12px;
    border-radius: 5px;
    color: white;
    width: 100%;
    box-sizing: border-box;
}

.input-group select {
    background: #111;
    border: 1px solid #444;
    padding: 12px;
    padding-right: 40px; /* Make room for the arrow */
    border-radius: 5px;
    color: white;
    width: 100%;
    box-sizing: border-box;
    appearance: none; /* Hide default arrow */
    background-image:
url('data:image/svg+xml;charset=US-ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2F
```

```
www.w3.org%2F2000%2Fsvg%22%20width%3D%22292.4%22%20height%3D%22292.4%22%3E%3Cpath%20fill%3D%22%2300A8FF%22%20d%3D%22M287%2069.4a17.6%2017.6%200%200%200-24.7%200L146.2%20185.6%2029.4%2069.4a17.6%2017.6%200%200%200-24.7%2024.7l128.9%20129a17.6%2017.6%200%200%200%2024.7%200l128.9-129a17.6%2017.6%200%200%200%200-24.7z%22%2F%3E%3C%2Fsvg%3E');
    background-repeat: no-repeat;
    background-position: right 12px center;
    background-size: 12px;
    cursor: pointer;
}

.input-group select:focus {
    border-color: #00A8FF;
    outline: none;
    color: white;
}

.save-btn {
    grid-column: span 2; /* Button stretches across both columns */
    background: #00A8FF;
    color: white;
    padding: 15px;
    font-size: 18px;
    font-family: 'Orbitron', sans-serif;
    margin-top: 20px;
}

/* --- Register Page Updates --- */

/* Puts items side-by-side */
.form-row {
    display: flex;
    justify-content: space-between;
    margin-bottom: 10px;
}

/* Style the Dropdown to match inputs */
/* Style the Dropdown to match inputs */
select {
    padding: 15px;
```

```
    background: #1F1F1F;
    border: 1px solid #333;
    border-radius: 8px;
    color: #888;

    /* --- NEW: ADD THE ARROW --- */
    appearance: none; /* 1. Hide default ugly arrow */
    background-image:
url('data:image/svg+xml;charset=US-ASCII,%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%22292.4%22%20height%3D%22292.4%22%3E%3Cpath%20fill%3D%22%2300A8FF%22%20d%3D%22M287%2069.4a17.6%2017.6%200%200%200-24.7%200L146.2%20185.6%2029.4%2069.4a17.6%2017.6%200%200%200-24.7%2024.7l128.9%20129a17.6%2017.6%200%200%200%2024.7%200l128.9-129a17.6%2017.6%200%200%200%200-24.7z%22%2F%3E%3C%2Fsvg%3E');
    background-repeat: no-repeat;
    background-position: right 15px center; /* Position on the right */
    background-size: 12px; /* Size of the arrow */
    padding-right: 40px; /* Make room for the arrow so text doesn't overlap */

    cursor: pointer;
    margin-bottom: 20px;
}

/* Change text color when option is selected */
select:focus {
    color: white;
    border-color: #00A8FF;
    outline: none;
}

/* --- CALORIES PAGE STYLES --- */

/* Tracker Container: Split-screen layout */
.tracker-container {
    display: flex;
    flex-direction: row;
    gap: 30px;
    width: 100%;
    max-width: 1400px;
```

```
    margin: 0 auto;
    height: calc(100vh - 200px);
    min-height: 600px;
    align-items: stretch;
}

/* Left Column: Daily Log Section */
.log-section {
    width: 40%;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    padding: 30px;
    display: flex;
    flex-direction: column;
    overflow: hidden;
}

/* Macro Summary Row */
.macro-summary {
    display: flex;
    flex-direction: row;
    gap: 12px;
    margin-bottom: 30px;
    flex-wrap: nowrap;
}

.macro-box {
    flex: 1;
    min-width: 0; /* Prevents flex items from overflowing */
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    padding: 15px 10px;
    text-align: center;
}

.macro-label {
    font-family: 'Inter', sans-serif;
    font-size: 14px;
```

```
    color: #888 !important;
    margin-bottom: 10px;
    text-transform: uppercase;
    letter-spacing: 1px;
}

.macro-value {
    font-family: 'Orbitron', sans-serif;
    font-size: 24px;
    font-weight: 700;
    color: #00A8FF !important;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
}

/* Surplus value can be green or red, override the default blue */
#total-surplus {
    color: #00ff00 !important; /* Default green */
}

/* Food Feed: Scrollable list */
.food-feed {
    flex: 1;
    overflow-y: auto;
    display: flex;
    flex-direction: column;
    gap: 12px;
    padding-right: 10px;
}

/* Custom scrollbar for food feed */
.food-feed::-webkit-scrollbar {
    width: 6px;
}

.food-feed::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.05);
    border-radius: 10px;
}

.food-feed::-webkit-scrollbar-thumb {
```

```
    background: #00A8FF;
    border-radius: 10px;
}

.food-feed::-webkit-scrollbar-thumb:hover {
    background: #008ecf;
}

/* Food Item */
.food-item {
    background: rgba(0, 0, 0, 0.5);
    border-left: 3px solid #00A8FF;
    border-radius: 8px;
    padding: 15px 20px;
    display: flex;
    flex-direction: column;
    gap: 10px;
    transition: all 0.3s ease;
    opacity: 1;
    visibility: visible;
    margin-bottom: 8px;
}

.food-item:hover {
    background: rgba(0, 0, 0, 0.7);
    border-left-color: #008ecf;
    transform: translateX(5px);
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
}

/* Delete button - positioned absolutely on the right, fades in on hover
*/
.food-delete-btn {
    position: absolute;
    right: 10px; /* Match the padding-right of container for perfect
alignment */
    top: 50%;
    transform: translateY(-50%);
    background: transparent;
    border: none;
```

```
    cursor: pointer;
    padding: 4px 6px; /* Reduced padding for smaller icon */
    display: flex;
    align-items: center;
    justify-content: center;
    border-radius: 4px;
    opacity: 0;
    visibility: hidden;
    pointer-events: none;
    transition: opacity 0.3s ease-in-out, visibility 0.3s ease-in-out,
background 0.2s ease, color 0.2s ease, transform 0.2s ease;
    color: #888;
    z-index: 10;
}

/* On hover, fade in delete button */
.food-item:hover .food-delete-btn {
    opacity: 1;
    visibility: visible;
    pointer-events: auto;
}

.food-delete-btn:hover {
    background: rgba(255, 0, 0, 0.15);
    color: #ff4444;
    transform: translateY(-50%) scale(1.1);
}

.food-delete-btn:active {
    transform: translateY(-50%) scale(0.95);
    color: #ff0000;
    background: rgba(255, 0, 0, 0.25);
}

.food-delete-btn svg {
    width: 14px; /* Reduced from 18px for a more subtle icon */
    height: 14px; /* Reduced from 18px for a more subtle icon */
    stroke: currentColor;
    stroke-width: 2;
}
```

```css
.food-item-main {
    display: flex;
    justify-content: space-between;
    align-items: center;
    width: 100%;
    position: relative;
}

.food-name {
    font-family: 'Inter', sans-serif;
    font-size: 16px;
    color: #fff !important;
    font-weight: 600;
    flex: 1;
}

/* Container for calories and delete button */
.food-calories-container {
    position: relative;
    display: flex;
    align-items: center;
    justify-content: flex-end; /* Align to middle-right */
    min-width: 120px; /* Ensure space for both elements */
    height: 100%;
    padding-right: 10px; /* Padding from the edge - matches delete button position */
}

.food-calories {
    font-family: 'Orbitron', sans-serif;
    font-size: 16px; /* Increased from 14px for better prominence */
    color: #00A8FF !important;
    font-weight: 600;
    transition: transform 0.3s ease-in-out;
    transform: translateX(0);
    white-space: nowrap;
    text-align: right; /* Align text to the right */
    margin-right: 0; /* No margin by default */
}
```

```
/* On hover, slide calories left to make room for delete button */
/* Increased slide distance to -75px to ensure 8-12px gap between text and icon */
/* Icon is ~26px wide (14px icon + 12px padding), so -75px provides ~12px gap */
.food-item:hover .food-calories {
    transform: translateX(-75px);
}

.food-macros {
    display: flex;
    gap: 10px;
    flex-wrap: wrap;
}

.macro-badge {
    font-family: 'Inter', sans-serif;
    font-size: 12px;
    padding: 4px 10px;
    border-radius: 12px;
    font-weight: 600;
    text-transform: uppercase;
    letter-spacing: 0.5px;
}

.macro-badge.protein {
    background: rgba(0, 168, 255, 0.2);
    color: #00A8FF;
    border: 1px solid rgba(0, 168, 255, 0.3);
}

.macro-badge.carbs {
    background: rgba(255, 193, 7, 0.2);
    color: #FFC107;
    border: 1px solid rgba(255, 193, 7, 0.3);
}

.macro-badge.fat {
    background: rgba(255, 87, 34, 0.2);
```

```
    color: #FF5722;
    border: 1px solid rgba(255, 87, 34, 0.3);
}

/* Right Column: Chat Section */
.calories-chat {
    width: 60%;
    display: flex;
    flex-direction: column;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    padding: 30px;
    overflow: hidden;
    align-items: stretch;
}

.calories-chat-window {
    flex: 1;
    background: rgba(0, 0, 0, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    padding: 20px;
    overflow-y: auto;
    margin-bottom: 15px;
    display: flex;
    flex-direction: column;
    gap: 10px;
    min-height: 0;
}

/* Chat Input Area */
.chat-input-area {
    display: flex;
    gap: 10px;
    width: 100%;
    box-sizing: border-box;
}

.calories-chat-input {
```

```
    flex: 1;
    min-width: 0;
    background: rgba(255, 255, 255, 0.1);
    border: 1px solid rgba(255, 255, 255, 0.2);
    padding: 15px;
    border-radius: 8px;
    color: white !important;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    box-sizing: border-box;
}

.calories-chat-input:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
    background: rgba(255, 255, 255, 0.15);
}

.calories-chat-input::placeholder {
    color: #888;
    opacity: 1;
}

.log-btn {
    flex-shrink: 0;
    width: 120px;
    background: #00A8FF !important;
    color: white !important;
    font-weight: bold;
    border: none;
    border-radius: 8px;
    padding: 15px 20px;
    font-family: 'Orbitron', sans-serif;
    font-size: 14px;
    cursor: pointer;
    transition: all 0.3s ease;
    box-sizing: border-box;
}
```

```css
.log-btn:hover {
    background: #008ecf !important;
    box-shadow: 0 0 15px rgba(0, 168, 255, 0.5);
    transform: translateY(-2px);
}

.log-btn:active {
    transform: translateY(0);
}

.log-btn:disabled {
    opacity: 0.6;
    cursor: not-allowed;
}

/* Workout History Feed Styles */
.workout-log-container {
    max-width: 800px;
    margin: 0 auto;
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 25px;
    padding: 40px;
    margin-bottom: 100px; /* Space for fixed button */
}

.workout-feed-container {
    display: flex;
    flex-direction: column;
    gap: 20px;
    width: 100%;
}

.log-card {
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    padding: 20px;
    display: flex;
```

```
    align-items: center;
    gap: 20px;
    transition: all 0.3s ease;
    position: relative;
    overflow: visible;
    cursor: pointer;
}

.log-card:hover {
    background: rgba(255, 255, 255, 0.08);
    transform: translateX(5px);
    box-shadow: 0 0 15px rgba(0, 168, 255, 0.2);
}

.log-card .delete-log-btn {
    cursor: pointer;
    z-index: 10;
}

.log-card:hover {
    background: rgba(255, 255, 255, 0.08);
    transform: translateX(5px);
    box-shadow: 0 0 15px rgba(0, 168, 255, 0.2);
}

.log-card.today-highlight {
    border: 2px solid #00A8FF;
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.4);
    background: rgba(0, 168, 255, 0.1);
    order: -1; /* Force to top using CSS order */
}

.day-badge {
    font-family: 'Orbitron', sans-serif;
    font-size: 32px;
    font-weight: 700;
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    min-width: 80px;
    text-align: center;
```

```
    letter-spacing: 2px;
}

.log-card.today-highlight .day-badge {
    color: #00A8FF;
    text-shadow: 0 0 15px rgba(0, 168, 255, 0.8);
}

.log-summary {
    font-family: 'Inter', sans-serif;
    font-size: 18px;
    font-weight: 600;
    color: #fff;
    flex: 1;
}

.delete-log-btn {
    position: absolute;
    right: 15px;
    top: 50%;
    transform: translateY(-50%);
    width: 32px;
    height: 32px;
    border-radius: 50%;
    border: 2px solid #ff4444;
    background: transparent;
    color: #ff4444;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    opacity: 0;
    transition: opacity 0.3s ease, background 0.2s ease, border-color 0.2s
ease, transform 0.2s ease;
    z-index: 10;
}

.delete-log-btn:hover {
    background: rgba(255, 68, 68, 0.1);
    border-color: #ff0000;
```

```
    color: #ff0000;
    transform: translateY(-50%) scale(1.1);
}

.delete-log-btn:active {
    transform: translateY(-50%) scale(0.95);
}

.delete-log-btn i {
    font-size: 14px;
}

.log-card:hover .delete-log-btn {
    opacity: 1;
}

.details-btn {
    position: absolute;
    right: 55px; /* Position to the left of delete button (32px button + 15px right + 8px gap) */
    top: 50%;
    transform: translateY(-50%);
    width: 32px;
    height: 32px;
    border-radius: 50%;
    border: 2px solid #00A8FF;
    background: transparent;
    color: #00A8FF;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    opacity: 0;
    transition: opacity 0.3s ease, background 0.2s ease, border-color 0.2s ease, transform 0.2s ease;
    z-index: 10;
}

.details-btn:hover {
    background: rgba(0, 168, 255, 0.1);
```

```
    border-color: #00A8FF;
    color: #00A8FF;
    transform: translateY(-50%) scale(1.1);
}

.details-btn:active {
    transform: translateY(-50%) scale(0.95);
}

.details-btn i {
    font-size: 14px;
}

.log-card:hover .details-btn {
    opacity: 1;
}

.empty-state {
    text-align: center;
    padding: 60px 40px;
    border: 2px dashed rgba(255, 255, 255, 0.2);
    border-radius: 15px;
    color: #888;
    font-family: 'Inter', sans-serif;
    font-size: 16px;
    background: rgba(255, 255, 255, 0.02);
}

.empty-state p {
    margin: 0;
}

.create-btn {
    position: fixed;
    bottom: 30px;
    right: 50px;
    background: #00A8FF;
    color: white;
    font-family: 'Orbitron', sans-serif;
    font-size: 16px;
```

```
    font-weight: 600;
    padding: 15px 40px;
    border-radius: 50px;
    text-decoration: none;
    border: none;
    cursor: pointer;
    transition: all 0.3s ease;
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.4);
    z-index: 1000;
    display: inline-block;
}

.create-btn:hover {
    background: #008ecf;
    box-shadow: 0 0 30px rgba(0, 168, 255, 0.6);
    transform: translateY(-3px);
}

.create-btn:active {
    transform: translateY(-1px);
}

/* Details Modal Styles */
.details-modal {
    position: fixed;
    top: 0;
    left: 0;
    width: 100%;
    height: 100%;
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 2000;
}

.modal-overlay {
    position: absolute;
    top: 0;
    left: 0;
    width: 100%;
```

```
    height: 100%;
    background: rgba(0, 0, 0, 0.8);
    backdrop-filter: blur(5px);
}

.modal-content {
    position: relative;
    max-width: 600px;
    width: 90%;
    max-height: 90vh;
    overflow-y: auto;
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 25px;
    padding: 40px;
    z-index: 2001;
    box-shadow: 0 0 50px rgba(0, 168, 255, 0.3);
}

.modal-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 30px;
    padding-bottom: 20px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.modal-header-left {
    display: flex;
    align-items: center;
    gap: 12px;
    flex: 1;
}

.title-edit-group {
    display: inline-flex;
    align-items: center;
    gap: 2px !important;
```

```
}

.modal-workout-name-input {
    background: transparent;
    border: none;
    color: #00A8FF;
    font-family: 'Orbitron', sans-serif;
    font-size: 28px;
    font-weight: 700;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    padding: 0 !important;
    margin: 0 !important;
    flex: 1;
    min-width: 200px;
}

.title-edit-icon {
    margin: 0 !important;
    padding: 0 !important;
    font-size: 15px;
    color: #00c3ff !important;
    cursor: pointer;
    transform: translateY(1px);
    opacity: 0.6;
    transition: opacity 0.3s ease;
}

.title-edit-icon:hover {
    opacity: 1;
}

.modal-workout-name-input:focus {
    outline: none;
    text-shadow: 0 0 15px rgba(0, 168, 255, 0.8);
}

.modal-header-right {
    display: flex;
    align-items: center;
    gap: 15px;
```

```css
}

.modal-day-badge {
    font-family: 'Orbitron', sans-serif;
    font-size: 14px;
    font-weight: 600;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    padding: 6px 12px;
    background: rgba(255, 255, 255, 0.05);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 8px;
}

.modal-close-btn {
    width: 36px;
    height: 36px;
    border-radius: 50%;
    border: 2px solid rgba(255, 255, 255, 0.3);
    background: transparent;
    color: #fff;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: all 0.3s ease;
}

.modal-close-btn:hover {
    border-color: #ff4444;
    color: #ff4444;
    background: rgba(255, 68, 68, 0.1);
    transform: scale(1.1);
}

.modal-close-btn i {
    font-size: 16px;
}
```

```css
.modal-body {
    display: flex;
    flex-direction: column;
    gap: 30px;
}

.modal-workout-notes-section {
    display: flex;
    flex-direction: column;
    gap: 12px;
}

.modal-notes-header {
    display: flex;
    align-items: center;
    gap: 8px;
}

.modal-notes-label {
    font-family: 'Inter', sans-serif;
    font-size: 12px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.modal-workout-notes-textarea {
    width: 100%;
    background: rgba(0, 0, 0, 0.3);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 10px;
    padding: 15px;
    color: #fff;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    line-height: 1.6;
    min-height: 100px;
    resize: vertical;
    transition: all 0.3s ease;
```

```
}

.modal-workout-notes-textarea:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
}

.modal-exercises-section {
    display: flex;
    flex-direction: column;
    gap: 20px;
}

.modal-exercises-title {
    font-family: 'Orbitron', sans-serif;
    font-size: 18px;
    font-weight: 700;
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
    margin: 0;
}

.modal-exercises-list {
    display: flex;
    flex-direction: column;
    gap: 20px;
    max-height: 400px;
    overflow-y: auto;
    padding-right: 10px;
}

/* Exercise Block Styles */
.exercise-modal-block {
    background: rgba(255, 255, 255, 0.03);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    padding: 20px;
    display: flex;
    flex-direction: column;
```

```
    gap: 15px;
    transition: all 0.3s ease;
}

.exercise-modal-block:hover {
    background: rgba(255, 255, 255, 0.05);
    border-color: rgba(255, 255, 255, 0.15);
}

.exercise-header-editable {
    display: flex;
    align-items: center;
    gap: 10px;
    background: rgba(0, 168, 255, 0.1);
    border: 1px solid rgba(0, 168, 255, 0.3);
    border-radius: 10px;
    padding: 12px 15px;
}

.exercise-name-input {
    flex: 1;
    background: transparent;
    border: none;
    color: #00A8FF;
    font-family: 'Orbitron', sans-serif;
    font-size: 18px;
    font-weight: 700;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    padding: 0;
}

.exercise-name-input:focus {
    outline: none;
    text-shadow: 0 0 15px rgba(0, 168, 255, 0.8);
}

.exercise-stats-row {
    display: flex;
    gap: 20px;
}
```

```css
.stat-group {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.stat-label {
    font-family: 'Inter', sans-serif;
    font-size: 11px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.old-new-inputs {
    display: flex;
    gap: 8px;
}

.stat-input {
    flex: 1;
    background: #111;
    border: 1px solid #333;
    border-radius: 8px;
    padding: 8px 10px;
    color: #fff;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    transition: all 0.3s ease;
}

.stat-input:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
}
```

```css
.stat-input.old-value {
    background: rgba(255, 255, 255, 0.02);
    color: #888;
}

.exercise-weight-row {
    display: flex;
    gap: 20px;
}

.weight-group {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.weight-label {
    font-family: 'Inter', sans-serif;
    font-size: 11px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.weight-input-wrapper {
    display: flex;
    align-items: center;
    gap: 10px;
    background: #111;
    border: 1px solid #333;
    border-radius: 8px;
    padding: 8px 12px;
    transition: all 0.3s ease;
}

.weight-input-wrapper:focus-within {
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
```

```
}

.weight-input {
    flex: 1;
    background: transparent;
    border: none;
    color: #fff;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    padding: 0;
}

.weight-input:focus {
    outline: none;
}

.exercise-notes-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.exercise-notes-label {
    font-family: 'Inter', sans-serif;
    font-size: 11px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.exercise-notes-textarea {
    width: 100%;
    background: rgba(0, 0, 0, 0.3);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 8px;
    padding: 12px;
    color: #fff;
    font-family: 'Inter', sans-serif;
    font-size: 13px;
```

```
    line-height: 1.5;
    min-height: 60px;
    resize: vertical;
    transition: all 0.3s ease;
}

.exercise-notes-textarea:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
}

.edit-icon {
    color: #00A8FF;
    font-size: 14px;
    opacity: 0.6;
    transition: opacity 0.3s ease;
}

.edit-icon:hover {
    opacity: 1;
}

/* Custom scrollbar for exercises list */
.modal-exercises-list::-webkit-scrollbar {
    width: 6px;
}

.modal-exercises-list::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.05);
    border-radius: 10px;
}

.modal-exercises-list::-webkit-scrollbar-thumb {
    background: #00A8FF;
    border-radius: 10px;
}

.modal-exercises-list::-webkit-scrollbar-thumb:hover {
    background: #008ecf;
```

```css
}

/* Custom scrollbar for modal */
.modal-content::-webkit-scrollbar {
    width: 6px;
}

.modal-content::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.05);
    border-radius: 10px;
}

.modal-content::-webkit-scrollbar-thumb {
    background: #00A8FF;
    border-radius: 10px;
}

.modal-content::-webkit-scrollbar-thumb:hover {
    background: #008ecf;
}

/* Modal Footer */
.modal-footer {
    margin-top: 24px;
    padding-top: 20px;
    border-top: 1px solid rgba(255, 255, 255, 0.1);
    display: flex;
    justify-content: flex-end;
}

.modal-save-btn {
    padding: 10px 22px;
    border-radius: 999px;
    border: none;
    background: #00c3ff;
    color: #000;
    font-family: 'Orbitron', sans-serif;
    font-weight: 600;
    font-size: 14px;
    text-transform: uppercase;
```

```
    letter-spacing: 1px;
    cursor: pointer;
    box-shadow: 0 0 15px rgba(0, 195, 255, 0.6);
    transition: transform 0.15s ease, box-shadow 0.15s ease, background
0.15s ease;
}

.modal-save-btn:hover {
    transform: translateY(-1px);
    box-shadow: 0 0 20px rgba(0, 195, 255, 0.9);
    background: #27d9ff;
}

.modal-save-btn:active {
    transform: translateY(0);
    box-shadow: 0 0 10px rgba(0, 195, 255, 0.5);
}

/* Create Log Form Styles */
.create-log-container {
    max-width: 600px;
    margin: 0 auto;
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 25px;
    padding: 40px;
    margin-bottom: 50px;
}

.form-header {
    display: flex;
    flex-direction: row;
    gap: 24px;
    width: 100%;
    margin-bottom: 30px;
}

.form-header .form-group {
    flex: 1;
```

```
    display: flex;
    flex-direction: column;
}

.form-header .form-group input,
.form-header .form-group select {
    width: 100%;
    box-sizing: border-box;
}

.form-header .form-group select.form-input {
    font-size: 18px;
    padding: 15px 20px;
    padding-right: 40px; /* Keep space for dropdown arrow */
    font-weight: 600;
    background-position: right 20px center; /* Adjust arrow position for
larger padding */
}

.form-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
    margin-bottom: 15px;
}

.form-label {
    font-family: 'Inter', sans-serif;
    font-size: 12px;
    color: #888;
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.form-input {
    width: 100%;
    background: #111;
    border: 1px solid #333;
    border-radius: 8px;
```

```
    padding: 12px 15px;
    color: #fff;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    transition: all 0.3s ease;
    box-sizing: border-box;
}

.form-input:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
    background: #1a1a1a;
}

.form-input::placeholder {
    color: #555;
}

.form-input.large-input {
    font-size: 18px;
    padding: 15px 20px;
    font-weight: 600;
}

.form-input.small-input {
    padding: 10px 12px;
    font-size: 14px;
}

.form-textarea {
    resize: vertical;
    min-height: 80px;
    font-family: 'Inter', sans-serif;
    line-height: 1.5;
}

select.form-input {
    cursor: pointer;
    appearance: none;
```

```
    background-image: url("data:image/svg+xml,%3Csvg
xmlns='http://www.w3.org/2000/svg' width='12' height='12' viewBox='0 0 12
12'%3E%3Cpath fill='%2300A8FF' d='M6 9L1 4h10z'/%3E%3C/svg%3E");
    background-repeat: no-repeat;
    background-position: right 15px center;
    padding-right: 40px;
}

select.form-input option {
    background: #111;
    color: #fff;
}

.section-subtitle {
    font-family: 'Orbitron', sans-serif;
    font-size: 18px;
    color: #00A8FF;
    margin-bottom: 20px;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
}

.exercises-section {
    margin-bottom: 30px;
}

.exercise-entry {
    position: relative;
    background: rgba(255, 255, 255, 0.03);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    padding: 20px;
    margin-bottom: 20px;
    transition: all 0.3s ease;
}

.exercise-entry:hover {
    background: rgba(255, 255, 255, 0.05);
    border-color: rgba(255, 255, 255, 0.15);
}
```

```css
.remove-exercise {
    position: absolute;
    top: -10px;
    right: -10px;
    background: #ff4444;
    color: white;
    border: none;
    width: 24px;
    height: 24px;
    border-radius: 50%;
    font-size: 18px;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: 0.2s ease-in-out;
    z-index: 10;
    line-height: 1;
    padding: 0;
}

.remove-exercise:hover {
    background: #ff2222;
    transform: scale(1.1);
}

.inline-inputs {
    display: flex;
    gap: 16px;
    margin-top: 16px;
    width: 100%;
    margin-bottom: 15px;
}

.inline-inputs .input-group {
    flex: 1;
    display: flex;
    flex-direction: column;
    gap: 8px;
}
```

```
.inline-inputs .input-group .form-label {
    font-size: 11px;
    white-space: nowrap;
    min-height: 18px;
}

.inline-inputs .input-group input {
    width: 100%;
    box-sizing: border-box;
}

.add-exercise-btn {
    width: 50px;
    height: 50px;
    border-radius: 50%;
    background: rgba(0, 168, 255, 0.1);
    border: 2px solid #00A8FF;
    color: #00A8FF;
    cursor: pointer;
    display: flex;
    align-items: center;
    justify-content: center;
    margin: 20px auto;
    transition: all 0.3s ease;
    font-size: 20px;
}

.add-exercise-btn:hover {
    background: rgba(0, 168, 255, 0.2);
    transform: scale(1.1);
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.4);
}

.add-exercise-btn:active {
    transform: scale(0.95);
}

.save-log-btn {
    width: 100%;
```

```
    background: #00A8FF;
    color: white;
    font-family: 'Orbitron', sans-serif;
    font-size: 18px;
    font-weight: 700;
    padding: 18px 40px;
    border: none;
    border-radius: 12px;
    cursor: pointer;
    transition: all 0.3s ease;
    text-transform: uppercase;
    letter-spacing: 2px;
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.4);
    margin-top: 20px;
}

.save-log-btn:hover {
    background: #008ecf;
    box-shadow: 0 0 30px rgba(0, 168, 255, 0.6);
    transform: translateY(-2px);
}

.save-log-btn:active {
    transform: translateY(0);
}

/* --- AI NUTRITION COACH PAGE STYLES --- */

.coach-console {
    width: 90%;
    max-width: 1000px;
    height: 80vh;
    margin: 20px auto;
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    display: flex;
    flex-direction: column;
    padding: 30px;
```

```
}

.coach-header {
    margin-bottom: 20px;
}

.coach-header h1 {
    font-family: 'Orbitron', sans-serif;
    font-size: 32px;
    font-weight: 700;
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    margin: 0;
    display: flex;
    align-items: center;
}

.online-dot {
    width: 10px;
    height: 10px;
    background: #00ff00;
    border-radius: 50%;
    box-shadow: 0 0 10px #00ff00;
    display: inline-block;
    margin-left: 10px;
}

#coach-chat-window {
    flex-grow: 1;
    overflow-y: auto;
    padding: 30px;
    background: rgba(0, 0, 0, 0.6);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 15px;
    margin-bottom: 20px;
    display: flex;
    flex-direction: column;
    gap: 10px;
}
```

```css
/* Custom scrollbar for coach chat window */
#coach-chat-window::-webkit-scrollbar {
    width: 6px;
}

#coach-chat-window::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.05);
    border-radius: 10px;
}

#coach-chat-window::-webkit-scrollbar-thumb {
    background: #00A8FF;
    border-radius: 10px;
}

#coach-chat-window::-webkit-scrollbar-thumb:hover {
    background: #008ecf;
}

/* --- SETTINGS PAGE STYLES --- */

.settings-container {
    max-width: 900px;
    margin: 0 auto;
    padding: 40px 20px;
}

.settings-header {
    font-family: 'Orbitron', sans-serif;
    font-size: 36px;
    font-weight: 700;
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.5);
    margin-bottom: 40px;
    text-align: center;
}

.settings-form {
    width: 100%;
}
```

```css
.settings-grid {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 30px;
    margin-bottom: 30px;
}

.settings-card {
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    border: 1px solid rgba(255, 255, 255, 0.1);
    border-radius: 20px;
    padding: 30px;
    display: flex;
    flex-direction: column;
    gap: 20px;
}

.settings-card-title {
    font-family: 'Orbitron', sans-serif;
    font-size: 20px;
    font-weight: 700;
    color: #00A8FF;
    text-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
    margin: 0 0 10px 0;
    padding-bottom: 15px;
    border-bottom: 1px solid rgba(255, 255, 255, 0.1);
}

.settings-card .input-group {
    display: flex;
    flex-direction: column;
    gap: 8px;
}

.settings-card .input-group label {
    font-family: 'Inter', sans-serif;
    font-size: 12px;
    color: #888;
```

```
    text-transform: uppercase;
    letter-spacing: 1px;
    font-weight: 600;
}

.settings-input {
    width: 100%;
    background: #111;
    border: 1px solid #444;
    padding: 12px 15px;
    border-radius: 8px;
    color: white;
    font-family: 'Inter', sans-serif;
    font-size: 14px;
    transition: all 0.3s ease;
    box-sizing: border-box;
}

.settings-input:focus {
    outline: none;
    border-color: #00A8FF;
    box-shadow: 0 0 10px rgba(0, 168, 255, 0.3);
    background: #1a1a1a;
}

.settings-input::placeholder {
    color: #555;
}

.input-with-icon {
    position: relative;
    display: block;
}

.input-with-icon .settings-input {
    padding-right: 40px;
    width: 100%;
}

.edit-icon {
```

```
    position: absolute;
    right: 15px;
    top: 50%;
    transform: translateY(-50%);
    color: #00A8FF;
    cursor: pointer;
    font-size: 16px;
    opacity: 0.7;
    transition: all 0.3s ease;
    user-select: none;
    pointer-events: auto;
    line-height: 1;
    display: flex;
    align-items: center;
    justify-content: center;
}

.edit-icon:hover {
    opacity: 1;
    transform: translateY(-50%) scale(1.1);
    text-shadow: 0 0 8px rgba(0, 168, 255, 0.6);
}

.eye-icon {
    position: absolute;
    right: 15px;
    top: 50%;
    transform: translateY(-50%);
    color: #00A8FF;
    cursor: pointer;
    font-size: 18px;
    opacity: 0.7;
    transition: all 0.3s ease;
    user-select: none;
    pointer-events: auto;
    line-height: 1;
    display: flex;
    align-items: center;
    justify-content: center;
}
```

```css
.eye-icon:hover {
    opacity: 1;
    transform: translateY(-50%) scale(1.1);
    filter: drop-shadow(0 0 4px rgba(0, 168, 255, 0.6));
}

.settings-actions {
    margin-top: 30px;
    display: flex;
    justify-content: center;
}

.settings-save-btn {
    width: 100%;
    max-width: 500px;
    background: #00A8FF;
    color: white;
    padding: 15px 40px;
    border: none;
    border-radius: 8px;
    font-family: 'Orbitron', sans-serif;
    font-size: 16px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    text-transform: uppercase;
    letter-spacing: 1px;
    box-shadow: 0 0 20px rgba(0, 168, 255, 0.4);
}

.settings-save-btn:hover {
    background: #008ecf;
    box-shadow: 0 0 30px rgba(0, 168, 255, 0.6);
    transform: translateY(-2px);
}

.settings-save-btn:active {
    transform: translateY(0);
}
```

```
.settings-actions-bottom {
    margin-top: 40px;
    padding-top: 30px;
    border-top: 1px solid rgba(255, 255, 255, 0.1);
    display: flex;
    justify-content: center;
}

.logout-btn {
    background: transparent;
    color: #ff4444;
    border: 2px solid #ff4444;
    padding: 12px 30px;
    border-radius: 8px;
    font-family: 'Orbitron', sans-serif;
    font-size: 14px;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
    text-transform: uppercase;
    letter-spacing: 1px;
}

.logout-btn:hover {
    background: rgba(255, 68, 68, 0.1);
    box-shadow: 0 0 20px rgba(255, 68, 68, 0.5);
    transform: translateY(-2px);
    border-color: #ff0000;
    color: #ff0000;
}

.logout-btn:active {
    transform: translateY(0);
}

/* Responsive: Stack columns on smaller screens */
@media (max-width: 768px) {
    .settings-grid {
        grid-template-columns: 1fr;
```

```
        gap: 20px;
    }

    .settings-container {
        padding: 20px 15px;
    }

    .settings-header {
        font-size: 28px;
        margin-bottom: 30px;
    }
}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Calories</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
   <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories" class="active">Calories</a></li>
            <li><a href="/workout">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
            <li><a href="/settings">Settings</a></li>
        </ul>
   </nav>
   <main class="main-content">
```

```
        <div class="tracker-container">
            <!-- LEFT COLUMN: Daily Log -->
            <div class="log-section">
                <div style="display: flex; justify-content: space-between;
align-items: center; margin-bottom: 20px;">
                    <h2 class="section-title" style="margin: 0;">Daily
Log</h2>
                    <div class="date-display" id="current-date"></div>
                </div>

                <!-- Macro Summary Row -->
                <div class="macro-summary">
                    <div class="macro-box">
                        <div class="macro-label">Protein</div>
                        <div class="macro-value"
id="total-protein">0g</div>
                    </div>
                    <div class="macro-box">
                        <div class="macro-label">Carbs</div>
                        <div class="macro-value" id="total-carbs">0g</div>
                    </div>
                    <div class="macro-box">
                        <div class="macro-label">Fats</div>
                        <div class="macro-value" id="total-fats">0g</div>
                    </div>
                    <div class="macro-box">
                        <div class="macro-label">Calories</div>
                        <div class="macro-value"
id="total-calories">0</div>
                    </div>
                    <div class="macro-box">
                        <div class="macro-label">Surplus</div>
                        <div class="macro-value"
id="total-surplus">0</div>
                    </div>
                </div>

                <!-- Food Items Feed -->
                <div class="food-feed" id="food-feed">
                    <!-- Food items will be dynamically added here -->
```

```
                </div>
            </div>

            <!-- RIGHT COLUMN: AI Chat Interface -->
            <div class="chat-section calories-chat">
                <h2 class="section-title">AI Nutrition Coach</h2>
                <div class="chat-window calories-chat-window"></div>
                <div class="chat-input-area">
                    <input type="text" placeholder="Type a message..."
class="chat-input calories-chat-input" autocomplete="off">
                    <button type="button" class="log-btn">LOG IT</button>
                </div>
            </div>
        </div>
    </main>
    <script>
        // Get current user from localStorage
        const storedUser = localStorage.getItem('athleticore_user');
        const currentUser = storedUser ? JSON.parse(storedUser) : null;

        if (!currentUser) {
            window.location.href = '/login';
        }

        // Initialize state variables
        let totalProtein = 0;
        let totalCarbs = 0;
        let totalFat = 0;
        let totalCalories = 0;
        let goalCalories = null; // Will be set from API
        let surplus = 0; // Always starts at 0 (green) each day
        let chatHistory = [];
        let currentDate = new Date().toISOString().split('T')[0]; //
YYYY-MM-DD format

        // DOM elements (will be set in DOMContentLoaded)
        let chatWindow, chatInput, logBtn, foodFeed;

        // Function to get today's date in YYYY-MM-DD format
        function getTodayDate() {
```

```
            return new Date().toISOString().split('T')[0];
        }

        // Function to format date for display
        function formatDateDisplay(dateString) {
            const date = new Date(dateString + 'T00:00:00');
            const options = { weekday: 'short', year: 'numeric', month:
'short', day: 'numeric' };
            return date.toLocaleDateString('en-US', options);
        }

        // Function to check if date has changed and reset if needed
        function checkDateReset() {
            const today = getTodayDate();
            if (today !== currentDate) {
                // Date has changed - reset everything
                currentDate = today;
                totalProtein = 0;
                totalCarbs = 0;
                totalFat = 0;
                totalCalories = 0;
                surplus = 0;
                chatHistory = [];

                // Clear the food feed
                if (foodFeed) {
                    foodFeed.innerHTML = '';
                }

                // Clear the chat window
                if (chatWindow) {
                    chatWindow.innerHTML = '';
                    appendMessage('Hi! I\'m your AI Nutrition Coach. Tell
me what you ate and I\'ll log it for you!', 'bot');
                }

                // Update macro displays
                updateMacroDisplays();

                // Update date display
```

```
                updateDateDisplay();

                // Reload today's logs
                loadTodayLogs();

                console.log('Date reset: New day started!');
            }
        }

        // Function to update date display
        function updateDateDisplay() {
            const dateDisplay = document.getElementById('current-date');
            if (dateDisplay) {
                dateDisplay.textContent = formatDateDisplay(currentDate);
            }
        }

        // Function to calculate surplus
        // Surplus = (Total Calories - Goal Calories)
        // If negative (under goal), return 0 (green).
        // If positive (over goal), return the difference (red).
        function calculateSurplus() {
            if (goalCalories === null || goalCalories === undefined) {
                return 0; // No goal set, show 0 (green)
            }
            const difference = totalCalories - goalCalories;
            // Always return 0 or positive value (never negative)
            return difference > 0 ? difference : 0;
        }

        // Function to update macro displays
        function updateMacroDisplays() {
            const proteinEl = document.getElementById('total-protein');
            const carbsEl = document.getElementById('total-carbs');
            const fatsEl = document.getElementById('total-fats');
            const caloriesEl = document.getElementById('total-calories');
            const surplusEl = document.getElementById('total-surplus');

            if (proteinEl) proteinEl.textContent =
Math.round(totalProtein) + 'g';
```

```
            if (carbsEl) carbsEl.textContent = Math.round(totalCarbs) +
'g';
            if (fatsEl) fatsEl.textContent = Math.round(totalFat) + 'g';
            if (caloriesEl) caloriesEl.textContent =
Math.round(totalCalories);

            // Update surplus - always recalculate to ensure accuracy
            if (surplusEl) {
                surplus = calculateSurplus();
                surplusEl.textContent = Math.round(surplus);

                // Color logic: Green when 0 (at or under goal), Red when
> 0 (over goal)
                // Use setProperty with important flag to override CSS
!important rules
                if (surplus > 0) {
                    surplusEl.style.setProperty('color', '#ff0000',
'important'); // Red - exceeded goal
                } else {
                    surplusEl.style.setProperty('color', '#00ff00',
'important'); // Green - at or under goal
                }
            }
        }

        // Function to append message to chat window
        function appendMessage(text, role = 'bot') {
            if (!chatWindow) {
                console.error('Chat window element not found');
                return;
            }

            const messageEl = document.createElement('div');
            messageEl.classList.add('message', role === 'user' ?
'user-msg' : 'bot-msg');
            messageEl.textContent = text;
            messageEl.style.opacity = '1';
            messageEl.style.visibility = 'visible';
            chatWindow.appendChild(messageEl);
            chatWindow.scrollTop = chatWindow.scrollHeight;
```

```
        }

        // Function to update macro totals and display
        function updateMacros(foodItem) {
            console.log('Updating macros with:', foodItem);

            const protein = parseFloat(foodItem.protein) || 0;
            const carbs = parseFloat(foodItem.carbs) || 0;
            const fat = parseFloat(foodItem.fat) || 0;
            const calories = parseFloat(foodItem.calories) || 0;

            totalProtein += protein;
            totalCarbs += carbs;
            totalFat += fat;
            totalCalories += calories;

            // Update macro displays using the helper function
            updateMacroDisplays();

            console.log('Macros updated - Protein:', totalProtein,
'Carbs:', totalCarbs, 'Fat:', totalFat, 'Calories:', totalCalories);
        }

        // Function to handle delete action
        async function handleDelete(logId, buttonElement) {
            if (!logId) {
                console.error('No log ID provided for deletion');
                return;
            }

            // Get the food item element (parent of the button)
            const foodItemEl = buttonElement.closest('.food-item');
            if (!foodItemEl) {
                console.error('Food item element not found');
                return;
            }

            // Get food name from the element
            const foodNameEl = foodItemEl.querySelector('.food-name');
```

```
            const foodName = foodNameEl ? foodNameEl.textContent.trim() :
'Food item';

            // Get macro values from data attributes
            const calories =
parseFloat(foodItemEl.getAttribute('data-calories')) || 0;
            const protein =
parseFloat(foodItemEl.getAttribute('data-protein')) || 0;
            const carbs =
parseFloat(foodItemEl.getAttribute('data-carbs')) || 0;
            const fat = parseFloat(foodItemEl.getAttribute('data-fat')) ||
0;

            // Optimistic UI update: Remove the item immediately
            foodItemEl.style.transition = 'opacity 0.3s ease, transform
0.3s ease';
            foodItemEl.style.opacity = '0';
            foodItemEl.style.transform = 'translateX(-20px)';

            // Remove from DOM after animation
            setTimeout(() => {
                foodItemEl.remove();
            }, 300);

            // Subtract macros from totals
            totalProtein = Math.max(0, totalProtein - protein);
            totalCarbs = Math.max(0, totalCarbs - carbs);
            totalFat = Math.max(0, totalFat - fat);
            totalCalories = Math.max(0, totalCalories - calories);

            // Update displays (including surplus)
            updateMacroDisplays();

            // Automatically add "Deleted [Food Name]" message to chat
            const deleteMessage = `Deleted ${foodName}`;
            appendMessage(deleteMessage, 'bot');
            chatHistory.push({ role: 'assistant', content: deleteMessage
});

            // Call API to delete from database
```

```
            try {
                const response = await fetch(`/api/calories/log/${logId}`, {
                    method: 'DELETE',
                    headers: {
                        'Content-Type': 'application/json'
                    }
                });

                if (!response.ok) {
                    const errorData = await response.json().catch(() => ({}));
                    throw new Error(errorData.error || 'Failed to delete log entry');
                }

                console.log('Food item deleted successfully:', logId);
            } catch (error) {
                console.error('Error deleting food item:', error);
                // If deletion fails, we could reload the page or show an error message
                // For now, we'll just log it since we already removed it from UI
                appendMessage('Error: Could not delete entry from database. Please refresh the page.', 'bot');
            }
        }

        // Function to add food item to the feed
        function addFoodItem(foodItem) {
            if (!foodFeed) {
                console.error('Food feed element not found');
                return;
            }

            const foodItemEl = document.createElement('div');
            foodItemEl.classList.add('food-item');

            // Store the log ID as a data attribute for deletion
            if (foodItem.id) {
```

```
                foodItemEl.setAttribute('data-log-id', foodItem.id);
            }

            // Store macro values as data attributes for deletion
calculations
            foodItemEl.setAttribute('data-calories', foodItem.calories ||
0);
            foodItemEl.setAttribute('data-protein', foodItem.protein ||
0);
            foodItemEl.setAttribute('data-carbs', foodItem.carbs || 0);
            foodItemEl.setAttribute('data-fat', foodItem.fat || 0);

            const foodName = foodItem.name || 'Food Item';
            const calories = Math.round(foodItem.calories || 0);
            const protein = Math.round(foodItem.protein || 0);
            const carbs = Math.round(foodItem.carbs || 0);
            const fat = Math.round(foodItem.fat || 0);

            // Only show delete button if we have a log ID
            const deleteButtonHtml = foodItem.id ? `
                <button class="food-delete-btn"
onclick="handleDelete(${foodItem.id}, this)" title="Delete this entry">
                    <svg width="14" height="14" viewBox="0 0 16 16"
fill="none" xmlns="http://www.w3.org/2000/svg">
                        <path d="M12 4L4 12M4 4L12 12"
stroke="currentColor" stroke-width="2" stroke-linecap="round"/>
                    </svg>
                </button>
            ` : '';

            foodItemEl.innerHTML = `
                <div class="food-item-main">
                    <div class="food-name">${foodName}</div>
                    <div class="food-calories-container">
                        <div class="food-calories">${calories} kcal</div>
                        ${deleteButtonHtml}
                    </div>
                </div>
                <div class="food-macros">
```

```
                    <span class="macro-badge protein">P:
${protein}g</span>
                    <span class="macro-badge carbs">C: ${carbs}g</span>
                    <span class="macro-badge fat">F: ${fat}g</span>
                </div>
            `;

            foodFeed.appendChild(foodItemEl);

            // Scroll to bottom to show new item
            foodFeed.scrollTop = foodFeed.scrollHeight;

            console.log('Food item added:', foodName, calories, 'kcal',
'P:', protein, 'C:', carbs, 'F:', fat);
        }

        // Function to send message to backend
        async function sendMessage() {
            if (!chatInput || !logBtn) {
                console.error('Chat input or button not found');
                return;
            }

            const message = chatInput.value.trim();
            if (!message) {
                return;
            }

            if (!currentUser?.username) {
                appendMessage('Please log in again.', 'bot');
                return;
            }

            // Add user message to chat
            appendMessage(message, 'user');
            chatHistory.push({ role: 'user', content: message });

            const messageToSend = message;
            chatInput.value = '';
            logBtn.disabled = true;
```

```
            try {
                const response = await fetch('/api/calories/chat', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        message: messageToSend,
                        username: currentUser.username,
                        history: chatHistory
                    })
                });

                if (!response.ok) {
                    let errorMessage = 'Failed to process message';
                    try {
                        const errorData = await response.json();
                        errorMessage = errorData.error || errorMessage;
                    } catch (e) {
                        errorMessage = `Server error: ${response.status}
${response.statusText}`;
                    }
                    throw new Error(errorMessage);
                }

                let data;
                try {
                    data = await response.json();
                    console.log('Response data:', data); // Debug log
                } catch (e) {
                    console.error('Failed to parse JSON response:', e);
                    throw new Error('Invalid response from server');
                }

                // Add bot reply to chat
                const replyText = data.reply || 'I\'ve logged that for
you!';
                appendMessage(replyText, 'bot');
```

```
                chatHistory.push({ role: 'assistant', content: replyText
});

                // If food data is returned, add it to the feed and update
macros
                if (data.food_data) {
                    console.log('Food data received:', data.food_data); //
Debug log
                    try {
                        addFoodItem(data.food_data);
                        updateMacros(data.food_data);
                        console.log('Successfully updated feed and
macros'); // Debug log
                    } catch (err) {
                        console.error('Error updating feed/macros:', err);
                    }
                } else {
                    console.log('No food_data in response. Nutrition
result may not be ready yet.'); // Debug log
                }
            } catch (error) {
                console.error('Error:', error);
                const errorMessage = error.message || 'Sorry, there was an
error. Please try again.';
                appendMessage(`Error: ${errorMessage}`, 'bot');
            } finally {
                logBtn.disabled = false;
                chatInput.focus();
            }
        }

        // Function to fetch goal calories from TDEE profile
        async function fetchGoalCalories() {
            if (!currentUser?.id) {
                return;
            }

            try {
                // Goal calories will be included in the loadTodayLogs
response
```

```
                // But we can also fetch it separately if needed
                const response = await
fetch(`/api/tdee/profile/${currentUser.id}`);
                if (response.ok) {
                    const data = await response.json();
                    if (data.profile && data.profile.goal_calories) {
                        goalCalories = data.profile.goal_calories;
                        updateMacroDisplays(); // Update surplus display
                    }
                }
            } catch (error) {
                console.error('Failed to fetch goal calories:', error);
            }
        }

        // Function to load today's existing logs
        async function loadTodayLogs() {
            if (!currentUser?.id) {
                return;
            }

            try {
                const response = await
fetch(`/api/calories/logs/${currentUser.id}/today`);
                if (!response.ok) {
                    return;
                }

                const data = await response.json();
                const logs = data.logs || [];

                // Update goal calories and total calories from API
response
                if (data.goal_calories !== undefined && data.goal_calories
!== null) {
                    goalCalories = data.goal_calories;
                }
                if (data.total_calories !== undefined) {
                    totalCalories = data.total_calories || 0;
                }
```

```
                // Don't set surplus from API - always recalculate it to
ensure accuracy
                // Surplus will be calculated in updateMacroDisplays()

                // Add each log to the feed and update macros
                if (logs.length > 0) {
                    logs.forEach(log => {
                        if (log.calories && log.description) {
                            const foodItem = {
                                id: log.id, // Include log ID for deletion
                                name: log.description,
                                calories: log.calories || 0,
                                protein: log.protein_g || 0,
                                carbs: log.carbs_g || 0,
                                fat: log.fat_g || 0
                            };
                            addFoodItem(foodItem);
                            // Don't call updateMacros here - we already
have totals from API
                            // Just add the visual item
                        }
                    });

                    // Recalculate macros from logs to ensure consistency
                    totalProtein = 0;
                    totalCarbs = 0;
                    totalFat = 0;
                    logs.forEach(log => {
                        if (log.protein_g) totalProtein += log.protein_g;
                        if (log.carbs_g) totalCarbs += log.carbs_g;
                        if (log.fat_g) totalFat += log.fat_g;
                    });

                    appendMessage(`Loaded ${logs.length} food item(s) from
today!`, 'bot');
                } else {
                    console.log('No logs found for today');
                }

                // Update all displays including surplus
```

```
                updateMacroDisplays();
            } catch (error) {
                console.error('Failed to load today\'s logs:', error);
            }
        }

        // Initialize when page loads
        document.addEventListener('DOMContentLoaded', () => {
            // Get DOM elements
            chatWindow = document.querySelector('.calories-chat-window');
            chatInput = document.querySelector('.calories-chat-input');
            logBtn = document.querySelector('.log-btn');
            foodFeed = document.getElementById('food-feed');

            // Check if elements exist
            if (!chatWindow || !chatInput || !logBtn || !foodFeed) {
                console.error('Missing DOM elements:', {
                    chatWindow: !!chatWindow,
                    chatInput: !!chatInput,
                    logBtn: !!logBtn,
                    foodFeed: !!foodFeed
                });
                return;
            }

            // Initialize current date
            currentDate = getTodayDate();
            updateDateDisplay();

            // Initialize surplus to 0 (green) on page load
            surplus = 0;
            updateMacroDisplays(); // This will set the color to green

            // Check for date reset
            checkDateReset();

            // Add welcome message
            appendMessage('Hi! I\'m your AI Nutrition Coach. Tell me what
you ate and I\'ll log it for you!', 'bot');
```

```
            // Set up event listeners
        logBtn.addEventListener('click', sendMessage);
        chatInput.addEventListener('keydown', (event) => {
            if (event.key === 'Enter') {
                event.preventDefault();
                sendMessage();
            }
            });

            // Load existing logs (this will also fetch goal calories)
            loadTodayLogs();

            // Also fetch goal calories separately as backup
            fetchGoalCalories();

            // Check for date change every minute (in case user keeps page
open past midnight)
            setInterval(checkDateReset, 60000); // Check every minute
        });
    </script>
</body>
</html>



<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Nutrition Coach - Athleticore.AI</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
    <nav class="top-nav">
```

```
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout">Workout</a></li>
            <li><a href="/coach" class="active">Coach</a></li>
            <li><a href="/settings">Settings</a></li>
        </ul>
    </nav>

    <main class="main-content">
        <div class="coach-console">
            <div class="coach-header">
                <h1>AI Nutrition Coach<span
class="online-dot"></span></h1>
            </div>

            <div class="chat-window" id="coach-chat-window"></div>

            <div class="chat-input-area">
                <input type="text" id="coach-input" class="chat-input"
placeholder="Ask me anything..." autocomplete="off">
                <button type="button" id="coach-send-btn"
class="send-btn">Send</button>
            </div>
        </div>
    </main>

    <script>
        // Select elements
        const chatWindow = document.getElementById('coach-chat-window');
        const chatInput = document.getElementById('coach-input');
        const sendBtn = document.getElementById('coach-send-btn');

        // Get user from localStorage
        let currentUser = null;
```

```
        // Function to get user from localStorage
        function getUser() {
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                return null;
            }
            try {
                return JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                return null;
            }
        }

        // Function to append message to chat window
        function appendMessage(text, role = 'bot') {
            const messageEl = document.createElement('div');
            messageEl.classList.add('message', role === 'user' ?
'user-msg' : 'bot-msg');
            messageEl.textContent = text;
            chatWindow.appendChild(messageEl);
            chatWindow.scrollTop = chatWindow.scrollHeight;
        }

        // Function to load chat history
        async function loadChatHistory() {
            if (!currentUser || !currentUser.id) {
                return;
            }

            try {
                const response = await
fetch(`/api/chat/history?user_id=${currentUser.id}`);

                if (!response.ok) {
                    throw new Error('Failed to load chat history');
                }

                const data = await response.json();
                const messages = data.messages || [];
```

```
                // Clear chat window
                chatWindow.innerHTML = '';

                // Append all messages
                messages.forEach(msg => {
                    const role = msg.sender === 'user' ? 'user' : 'bot';
                    appendMessage(msg.message, role);
                });
            } catch (error) {
                console.error('Error loading chat history:', error);
            }
        }

        // Function to send message
        async function sendMessage() {
            const message = chatInput.value.trim();
            if (!message) {
                return;
            }

            // Check if user is logged in
            if (!currentUser || !currentUser.id) {
                appendMessage('Please log in first to use the AI coach.',
'bot');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 2000);
                return;
            }

            // Append user message immediately
            appendMessage(message, 'user');
            chatInput.value = '';
            sendBtn.disabled = true;

            try {
                // POST request to API
                const response = await fetch('/api/chat/send', {
                    method: 'POST',
```

```
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify({
                        user_id: currentUser.id,
                        message: message
                    })
                });

                if (!response.ok) {
                    const errorData = await response.json();
                    throw new Error(errorData.error || 'Network response was not ok');
                }

                const data = await response.json();

                // Append AI response
                if (data.ai_message && data.ai_message.message) {
                    appendMessage(data.ai_message.message, 'bot');
                } else {
                    appendMessage('Sorry, I did not receive a response.', 'bot');
                }
            } catch (error) {
                console.error('Error:', error);
                appendMessage('There was an error reaching the coach. Please try again.', 'bot');
            } finally {
                sendBtn.disabled = false;
                chatInput.focus();
            }
        }

        // Initialize on page load
        document.addEventListener('DOMContentLoaded', () => {
            currentUser = getUser();

            if (!currentUser || !currentUser.id) {
```

```
                appendMessage('Please log in to start chatting with your AI coach.', 'bot');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 2000);
                return;
            }

            // Load chat history
            loadChatHistory();
        });

        // Event listeners
        sendBtn.addEventListener('click', sendMessage);

        chatInput.addEventListener('keydown', (event) => {
            if (event.key === 'Enter') {
                event.preventDefault();
                sendMessage();
            }
        });
    </script>
</body>
</html>



<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Create Workout Log - Athleticore</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=Orbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
```

```
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
    <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout" class="active">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
            <li><a href="/settings">Settings</a></li>
        </ul>
    </nav>
    <main class="main-content">
        <h2 class="section-title">Create Workout Log</h2>

        <div class="create-log-container">
            <form id="workout-log-form">
                <!-- Form Header -->
                <div class="form-header">
                    <div class="form-group">
                        <label for="log-name" class="form-label">Log
Name</label>
                        <input type="text" id="log-name" name="log-name"
class="form-input large-input" placeholder="e.g., Chest Day" required>
                    </div>

                    <div class="form-group">
                        <label for="day-of-week" class="form-label">Day of
Week</label>
                        <select id="day-of-week" name="day-of-week"
class="form-input" required>
                            <option value="">Select Day</option>
                            <option value="Monday">Monday</option>
                            <option value="Tuesday">Tuesday</option>
                            <option value="Wednesday">Wednesday</option>
                            <option value="Thursday">Thursday</option>
```

```
                            <option value="Friday">Friday</option>
                            <option value="Saturday">Saturday</option>
                            <option value="Sunday">Sunday</option>
                        </select>
                    </div>
                </div>

                <!-- Exercise Entry Container -->
                <div class="exercises-section">
                    <h3 class="section-subtitle">Exercises</h3>
                    <div id="exercises-list">
                        <!-- First exercise entry -->
                        <div class="exercise-entry">
                            <button type="button" class="remove-exercise"
onclick="removeExercise(this)">x</button>
                            <div class="form-group">
                                <label class="form-label">Exercise
Name</label>
                                <input type="text" class="form-input"
name="exercise-name" placeholder="e.g., Bench Press" required>
                            </div>

                            <div class="inline-inputs">
                                <div class="input-group">
                                    <label class="form-label">Weight
(kg)</label>
                                    <input type="number" class="form-input
small-input" name="weight" placeholder="0" min="0" step="0.5">
                                </div>
                                <div class="input-group">
                                    <label class="form-label">Prev Weight
(kg)</label>
                                    <input type="number" class="form-input
small-input" name="prev-weight" placeholder="0" min="0" step="0.5">
                                </div>
                                <div class="input-group">
                                    <label class="form-label">Reps</label>
                                    <input type="number" class="form-input
small-input" name="reps" placeholder="0" min="0">
                                </div>
```

```
                                    <div class="input-group">
                                        <label class="form-label">Sets</label>
                                        <input type="number" class="form-input
small-input" name="sets" placeholder="0" min="0">
                                    </div>
                                </div>

                                <div class="form-group">
                                    <label class="form-label">Notes</label>
                                    <textarea class="form-input form-textarea"
name="notes" placeholder="Add any notes about this exercise..."
rows="3"></textarea>
                                </div>
                            </div>
                        </div>

                        <!-- Add Exercise Button -->
                        <button type="button" class="add-exercise-btn"
onclick="addExerciseField()">
                            <i class="fas fa-plus"></i>
                        </button>
                    </div>

                    <!-- Save Button -->
                    <button type="submit" class="save-log-btn">Save
Log</button>
                </form>
            </div>
        </main>

        <!-- Toast Notification Container -->
        <div class="toast-container" id="toast-container"></div>

        <script>
            // Toast notification function
            function showToast(message, type = 'success') {
                const container = document.getElementById('toast-container');
                if (!container) return;

                const toast = document.createElement('div');
```

```
            toast.className = `toast ${type}`;

            const icon = type === 'success' ? '✓' : '✕';
            toast.innerHTML = `
                <span class="toast-icon">${icon}</span>
                <span class="toast-message">${message}</span>
            `;

            container.appendChild(toast);

            // Trigger animation
            setTimeout(() => {
                toast.classList.add('show');
            }, 10);

            // Remove toast after 3 seconds
            setTimeout(() => {
                toast.classList.remove('show');
                setTimeout(() => {
                    if (toast.parentNode) {
                        toast.parentNode.removeChild(toast);
                    }
                }, 300);
            }, 3000);
        }
    </script>
    <script>
        // Function to remove an exercise field
        function removeExercise(btn) {
            const exerciseEntry = btn.closest('.exercise-entry');
            if (exerciseEntry) {
                exerciseEntry.remove();
            }
        }

        // Function to add a new exercise field
        function addExerciseField() {
            const exercisesList =
document.getElementById('exercises-list');
            const newExercise = document.createElement('div');
```

```
            newExercise.className = 'exercise-entry';

            newExercise.innerHTML = `
                <button type="button" class="remove-exercise"
onclick="removeExercise(this)">×</button>
                <div class="form-group">
                    <label class="form-label">Exercise Name</label>
                    <input type="text" class="form-input"
name="exercise-name" placeholder="e.g., Bench Press" required>
                </div>

                <div class="inline-inputs">
                    <div class="input-group">
                        <label class="form-label">Weight (kg)</label>
                        <input type="number" class="form-input
small-input" name="weight" placeholder="0" min="0" step="0.5">
                    </div>
                    <div class="input-group">
                        <label class="form-label">Prev Weight (kg)</label>
                        <input type="number" class="form-input
small-input" name="prev-weight" placeholder="0" min="0" step="0.5">
                    </div>
                    <div class="input-group">
                        <label class="form-label">Reps</label>
                        <input type="number" class="form-input
small-input" name="reps" placeholder="0" min="0">
                    </div>
                    <div class="input-group">
                        <label class="form-label">Sets</label>
                        <input type="number" class="form-input
small-input" name="sets" placeholder="0" min="0">
                    </div>
                </div>

                <div class="form-group">
                    <label class="form-label">Notes</label>
                    <textarea class="form-input form-textarea"
name="notes" placeholder="Add any notes about this exercise..."
rows="3"></textarea>
                </div>
```

```
            `;

            exercisesList.appendChild(newExercise);
        }

        // Helper function to convert day of week to ISO date
        function getDateForDayOfWeek(dayOfWeek) {
            const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
'Thursday', 'Friday', 'Saturday'];
            const today = new Date();
            const currentDay = today.getDay();
            const targetDay = days.indexOf(dayOfWeek);

            // Calculate days difference
            let daysDiff = targetDay - currentDay;
            if (daysDiff < 0) {
                daysDiff += 7; // If target day has passed this week, use
next week
            }

            const targetDate = new Date(today);
            targetDate.setDate(today.getDate() + daysDiff);

            // Return ISO date string (YYYY-MM-DD)
            return targetDate.toISOString().split('T')[0];
        }

        // Handle form submission

document.getElementById('workout-log-form').addEventListener('submit',
async function(e) {
            e.preventDefault();

            // Get user from localStorage
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                showToast('Please log in first', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
```

```
                return;
            }

            let user;
            try {
                user = JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                showToast('Session expired. Please log in again.',
'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            if (!user || !user.id) {
                showToast('User information not found. Please log in
again.', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            // Get form data
            const workoutName =
document.getElementById('log-name').value.trim();
            const dayOfWeek =
document.getElementById('day-of-week').value;

            if (!workoutName || !dayOfWeek) {
                showToast('Please fill in all required fields', 'error');
                return;
            }

            // Convert day of week to ISO date
            const logDate = getDateForDayOfWeek(dayOfWeek);

            // Collect all exercises
```

```
            const exerciseEntries = document.querySelectorAll('.exercise-entry');
            const exercises = [];

            exerciseEntries.forEach((entry, index) => {
                const exerciseName = entry.querySelector('input[name="exercise-name"]').value.trim();
                const weight = entry.querySelector('input[name="weight"]').value;
                const prevWeight = entry.querySelector('input[name="prev-weight"]').value;
                const reps = entry.querySelector('input[name="reps"]').value;
                const sets = entry.querySelector('input[name="sets"]').value;
                const notes = entry.querySelector('textarea[name="notes"]').value.trim();

                if (exerciseName) { // Only add if exercise name is provided
                    // Parse sets and reps - they are required
                    const setsStr = sets ? sets.trim() : '';
                    const repsStr = reps ? reps.trim() : '';
                    const setsValue = setsStr ? parseInt(setsStr) : null;
                    const repsValue = repsStr ? parseInt(repsStr) : null;

                    // Validate required fields
                    if (!setsStr || setsValue === null || isNaN(setsValue) || setsValue <= 0) {
                        showToast(`Exercise "${exerciseName}": Sets is required and must be greater than 0`, 'error');
                        return;
                    }

                    if (!repsStr || repsValue === null || isNaN(repsValue) || repsValue <= 0) {
                        showToast(`Exercise "${exerciseName}": Reps is required and must be greater than 0`, 'error');
                        return;
                    }
```

```
                    exercises.push({
                        exercise_name: exerciseName, // API expects
exercise_name
                        weight_kg: weight && weight.trim() ?
parseFloat(weight) : 0.0,
                        previous_weight: prevWeight && prevWeight.trim() ?
parseFloat(prevWeight) : 0.0, // API expects previous_weight
                        reps: repsValue,
                        sets: setsValue,
                        order_index: index, // Add order_index
                        notes: notes || null
                    });
                }
            });

            if (exercises.length === 0) {
                showToast('Please add at least one exercise', 'error');
                return;
            }

            // Build final JSON object matching API expectations
            const workoutData = {
                user_id: user.id,
                workout_name: workoutName, // API expects workout_name
                log_date: logDate, // API expects log_date (ISO format)
                notes: null, // Optional, can be added later if needed
                exercises: exercises
            };

            // Debug: Log the data being sent
            console.log('Sending workout data:',
JSON.stringify(workoutData, null, 2));

            // Send to backend API
            try {
                const response = await fetch('/api/workouts', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
```

```
                    },
                    body: JSON.stringify(workoutData)
                });

                let data;
                try {
                    data = await response.json();
                } catch (parseError) {
                    console.error('Failed to parse response:',
parseError);
                    const text = await response.text();
                    console.error('Response text:', text);
                    alert('Error: Server returned invalid response.
Status: ' + response.status);
                    return;
                }

                console.log('API Response:', data);

                if (!response.ok) {
                    // Show the actual error message from the server
                    const errorMsg = data.error || 'Failed to create
workout';
                    console.error('API Error:', errorMsg, 'Status:',
response.status);

                    // Show a more user-friendly error message
                    let userMessage = errorMsg;
                    if (errorMsg.includes('no such table')) {
                        userMessage = 'Database not initialized. Please
contact support.';
                    } else if (errorMsg.includes('Database error')) {
                        userMessage = 'Database error occurred. Please try
again.';
                    }

                    showToast(userMessage, 'error');
                    return;
                }
```

```
                // Success - show toast and redirect to workout page
                showToast('Workout log created successfully!', 'success');
                setTimeout(() => {
                    window.location.href = '/workout';
                }, 1500);
            } catch (error) {
                console.error('Error creating workout:', error);
                showToast(error.message || 'Failed to create workout.
Please try again.', 'error');
            }
        });
    </script>
</body>
</html>



<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Login</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
    <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard" class="active">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
```

```
            <li><a href="/settings">Settings</a></li>
        </ul>
    </nav>
    <main class="main-content">
        <div class="hero-text">
            <h1 class="typing-effect" id="welcome-heading">Welcome Back,
Athlete.</h1>
            <p class="subtitle">Your data is ready.</p>
        </div>

        <div class="dashboard-cards">
            <div class="card" data-tilt>
                <h3>Calories Left</h3>
                <div class="progress-container" style="width: 160px;
height: 160px;">
                    <svg class="progress-ring" width="160" height="160">
                        <circle class="progress-ring__circle-bg"
                            stroke="#333" stroke-width="10"
fill="transparent"
                            r="70" cx="80" cy="80"/>

                        <circle class="progress-ring__circle"
                            stroke="#00A8FF" stroke-width="10"
fill="transparent"
                            r="70" cx="80" cy="80"/>
                    </svg>

                    <div class="progress-text">
                        <span class="cal-value"
id="calories-left-value">0</span>
                        <span class="cal-label">kcal</span>
                    </div>
                </div>
            </div>

            <div class="card" data-tilt>
                <h3>Current Goal</h3>
                <div class="big-number" id="current-goal">-</div>
            </div>
```

```
            <div class="card" data-tilt>
                <h3>Next Workout</h3>
                <div class="big-number" id="next-workout">-</div>
            </div>
        </div>
    </main>
    <script
src="https://cdnjs.cloudflare.com/ajax/libs/vanilla-tilt/1.7.0/vanilla-til
t.min.js"></script>
    <script>
    // Get user from localStorage
    const storedUser = localStorage.getItem('athleticore_user');
    let currentUser = null;

    if (storedUser) {
        try {
            currentUser = JSON.parse(storedUser);
            if (currentUser?.username) {
                const heading =
document.getElementById('welcome-heading');
                heading.textContent = `Welcome Back,
${currentUser.username}.`;
            }
        } catch (err) {
            console.error('Failed to parse stored user', err);
        }
    }

    // --- PART 1: 3D TILT EFFECT ---
    VanillaTilt.init(document.querySelectorAll(".card"), {
        max: 15,
        speed: 400,
        glare: true,
        "max-glare": 0.5
    });

    // --- PART 2: Load Dashboard Data and Update Circle ---
    async function loadDashboardData() {
        if (!currentUser || !currentUser.id) {
            console.log('No user found');
```

```
            return;
        }

        try {
            const response = await
fetch(`/api/dashboard?user_id=${currentUser.id}`);

            if (!response.ok) {
                const errorData = await response.json();
                throw new Error(errorData.error || 'Failed to load
dashboard data');
            }

            const data = await response.json();

            // Update Current Goal
            const currentGoalEl = document.getElementById('current-goal');
            if (currentGoalEl && data.current_goal) {
                const goalText = data.current_goal.charAt(0).toUpperCase()
+ data.current_goal.slice(1);
                currentGoalEl.textContent = goalText;
            }

            // Update Next Workout
            const nextWorkoutEl = document.getElementById('next-workout');
            if (nextWorkoutEl) {
                if (data.next_workout) {
                    nextWorkoutEl.textContent = data.next_workout;
                } else {
                    nextWorkoutEl.textContent = 'No workout scheduled';
                }
            }

            // Update Calories Left
            const caloriesLeftEl =
document.getElementById('calories-left-value');
            if (caloriesLeftEl) {
                caloriesLeftEl.textContent =
Math.round(data.calories_left).toLocaleString();
            }
```

```
            // Update Circle Progress
            const circle =
document.querySelector('.progress-ring__circle');
            if (circle) {
                const radius = circle.r.baseVal.value;
                const circumference = radius * 2 * Math.PI;

                // Set the circle line length
                circle.style.strokeDasharray = `${circumference}
${circumference}`;
                circle.style.strokeDashoffset = circumference;

                // Function to set progress
                function setProgress(percent) {
                    const offset = circumference - (percent / 100) *
circumference;
                    circle.style.strokeDashoffset = offset;
                }

                // Animate to the actual progress percentage
                setTimeout(() => {
                    setProgress(data.calorie_progress_percent || 0);
                }, 500);
            }

        } catch (error) {
            console.error('Error loading dashboard data:', error);
        }
    }

    // Load data when page loads
    document.addEventListener('DOMContentLoaded', () => {
        loadDashboardData();
    });
</script>
</body>
</html>

<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Login</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body>
   <div class="login-container">
    <div class="logo">ATHLETICORE<span class="ai-text">.AI</span></div>
    <p class="subtitle">Account Recovery</p>
    <form id="forgot-form" novalidate>
        <input type="email" placeholder="email" name="email" required
autocomplete="email">
        <button type="submit">Send Temporary Password</button>
        <p class="form-status" aria-live="polite"></p>
    </form>
    <div class="form-footer">
        <a href="/login">Back to Login</a>
    </div>
   </div>
   <script>
        const forgotForm = document.getElementById('forgot-form');
        const forgotStatus = forgotForm.querySelector('.form-status');

        forgotForm.addEventListener('submit', async (event) => {
            event.preventDefault();
            forgotStatus.textContent = 'Requesting reset...';

            const formData = new FormData(forgotForm);
            const payload = {
                email: formData.get('email').trim()
            };

            try {
```

```
                const response = await fetch('/api/auth/forgot-password',
{
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(payload)
                });

                const data = await response.json();

                if (!response.ok) {
                    forgotStatus.textContent = data.error || 'Unable to
reset password.';
                    forgotStatus.style.color = '#ff6b6b';
                    return;
                }

                forgotStatus.textContent = data.message || 'Temporary
password sent. Check your email.';
                forgotStatus.style.color = '#4ade80';
            } catch (error) {
                console.error(error);
                forgotStatus.textContent = 'Network error. Please try
again.';
                forgotStatus.style.color = '#ff6b6b';
            }
        });
    </script>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Login</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

```
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body>
   <div class="login-container">
    <div class="logo">ATHLETICORE<span class="ai-text">.AI</span></div>
    <form id="login-form" novalidate>
        <input type="text" placeholder="username or email"
name="identifier" required autocomplete="username">
        <input type="password" placeholder="password" name="password"
required autocomplete="current-password">
        <button type="submit">Login</button>
        <p class="form-status" aria-live="polite"></p>
    </form>
    <div class="form-footer">
    <a href="/register">Create Account</a>
    <a href="/forgot-password">Forgot Password?</a>
    </div>
   </div>
   <script>
        const loginForm = document.getElementById('login-form');
        const loginStatus = loginForm.querySelector('.form-status');

        loginForm.addEventListener('submit', async (event) => {
            event.preventDefault();
            loginStatus.textContent = 'Signing you in...';

            const formData = new FormData(loginForm);
            const payload = {
                identifier: formData.get('identifier').trim(),
                password: formData.get('password')
            };

            try {
                const response = await fetch('/api/auth/login', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
```

```
                    },
                    body: JSON.stringify(payload)
                });

                const data = await response.json();

                if (!response.ok) {
                    loginStatus.textContent = data.error || 'Login failed. Please try again.';
                    loginStatus.style.color = '#ff6b6b';
                    return;
                }

                loginStatus.textContent = 'Welcome back, redirecting...';
                loginStatus.style.color = '#4ade80';

                // Optionally store the user profile for later use
                localStorage.setItem('athleticore_user', JSON.stringify(data.user));

                setTimeout(() => {
                    window.location.href = '/dashboard';
                }, 800);
            } catch (error) {
                console.error(error);
                loginStatus.textContent = 'Network error. Please try again.';
                loginStatus.style.color = '#ff6b6b';
            }
        });
    </script>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Login</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
```

```
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body>
   <div class="login-container">
    <div class="logo">ATHLETICORE<span class="ai-text">.AI</span></div>
    <p class="subtitle">Create Account</p>
    <form id="register-form" novalidate>
        <input type="email" placeholder="email" name="email" required
autocomplete="email">
        <input type="text" placeholder="username" name="username" required
autocomplete="username">
        <input type="password" placeholder="password" name="password"
required autocomplete="new-password">
        <div class="form-row">
    <input type="number" placeholder="Age" name="age" required
style="width: 48%;">
    <select name="gender" required style="width: 48%;">
        <option value="" disabled selected>Gender</option>
        <option value="male">Male</option>
        <option value="female">Female</option>
    </select>
</div>

<div class="form-row">
    <input type="number" placeholder="Height (cm)" name="height" required
style="width: 48%;">
    <input type="number" placeholder="Weight (kg)" name="weight" required
style="width: 48%;">
</div>
        <button type="submit">Sign Up</button>
        <p class="form-status" aria-live="polite"></p>
    </form>
    <div class="form-footer">
        <a href="/login">Already have an account? Login</a>
    </div>
   </div>
```

```
    <script>
        const registerForm = document.getElementById('register-form');
        const registerStatus = registerForm.querySelector('.form-status');

        registerForm.addEventListener('submit', async (event) => {
            event.preventDefault();
            registerStatus.textContent = 'Creating your account...';

            const formData = new FormData(registerForm);
            const payload = {
                email: formData.get('email').trim(),
                username: formData.get('username').trim(),
                password: formData.get('password'),
                age: parseInt(formData.get('age')),
                gender: formData.get('gender'),
                height: parseInt(formData.get('height')),
                weight: parseInt(formData.get('weight'))
            };

            try {
                const response = await fetch('/api/auth/register', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(payload)
                });

                const data = await response.json();

                if (!response.ok) {
                    registerStatus.textContent = data.error ||
'Registration failed. Please try again.';
                    registerStatus.style.color = '#ff6b6b';
                    return;
                }

                registerStatus.textContent = 'Account created! Redirecting
to login...';
                registerStatus.style.color = '#4ade80';
```

```
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1200);
            } catch (error) {
                console.error(error);
                registerStatus.textContent = 'Network error. Please try
again.';
                registerStatus.style.color = '#ff6b6b';
            }
        });
    </script>
</body>
</html>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Account Settings - Athleticore.AI</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
    <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
            <li><a href="/settings" class="active">Settings</a></li>
        </ul>
```

```
    </nav>

    <main class="main-content">
        <div class="settings-container">
            <h1 class="settings-header">Account Settings</h1>

            <form id="settings-form" class="settings-form">
                <div class="settings-grid">
                    <!-- LEFT COLUMN: Account Security -->
                    <div class="settings-card">
                        <h2 class="settings-card-title">Account
Security</h2>

                        <div class="input-group">
                            <label for="username">Username</label>
                            <div class="input-with-icon">
                                <input type="text" id="username"
name="username" class="settings-input">
                                <span class="edit-icon">✎</span>
                            </div>
                        </div>

                        <div class="input-group">
                            <label for="email">Email</label>
                            <div class="input-with-icon">
                                <input type="email" id="email"
name="email" class="settings-input">
                                <span class="edit-icon">✎</span>
                            </div>
                        </div>

                        <div class="input-group">
                            <label for="password">Password</label>
                            <div class="input-with-icon">
                                <input type="password" id="password"
name="password" class="settings-input">
                                <span class="eye-icon"
id="toggle-password">👁️</span>
                            </div>
                        </div>
```

```
                        <div class="input-group"
id="confirm-password-group" style="display: none;">
                            <label for="confirm-password">Confirm
Password</label>
                            <input type="password" id="confirm-password"
name="confirm-password" class="settings-input">
                        </div>
                    </div>

                    <!-- RIGHT COLUMN: Body Stats -->
                    <div class="settings-card">
                        <h2 class="settings-card-title">Body Stats</h2>

                        <div class="input-group">
                            <label for="age">Age</label>
                            <input type="number" id="age" name="age"
class="settings-input" min="1" max="150">
                        </div>

                        <div class="input-group">
                            <label for="height">Height (cm)</label>
                            <input type="number" id="height" name="height"
class="settings-input" min="1" max="300" step="0.1">
                        </div>

                        <div class="input-group">
                            <label for="weight">Weight (kg)</label>
                            <input type="number" id="weight" name="weight"
class="settings-input" min="1" max="500" step="0.1">
                        </div>
                    </div>
                </div>

                <div class="settings-actions">
                    <button type="submit" class="save-btn
settings-save-btn">Save Changes</button>
                </div>
            </form>
```

```
            <div class="settings-actions-bottom">
                <button type="button" class="logout-btn"
id="logout-btn">Logout</button>
            </div>
        </div>
    </main>

    <!-- Toast Notification Container -->
    <div class="toast-container" id="toast-container"></div>

    <script>
        // Toast notification function
        function showToast(message, type = 'success') {
            const container = document.getElementById('toast-container');
            if (!container) return;

            const toast = document.createElement('div');
            toast.className = `toast ${type}`;

            const icon = type === 'success' ? '✓' : '✕';
            toast.innerHTML = `
                <span class="toast-icon">${icon}</span>
                <span class="toast-message">${message}</span>
            `;

            container.appendChild(toast);

            // Trigger animation
            setTimeout(() => {
                toast.classList.add('show');
            }, 10);

            // Remove toast after 3 seconds
            setTimeout(() => {
                toast.classList.remove('show');
                setTimeout(() => {
                    if (toast.parentNode) {
                        toast.parentNode.removeChild(toast);
                    }
                }, 300);
```

```
            }, 3000);
        }

        // Get user from localStorage
        function getUser() {
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                return null;
            }
            try {
                return JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                return null;
            }
        }

        // Load settings from backend
        async function loadSettings() {
            const user = getUser();
            if (!user || !user.id) {
                showToast('Please log in first', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            try {
                // Show loading state
                const usernameField = document.getElementById('username');
                const emailField = document.getElementById('email');
                const ageField = document.getElementById('age');
                const heightField = document.getElementById('height');
                const weightField = document.getElementById('weight');

                // Set placeholder to indicate loading
                usernameField.placeholder = 'Loading...';
                emailField.placeholder = 'Loading...';
```

```
                // Ensure user_id is a number
                const userId = parseInt(user.id);
                if (isNaN(userId)) {
                    throw new Error('Invalid user ID');
                }

                console.log('Loading settings for user_id:', userId);
                const response = await
fetch(`/api/settings?user_id=${userId}`);
                console.log('Response status:', response.status,
response.statusText);
                console.log('Response headers:',
response.headers.get('content-type'));

                // Get response text first (can only read once)
                const responseText = await response.text();

                // Check if response is OK
                if (!response.ok) {
                    let errorMessage = 'Failed to load settings';
                    try {
                        const errorData = JSON.parse(responseText);
                        errorMessage = errorData.error || errorMessage;
                    } catch (e) {
                        // If response is not JSON, use the text as error
message
                        errorMessage = responseText || errorMessage;
                    }
                    throw new Error(errorMessage);
                }

                // Parse JSON response
                let data;
                try {
                    data = JSON.parse(responseText);
                } catch (e) {
                    console.error('JSON parse error:', e);
                    console.error('Response text:', responseText);
                    throw new Error('Invalid response from server. Please
check the console for details.');
```

```
                }

                const settings = data.settings;

                if (!settings) {
                    throw new Error('No settings data received');
                }

                // Populate form fields with database values
                usernameField.value = settings.username || '';
                emailField.value = settings.email || '';
                ageField.value = settings.age || '';
                heightField.value = settings.height || '';
                weightField.value = settings.weight || '';

                // Reset placeholders
                usernameField.placeholder = '';
                emailField.placeholder = '';

                console.log('Settings loaded successfully:', settings);

            } catch (error) {
                console.error('Error loading settings:', error);
                showToast('Error loading settings: ' + error.message,
'error');

                // Reset placeholders on error
                const usernameField = document.getElementById('username');
                const emailField = document.getElementById('email');
                if (usernameField) usernameField.placeholder = '';
                if (emailField) emailField.placeholder = '';
            }
        }

        // Show Password Toggle
        const passwordInput = document.getElementById('password');
        const togglePassword = document.getElementById('toggle-password');
        const confirmPasswordGroup =
document.getElementById('confirm-password-group');
```

```
        const confirmPasswordInput =
document.getElementById('confirm-password');

        togglePassword.addEventListener('click', function() {
            if (passwordInput.type === 'password') {
                passwordInput.type = 'text';
                togglePassword.textContent = '👁';
            } else {
                passwordInput.type = 'password';
                togglePassword.textContent = '👁';
            }
        });

        // Show confirm password when password field is focused and has
value
        passwordInput.addEventListener('focus', function() {
            if (passwordInput.value.length > 0) {
                confirmPasswordGroup.style.display = 'block';
            }
        });

        passwordInput.addEventListener('input', function() {
            if (passwordInput.value.length > 0) {
                confirmPasswordGroup.style.display = 'block';
            } else {
                confirmPasswordGroup.style.display = 'none';
                confirmPasswordInput.value = '';
            }
        });

        // Save Logic
        const settingsForm = document.getElementById('settings-form');
        settingsForm.addEventListener('submit', async function(event) {
            event.preventDefault();

            const user = getUser();
            if (!user || !user.id) {
                showToast('Please log in first', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
```

```
                }, 1500);
                return;
            }

            // Get form values
            const username =
document.getElementById('username').value.trim();
            const email = document.getElementById('email').value.trim();
            const password = document.getElementById('password').value;
            const confirmPassword =
document.getElementById('confirm-password').value;
            const ageValue = document.getElementById('age').value.trim();
            const heightValue =
document.getElementById('height').value.trim();
            const weightValue =
document.getElementById('weight').value.trim();

            // Basic validation
            if (!username) {
                showToast('Username is required', 'error');
                return;
            }
            if (!email) {
                showToast('Email is required', 'error');
                return;
            }

            // Validate password if provided
            if (password) {
                if (!confirmPassword) {
                    showToast('Please confirm your password', 'error');
                    return;
                }
                if (password !== confirmPassword) {
                    showToast('Passwords do not match', 'error');
                    return;
                }
            }

            // Parse numeric values
```

```
            const age = ageValue ? parseInt(ageValue) : null;
            const height = heightValue ? parseInt(heightValue) : null;
            const weight = weightValue ? parseInt(weightValue) : null;

            // Validate numeric fields
            if (age !== null && (isNaN(age) || age <= 0)) {
                showToast('Age must be a positive number', 'error');
                return;
            }
            if (height !== null && (isNaN(height) || height <= 0)) {
                showToast('Height must be a positive number', 'error');
                return;
            }
            if (weight !== null && (isNaN(weight) || weight <= 0)) {
                showToast('Weight must be a positive number', 'error');
                return;
            }

            // Build payload with all fields
            const payload = {
                user_id: user.id,
                username: username,
                email: email
            };

            // Add optional fields only if they have values
            if (password) {
                payload.password = password;
                payload.confirm_password = confirmPassword;
            }
            if (age !== null) payload.age = age;
            if (height !== null) payload.height = height;
            if (weight !== null) payload.weight = weight;

            // Disable submit button
            const submitBtn =
settingsForm.querySelector('button[type="submit"]');
            submitBtn.disabled = true;
            submitBtn.textContent = 'Saving...';
```

```
            try {
                const response = await fetch('/api/settings', {
                    method: 'POST',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(payload)
                });

                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.error || 'Failed to update
settings');
                }

                // Update localStorage with new settings
                if (data.settings) {
                    user.username = data.settings.username;
                    user.email = data.settings.email;
                    user.age = data.settings.age;
                    user.height = data.settings.height;
                    user.weight = data.settings.weight;
                    localStorage.setItem('athleticore_user',
JSON.stringify(user));
                }

                // Clear password fields
                document.getElementById('password').value = '';
                document.getElementById('confirm-password').value = '';

document.getElementById('confirm-password-group').style.display = 'none';

                showToast('Settings saved successfully!', 'success');

            } catch (error) {
                console.error('Error saving settings:', error);
                showToast('Error: ' + error.message, 'error');
            } finally {
                submitBtn.disabled = false;
```

```
                submitBtn.textContent = 'Save Changes';
            }
        });

        // Logout Logic
        const logoutBtn = document.getElementById('logout-btn');
        logoutBtn.addEventListener('click', function() {
            if (confirm('Are you sure you want to logout?')) {
                localStorage.removeItem('athleticore_user');
                window.location.href = '/login';
            }
        });

        // Load settings when page loads
        let settingsLoaded = false;

        function initializeSettings() {
            if (settingsLoaded) return;
            settingsLoaded = true;

            // Wait a tiny bit to ensure all elements are ready
            setTimeout(() => {
                loadSettings();
            }, 100);
        }

        if (document.readyState === 'loading') {
            // DOM is still loading, wait for DOMContentLoaded
            document.addEventListener('DOMContentLoaded',
initializeSettings);
        } else {
            // DOM is already loaded
            initializeSettings();
        }
    </script>
</body>
</html>



<!DOCTYPE html>
```

```
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Login</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=O
rbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
   <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>
        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee" class="active">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
            <li><a href="/settings">Settings</a></li>
        </ul>
   </nav>
   <main class="main-content">
        <div class="chat-section">
            <h1>AI Coach</h1>
            <div class="chat-window"></div>
            <div class="chat-input-area">
                <input type="text" placeholder="Type a message..."
class="chat-input" autocomplete="off">
                <button type="button" class="send-btn">SEND</button>
            </div>
        </div>

        <section class="chat-form-section">
            <h2 class="section-title">TDEE Information</h2>
            <form class="stats-grid" id="tdee-info-form">
                <div class="input-group">
```

```
                    <label>Activity Level Factor</label>
                    <input type="number" name="activity_level"
id="activity-level-input" placeholder="1.2" step="0.1" min="1.0" max="2.5"
required>
                </div>
                <div class="input-group">
                    <label>TDEE (kcal)</label>
                    <input type="number" name="tdee_value"
id="tdee-value-input" placeholder="2000" min="0" required>
                </div>
                <div class="input-group">
                    <label>Goal Calories (kcal)</label>
                    <input type="number" name="goal_calories"
id="goal-calories-input" placeholder="2500" min="0" required>
                </div>
                <div class="input-group">
                    <label>Goal</label>
                    <select name="goal_type" id="goal-type-input"
required>
                        <option value="" disabled selected>Select</option>
                        <option value="cut">Cut</option>
                        <option value="bulk">Bulk</option>
                        <option value="maintain">Maintain</option>
                    </select>
                </div>
                <p class="form-status" aria-live="polite"></p>
                <button type="submit" class="save-btn"
id="save-tdee-btn">Save TDEE</button>
            </form>
        </section>
    </main>
<script>
    const chatWindow = document.querySelector(".chat-window");
    const chatInput = document.querySelector(".chat-input");
    const sendBtn = document.querySelector(".send-btn");
    const saveBtn = document.getElementById("save-tdee-btn");
    const statusText = document.querySelector(".form-status");
    const tdeeInfoForm = document.getElementById("tdee-info-form");

    const storedUser = localStorage.getItem("athleticore_user");
```

```
    const currentUser = storedUser ? JSON.parse(storedUser) : null;

    const chatHistory = [];
    let latestTdeeResult = null;

    function setStatus(message = "", color = "#fff") {
        if (!statusText) return;
        statusText.textContent = message;
        statusText.style.color = color;
    }

    function appendMessage(text, role = "bot") {
        const messageEl = document.createElement("div");
        messageEl.classList.add("message", role === "user" ? "user-msg" :
"bot-msg");
        messageEl.textContent = text;
        chatWindow.appendChild(messageEl);
        chatWindow.scrollTop = chatWindow.scrollHeight;
    }

    async function sendMessage() {
        const message = chatInput.value.trim();
        if (!message) {
            return;
        }

        appendMessage(message, "user");
        chatInput.value = "";
        sendBtn.disabled = true;
        chatHistory.push({ role: "user", content: message });

        try {
            const response = await fetch("/api/tdee/chat", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({
                    message,
                    username: currentUser?.username,
                    history: chatHistory
                })
```

```
            });

            if (!response.ok) {
                throw new Error("Network response was not ok");
            }

            const data = await response.json();
            const replyText = data.reply || "Sorry, I did not understand
that.";
            appendMessage(replyText);
            chatHistory.push({ role: "assistant", content: replyText });
            if (data.tdee_result) {
                latestTdeeResult = data.tdee_result;
                // Optionally populate form fields if AI provides values
                if (data.tdee_result.activity_level) {
                    document.getElementById("activity-level-input").value
= data.tdee_result.activity_level;
                }
                if (data.tdee_result.tdee_value) {
                    document.getElementById("tdee-value-input").value =
Math.round(data.tdee_result.tdee_value);
                }
                if (data.tdee_result.goal_calories) {
                    document.getElementById("goal-calories-input").value =
Math.round(data.tdee_result.goal_calories);
                }
                if (data.tdee_result.goal_type) {
                    document.getElementById("goal-type-input").value =
data.tdee_result.goal_type;
                }
            }
        } catch (error) {
            appendMessage("There was an error reaching the TDEE coach.
Please try again.");
            console.error(error);
        } finally {
            sendBtn.disabled = false;
            chatInput.focus();
        }
    }
```

```
    sendBtn.addEventListener("click", sendMessage);
    chatInput.addEventListener("keydown", (event) => {
        if (event.key === "Enter") {
            event.preventDefault();
            sendMessage();
        }
    });

    tdeeInfoForm.addEventListener("submit", async (event) => {
        event.preventDefault();
        if (!currentUser?.id) {
            setStatus("Please log in again.", "#ff6b6b");
            return;
        }

        const formData = new FormData(tdeeInfoForm);
        const tdeeData = Object.fromEntries(formData.entries());

        // Convert activity_level to string (it's stored as TEXT in
database)
        const activityLevelValue = tdeeData.activity_level ?
String(tdeeData.activity_level) : "";
        const tdeeValue = parseFloat(tdeeData.tdee_value);
        const goalCalories = parseFloat(tdeeData.goal_calories);

        // Calculate goal_offset from the difference
        const goalOffset = Math.round(goalCalories - tdeeValue);

        const payload = {
            user_id: currentUser.id,
            activity_level: activityLevelValue,
            tdee_value: tdeeValue,
            goal_type: tdeeData.goal_type,
            goal_offset: goalOffset,
            goal_calories: goalCalories
        };

        try {
            const response = await fetch("/api/tdee/profile", {
```

```
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify(payload)
            });

            if (!response.ok) {
                const errorData = await response.json();
                throw new Error(errorData.error || "Failed to save TDEE
profile");
            }

            setStatus("TDEE profile saved successfully!", "#4ade80");
        } catch (error) {
            console.error(error);
            setStatus(error.message, "#ff6b6b");
        }
    });

    async function loadSavedProfile() {
        if (!currentUser?.id) {
            return;
        }
        try {
            const response = await
fetch(`/api/tdee/profile/${currentUser.id}`);
            if (!response.ok) {
                return;
            }
            const data = await response.json();
            const profile = data.profile;
            if (!profile) {
                return;
            }

            // Populate TDEE info form
            if (profile.activity_level) {
                document.getElementById("activity-level-input").value =
profile.activity_level;
            }
            if (profile.tdee_value) {
```

```
                document.getElementById("tdee-value-input").value =
profile.tdee_value;
            }
            if (profile.goal_calories) {
                document.getElementById("goal-calories-input").value =
profile.goal_calories;
            }
            if (profile.goal_type) {
                document.getElementById("goal-type-input").value =
profile.goal_type;
            }

            latestTdeeResult = {
                tdee_value: profile.tdee_value,
                goal_type: profile.goal_type,
                goal_calories: profile.goal_calories,
                activity_level: profile.activity_level,
                ready_to_save: true
            };

            setStatus("Loaded your latest TDEE plan.", "#93c5fd");
        } catch (error) {
            console.error("Failed to load saved TDEE profile", error);
        }
    }

    loadSavedProfile();
</script>
</body>
</html>
```

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Athleticore Workout</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;600&family=Orbitron:wght@500;700&display=swap" rel="stylesheet">
    <link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/6.4.0/css/all.min.css">
    <link rel="stylesheet" href="../static/css/style.css">
</head>
<body class="dashboard-body">
    <nav class="top-nav">
        <div class="logo">ATHLETICORE<span
class="ai-text">.AI</span></div>

        <ul class="nav-links">
            <li><a href="/dashboard">Dashboard</a></li>
            <li><a href="/tdee">TDEE</a></li>
            <li><a href="/calories">Calories</a></li>
            <li><a href="/workout" class="active">Workout</a></li>
            <li><a href="/coach">Coach</a></li>
            <li><a href="/settings">Settings</a></li>
        </ul>
    </nav>
    <main class="main-content">
        <h2 class="section-title">Workout Log</h2>

        <div class="workout-log-container">
            <!-- Empty State (hidden by default) -->
            <div class="empty-state" id="empty-state" style="display:
none;">
                <p>You don't have any logs yet.</p>
            </div>

            <!-- Feed Container -->
            <div class="workout-feed-container" id="workout-feed">
                <!-- Log cards will be dynamically added here -->
```

```
            </div>
        </div>

        <!-- Fixed Create Log Button -->
        <a href="/create-log" class="create-btn">Create Log</a>
    </main>

    <!-- Toast Notification Container -->
    <div class="toast-container" id="toast-container"></div>

    <script>
        // Toast notification function
        function showToast(message, type = 'success') {
            const container = document.getElementById('toast-container');
            if (!container) return;

            const toast = document.createElement('div');
            toast.className = `toast ${type}`;

            const icon = type === 'success' ? '✓' : '✕';
            toast.innerHTML = `
                <span class="toast-icon">${icon}</span>
                <span class="toast-message">${message}</span>
            `;

            container.appendChild(toast);

            // Trigger animation
            setTimeout(() => {
                toast.classList.add('show');
            }, 10);

            // Remove toast after 3 seconds
            setTimeout(() => {
                toast.classList.remove('show');
                setTimeout(() => {
                    if (toast.parentNode) {
                        toast.parentNode.removeChild(toast);
                    }
                }, 300);
```

```
            }, 3000);
        }
    </script>
    <script>

        // Get current day of week
        function getCurrentDay() {
            const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
'Thursday', 'Friday', 'Saturday'];
            return days[new Date().getDay()];
        }

        // Helper function to format date to day of week
        function formatDateToDay(dateString) {
            const date = new Date(dateString + 'T00:00:00'); // Add time
to avoid timezone issues
            const days = ['Sunday', 'Monday', 'Tuesday', 'Wednesday',
'Thursday', 'Friday', 'Saturday'];
            return days[date.getDay()];
        }

        // DOM elements
        const emptyState = document.getElementById('empty-state');
        const workoutFeed = document.getElementById('workout-feed');
        const currentDay = getCurrentDay();

        // Store workouts data
        let workoutsData = [];

        // Function to create a log card from workout data
        function createLogCard(workoutItem, isToday = false) {
            const card = document.createElement('div');
            card.className = 'log-card';

            if (isToday) {
                card.classList.add('today-highlight');
            }

            const workout = workoutItem.workout;
            const day = formatDateToDay(workout.date);
```

```
            const dayAbbr = day.substring(0, 3).toUpperCase();

            // Store full workout data as JSON in data attribute
            const workoutData = JSON.stringify(workoutItem);
            card.setAttribute('data-workout', workoutData);
            card.setAttribute('data-workout-id', workout.id);

            card.innerHTML = `
                <div class="day-badge">${dayAbbr}</div>
                <div class="log-summary">${workout.workout_name}</div>
                <button class="delete-log-btn" onclick="deleteLog(this)">
                    <i class="fas fa-times"></i>
                </button>
            `;

            // Make card clickable (except delete button)
            card.style.cursor = 'pointer';
            card.addEventListener('click', (event) => {
                if (event.target.closest('.delete-log-btn')) return;
                const workoutDataStr = card.getAttribute('data-workout');
                if (workoutDataStr) {
                    try {
                        const workoutData = JSON.parse(workoutDataStr);
                        openDetailsModal(workoutData);
                    } catch (e) {
                        console.error('Failed to parse workout data:', e);
                    }
                }
            });

            return card;
        }

        // Function to delete a log
        async function deleteLog(btn) {
            const card = btn.closest('.log-card');
            if (!card) return;

            const workoutId = card.getAttribute('data-workout-id');
            if (!workoutId) return;
```

```
            // Get user from localStorage
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                showToast('Please log in first', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            let user;
            try {
                user = JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                showToast('Session expired. Please log in again.',
'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            if (!user || !user.id) {
                showToast('User information not found. Please log in
again.', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            // Apply fade-out animation immediately
            card.style.opacity = '0';
            card.style.transform = 'translateX(100px)';
            card.style.transition = 'opacity 0.3s ease, transform 0.3s
ease';

            // Call API to delete
```

```
            try {
                const response = await
fetch(`/api/workouts/${workoutId}?user_id=${user.id}`, {
                    method: 'DELETE',
                    headers: {
                        'Content-Type': 'application/json'
                    }
                });

                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.error || 'Failed to delete
workout');
                }

                // Show success message
                showToast('Workout deleted successfully!', 'success');

                // Remove from DOM after animation
                setTimeout(() => {
                    card.remove();

                    // Refresh the entire page to reload workouts
                    setTimeout(() => {
                        window.location.reload();
                    }, 800);
                }, 300);
            } catch (error) {
                console.error('Error deleting workout:', error);
                showToast('Error: ' + error.message, 'error');
                // Revert animation on error
                card.style.opacity = '1';
                card.style.transform = 'translateX(0)';
            }
        }

        // Function to render logs
        function renderLogs() {
            // Clear existing content
```

```
            workoutFeed.innerHTML = '';

            // If no logs, show empty state
            if (workoutsData.length === 0) {
                emptyState.style.display = 'block';
                workoutFeed.style.display = 'none';
                return;
            }

            // Hide empty state
            emptyState.style.display = 'none';
            workoutFeed.style.display = 'block';

            // Sort workouts: today's workout first, then by date (newest
first)
            const sortedWorkouts = [...workoutsData].sort((a, b) => {
                const aDay = formatDateToDay(a.workout.date);
                const bDay = formatDateToDay(b.workout.date);

                if (aDay === currentDay) return -1;
                if (bDay === currentDay) return 1;

                // Otherwise sort by date (newest first)
                return new Date(b.workout.date) - new
Date(a.workout.date);
            });

            // Render each workout
            sortedWorkouts.forEach(workoutItem => {
                const workoutDay =
formatDateToDay(workoutItem.workout.date);
                const isToday = workoutDay === currentDay;
                const card = createLogCard(workoutItem, isToday);
                workoutFeed.appendChild(card);
            });
        }

        // Function to open details modal
        function openDetailsModal(workoutItem) {
            const modal = document.getElementById('details-modal');
```

```
            if (!modal) return;

            const workout = workoutItem.workout;
            const exercises = workoutItem.exercises || [];

            // Store workout ID in modal dataset
            modal.dataset.workoutId = workout.id;

            // Format date to day of week
            const day = formatDateToDay(workout.date);

            // Populate workout-level fields
            const workoutNameInput =
document.getElementById('modal-workout-name');
            const workoutDayEl = document.getElementById('modal-day');
            const workoutNotesTextarea =
document.getElementById('modal-workout-notes');

            if (workoutNameInput) workoutNameInput.value =
workout.workout_name || '';
            if (workoutDayEl) workoutDayEl.textContent = day || 'N/A';
            if (workoutNotesTextarea) workoutNotesTextarea.value =
workout.notes || '';

            // Clear and populate exercises
            const exercisesContainer =
document.getElementById('modal-exercises-list');
            if (exercisesContainer) {
                exercisesContainer.innerHTML = '';

                exercises.forEach((exercise, index) => {
                    const exerciseBlock = createExerciseBlock(exercise,
index);
                    exercisesContainer.appendChild(exerciseBlock);
                });
            }

            // Show modal
            modal.style.display = 'flex';
        }
```

```
        // Function to create an editable exercise block
        function createExerciseBlock(exercise, index) {
            const block = document.createElement('div');
            block.className = 'exercise-modal-block';
            block.setAttribute('data-exercise-index', index);
            if (exercise.id) {
                block.setAttribute('data-exercise-id', exercise.id);
            }

            block.innerHTML = `
                <div class="exercise-header-editable">
                    <input type="text" class="exercise-name-input"
data-field="exercise-name" value="${exercise.exercise_name || ''}"
placeholder="Exercise Name">
                    <i class="fas fa-pen edit-icon"></i>
                </div>

                <div class="exercise-stats-row">
                    <div class="stat-group">
                        <label class="stat-label">Reps</label>
                        <div class="old-new-inputs">
                            <input type="number" class="stat-input
old-value" data-field="reps-old" value="" placeholder="Old" min="0">
                            <input type="number" class="stat-input
new-value" data-field="reps-new" value="${exercise.reps || ''}"
placeholder="New" min="0">
                        </div>
                    </div>

                    <div class="stat-group">
                        <label class="stat-label">Sets</label>
                        <div class="old-new-inputs">
                            <input type="number" class="stat-input
old-value" data-field="sets-old" value="" placeholder="Old" min="0">
                            <input type="number" class="stat-input
new-value" data-field="sets-new" value="${exercise.sets || ''}"
placeholder="New" min="0">
                        </div>
                    </div>
```

```
                </div>

                <div class="exercise-weight-row">
                    <div class="weight-group">
                        <label class="weight-label">Weight (KG)</label>
                        <div class="weight-input-wrapper">
                            <input type="number" class="weight-input"
data-field="weight" value="${exercise.weight_kg || ''}" placeholder="0"
min="0" step="0.5">
                            <i class="fas fa-pen edit-icon"></i>
                        </div>
                    </div>

                    <div class="weight-group">
                        <label class="weight-label">Previous Weight
(KG)</label>
                        <div class="weight-input-wrapper">
                            <input type="number" class="weight-input"
data-field="prev-weight" value="${exercise.previous_weight || ''}"
placeholder="0" min="0" step="0.5">
                            <i class="fas fa-pen edit-icon"></i>
                        </div>
                    </div>
                </div>

                <div class="exercise-notes-group">
                    <label class="exercise-notes-label">Notes</label>
                    <textarea class="exercise-notes-textarea"
data-field="exercise-notes" placeholder="Add notes about this
exercise...">${exercise.notes || ''}</textarea>
                </div>
            `;

            return block;
        }

        // Function to close details modal
        function closeDetailsModal() {
            const modal = document.getElementById('details-modal');
            if (modal) {
```

```
                modal.style.display = 'none';
            }
        }

        // Close modal when clicking outside (on overlay)
        document.addEventListener('DOMContentLoaded', () => {
            const modal = document.getElementById('details-modal');
            const overlay = modal ? modal.querySelector('.modal-overlay')
: null;
            if (overlay) {
                overlay.addEventListener('click', () => {
                    closeDetailsModal();
                });
            }
        });

        // Function to load workouts from API
        async function loadWorkouts() {
            // Get user from localStorage
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                console.log('No user found, showing empty state');
                emptyState.style.display = 'block';
                workoutFeed.style.display = 'none';
                return;
            }

            let user;
            try {
                user = JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                emptyState.style.display = 'block';
                workoutFeed.style.display = 'none';
                return;
            }

            if (!user || !user.id) {
                console.log('No user ID found');
                emptyState.style.display = 'block';
```

```
                workoutFeed.style.display = 'none';
                return;
            }

            try {
                const response = await
fetch(`/api/workouts?user_id=${user.id}`);
                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.error || 'Failed to load
workouts');
                }

                workoutsData = data.workouts || [];
                renderLogs();
            } catch (error) {
                console.error('Error loading workouts:', error);
                emptyState.style.display = 'block';
                workoutFeed.style.display = 'none';
            }
        }

        // Function to save workout changes
        async function saveWorkoutChanges() {
            const modal = document.getElementById('details-modal');
            if (!modal) return;

            const workoutId = modal.dataset.workoutId;
            if (!workoutId) {
                showToast('Workout ID not found', 'error');
                return;
            }

            // Get user from localStorage
            const storedUser = localStorage.getItem('athleticore_user');
            if (!storedUser) {
                showToast('Please log in first', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
```

```
                }, 1500);
                return;
            }

            let user;
            try {
                user = JSON.parse(storedUser);
            } catch (err) {
                console.error('Failed to parse stored user', err);
                showToast('Session expired. Please log in again.',
'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            if (!user || !user.id) {
                showToast('User information not found. Please log in
again.', 'error');
                setTimeout(() => {
                    window.location.href = '/login';
                }, 1500);
                return;
            }

            // Collect workout-level fields
            const workoutNameInput =
document.getElementById('modal-workout-name');
            const workoutNotesTextarea =
document.getElementById('modal-workout-notes');

            const workout_name = workoutNameInput ?
workoutNameInput.value.trim() : '';
            const notes = workoutNotesTextarea ?
workoutNotesTextarea.value.trim() : '';

            if (!workout_name) {
                showToast('Workout name is required', 'error');
                return;
```

```
            }

            // Collect exercises
            const exerciseBlocks =
document.querySelectorAll('#modal-exercises-list .exercise-modal-block');
            const exercises = [];

            exerciseBlocks.forEach((block, index) => {
                const exerciseId = block.dataset.exerciseId || null;

                const nameInput =
block.querySelector('[data-field="exercise-name"]');
                const repsNewInput =
block.querySelector('[data-field="reps-new"]');
                const setsNewInput =
block.querySelector('[data-field="sets-new"]');
                const weightInput =
block.querySelector('[data-field="weight"]');
                const prevWeightInput =
block.querySelector('[data-field="prev-weight"]');
                const notesInput =
block.querySelector('[data-field="exercise-notes"]');

                const exercise_name = nameInput ? nameInput.value.trim() :
'';
                if (!exercise_name) {
                    return; // Skip exercises without names
                }

                exercises.push({
                    id: exerciseId ? parseInt(exerciseId) : null,
                    exercise_name: exercise_name,
                    reps: repsNewInput ? parseInt(repsNewInput.value || 0)
: 0,
                    sets: setsNewInput ? parseInt(setsNewInput.value || 0)
: 0,
                    weight_kg: weightInput ? parseFloat(weightInput.value
|| 0) : 0,
                    previous_weight: prevWeightInput ?
parseFloat(prevWeightInput.value || 0) : 0,
```

```
                    notes: notesInput ? notesInput.value.trim() : '',
                    order_index: index
                });
            });

            if (exercises.length === 0) {
                showToast('At least one exercise is required', 'error');
                return;
            }

            const payload = {
                user_id: user.id,
                workout_name: workout_name,
                notes: notes,
                exercises: exercises
            };

            try {
                const response = await fetch(`/api/workouts/${workoutId}`,
{
                    method: 'PATCH',
                    headers: {
                        'Content-Type': 'application/json'
                    },
                    body: JSON.stringify(payload)
                });

                const data = await response.json();

                if (!response.ok) {
                    throw new Error(data.error || 'Failed to save
workout');
                }

                showToast('Workout updated successfully!', 'success');
                closeDetailsModal();

                // Refresh workouts list
                setTimeout(() => {
                    window.location.reload();
```

```
                }, 800);
            } catch (error) {
                console.error('Error saving workout:', error);
                showToast('Error: ' + error.message, 'error');
            }
        }

        // Initialize when page loads
        document.addEventListener('DOMContentLoaded', () => {
            loadWorkouts();

            // Wire up Save button
            const saveBtn = document.getElementById('modal-save-btn');
            if (saveBtn) {
                saveBtn.addEventListener('click', saveWorkoutChanges);
            }
        });
    </script>

    <!-- Details Modal -->
    <div id="details-modal" class="details-modal" style="display: none;">
        <div class="modal-overlay"></div>
        <div class="modal-content">
            <div class="modal-header">
                <div class="modal-header-left">
                    <div class="title-edit-group">
                        <input type="text" id="modal-workout-name"
class="modal-workout-name-input" placeholder="Workout Name">
                        <i class="fas fa-pen title-edit-icon"></i>
                    </div>
                </div>
                <div class="modal-header-right">
                    <span class="modal-day-badge" id="modal-day">-</span>
                    <button class="modal-close-btn"
onclick="closeDetailsModal()">
                        <i class="fas fa-times"></i>
                    </button>
                </div>
            </div>
            <div class="modal-body">
```

```
                <div class="modal-workout-notes-section">
                    <div class="modal-notes-header">
                        <label class="modal-notes-label">Notes</label>
                        <i class="fas fa-pen edit-icon"></i>
                    </div>
                    <textarea id="modal-workout-notes"
class="modal-workout-notes-textarea" placeholder="Add workout
notes..."></textarea>
                </div>

                <div class="modal-exercises-section">
                    <h3 class="modal-exercises-title">Exercises</h3>
                    <div id="modal-exercises-list"
class="modal-exercises-list">
                        <!-- Exercise blocks will be dynamically added
here -->
                    </div>
                </div>
            </div>
            <div class="modal-footer">
                <button class="modal-save-btn" id="modal-save-btn">Save
Changes</button>
            </div>
        </div>
    </div>
</body>
</html>


{
    "liveServer.settings.port": 5501
}
```